COMPLETE HANDBOOK

Data Science & AI Handbook

Course handbooks: Building Blocks (Class 1 onwards) · Building Blocks of ML & Applied AI · Python · Data Visualization & Analytics · ML & Text Mining · ML Ops · Generative AI · Power BI · Tableau

Learning sessions: Advanced Excel · SQL · Power BI · Tableau · Python · Machine Learning · AI & Agents · Tools · Interview Q&A · Reverse Engineering · Automation · Storytelling

AnalytixLabs · Advanced Certification in Data Science & AI (IIT), June 2026 batch

Interactive version (every tab clickable):
https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw

Contents

Course handbooks

01 · Building Blocks Essentials

- Overview
- Building Blocks · Class 1
- Building Blocks · Class 2
- Building Blocks · Class 3
- Building Blocks · Class 4
- Building Blocks · Class 5
- Building Blocks · Class 6
- Building Blocks · Class 7
- Building Blocks · Class 8
- What's new 2025–26
- Practice & case study
- Interview Q&A
- Resources

02 · Building Blocks of ML & Applied AI

- Overview
- Building Blocks ML · Class 1
- Building Blocks ML · Class 2
- Building Blocks ML · Class 3
- Building Blocks ML · Class 4
- Building Blocks ML · Class 5
- Building Blocks ML · Class 6
- Building Blocks ML · Class 7
- Building Blocks ML · Class 8
- Building Blocks ML · Class 9
- Building Blocks ML · Class 10
- Interview Q&A
- Resources

03 · Python for Data Science

- Overview
- Core Python
- Data structures
- Functions & files
- NumPy
- pandas
- Visualisation
- Dates, outliers, SQL

Statistics in Python
Regression models
Interview Q&A
Resources
04 · Data Visualization & Analytics
Overview
Excel: formulas
Excel: pivots & charts
SQL: core
SQL: advanced
AI for analytics
Interview Q&A
Resources
05 · ML, Text Mining & Intro to AI
Overview
Class 1–2: Hypothesis testing
Class 3: Intro to ML
Class 4: Lifecycle & models
Class 5: Gradient descent
Class 6: Pre-processing & fit
Class 7: KNN & metrics
Coming next
Interview Q&A
Resources
06 · ML Ops & Model Deployment
Overview
MLOps concepts
Git
Conda & Flask
Banking app walkthrough
Deploy: Heroku & Azure
Interview Q&A
Resources
07 · Generative AI
Overview
How LLMs work
Prompting techniques
Applied use cases
Image generation
Prompt library

Interview Q&A
Resources
08 · Power BI
Overview
Components & views
Power Query
Data modelling
DAX
Visuals & interactivity
Power BI Service
Practice project
Interview Q&A
Resources
09 · Tableau
Overview
Basics & Marks
Table calculations
Filters, groups & sets
Calculations & parameters
LOD expressions
Dashboards & stories
Optimisation & best practice
Interview Q&A
Resources

Learning sessions

S1 · Advanced Excel: Basic to Advanced
Roadmap
Level 1 · Foundations
Level 2 · Core functions
Level 3 · Logic & lookups
Level 4 · Data tools
Level 5 · Analysis
Level 6 · Modern Excel
Level 7 · Power tools
Level 8 · Charts & dashboards
Level 9 · Automation & AI
Errors & auditing
Interview Q&A
Resources

S2 · SQL: Basic to Advanced
Roadmap
1 · Foundations
2 · Summaries
3 · Combining
4 · Advanced querying
5 · Building
6 · Performance
7 · Analytics patterns
Dialects & new features
Interview Q&A
Resources
S3 · Power BI: Basic to Advanced
Roadmap
1 · First report
2 · Power Query
3 · Data modelling
4 · DAX
5 · Visuals & interactivity
6 · Service, Fabric & security
7 · Copilot, performance & DevOps
Interview Q&A
Resources
S4 · Tableau: Basic to Advanced
Roadmap
1 · Start
2 · Marks & filters
3 · Calculations
4 · Table calcs & LOD
5 · Dashboards
6 · Publish, performance & AI
Interview Q&A
Resources
S5 · Python: Basic to Advanced (for non-tech learners)
Start here
Level 1 · Basics
Level 2 · Data structures
Level 3 · Functions, files, errors
Level 4 · pandas
Level 5 · Charts

Level 6 · Stats & ML
Level 7 · Automation
Level 8 · Advanced
Interview Q&A
Resources
S6 · Machine Learning: Basic to Advanced (for non-tech learners)
What ML is
The ML workflow
Algorithms explained
Is the model good?
Advanced topics
Use cases by industry
Interview Q&A
Resources
S7 · AI, Generative AI & Agentic AI (for non-tech learners)
The big picture
How LLMs work
RAG & fine-tuning
Agentic AI
Using AI at work
Interview Q&A
Resources
S8 · Tools for Data Science & AI
Tool map
AI assistants
Build, code & research
Core data stack
Safe & smart use
Resources
S9 · Interview Questions & Answers
How interviews work
HR & behavioural
Excel
SQL
Statistics
Python
Power BI & Tableau
ML & GenAI
Case & guesstimates
Resources

S10 · Reverse Engineering for Non-Tech Learners

- The method
- Excel workbooks
- SQL queries
- Dashboards
- Notebooks & ML models
- Metrics, apps & AI
- Resources

S11 · Automation on Data

- Why & how
- Excel
- SQL
- Python
- Power BI & Tableau
- ML & AI
- End-to-end example
- Resources

S12 · Data Storytelling, Portfolio & Career

- Data storytelling
- Portfolio
- Git & GitHub basics
- Career roadmap
- Resources

COURSE HANDBOOK 01

Building Blocks Essentials

Class 1 to Class 8: Excel, RDBMS, DDL, SELECT & joins, SQL Server setup, Statistics parts 1–3

Building Blocks Essentials · 8 classes · modules: Excel Pre-Learning, SQL Pre-Learning, Building Blocks Assessment

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#m1**

Overview

Building Blocks Essentials is the foundation course of the programme. It teaches the three skills every data role uses daily: **Excel** for quick analysis, **SQL** for pulling data out of databases, and **statistics** for reading numbers correctly. Every later course (Python, Data Visualization, ML) assumes you know this material.



Where Building Blocks fits: Excel and SQL power the "collect, clean, explore" steps; statistics powers "explore" and "model".

Class	Title	What it covers
Building Blocks · Class 1	Microsoft Excel and basic functionality	Excel environment, terminology, key features, shortcuts; Number, Date & Time, Text and Aggregate functions; formatting and custom number formats; conditional formatting (incl. custom rules); freeze panes; sorting and filters; basic charts
Building Blocks · Class 2	Introduction to RDBMS	Server, database, DBMS, RDBMS; database design and structure; key RDBMS vendors; ER diagrams; keys and constraints; relationships and metadata; introduction to SQL; MS SQL Server and SSMS
Building Blocks · Class 3	Data Definition Language	Create, drop and select databases; create and drop tables; data types; inserting values; UPDATE; ALTER; primary and foreign keys with constraints; TRUNCATE; best practices
Building Blocks · Class 4	Data extraction with SELECT	Subsetting, sorting, filtering, removing duplicates; operators; order of operations; text, date, number and aggregate functions; joins (LEFT, RIGHT, INNER, OUTER, SELF, CROSS, NATURAL); appending; set operators; subqueries; business use cases
Building Blocks · Class 5	SQL recitation: SQL Server setup	Installing SQL Server and SSMS, the query editor, importing flat files
Building Blocks · Class 6	Statistics Part 1/4	What statistics is, data types and scales, population vs sample, central tendency

Class	Title	What it covers
Building Blocks · Class 7	Statistics Part 2/4	Dispersion, percentiles, shape, box plots, z-scores, correlation
Building Blocks · Class 8	Statistics Part 3/4	Probability, distributions, sampling, Central Limit Theorem

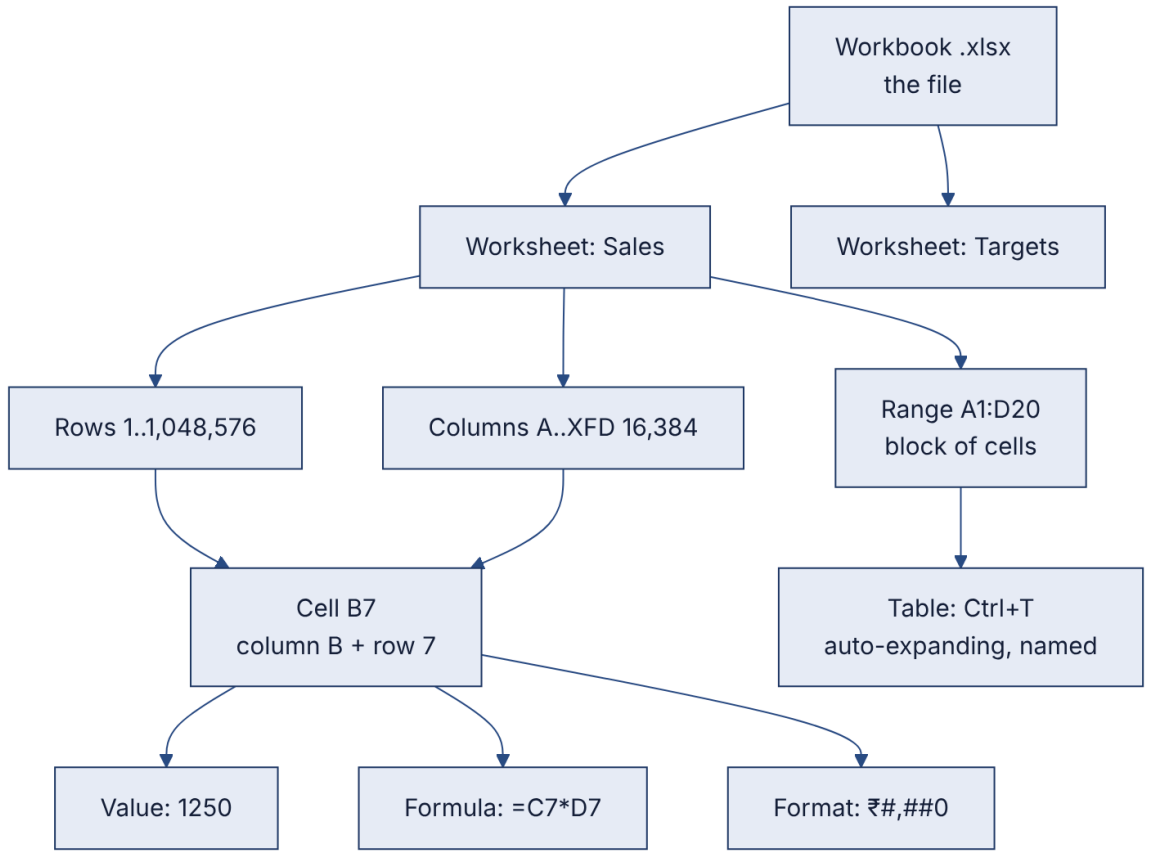
The course page labels Classes 6–8 as "pre-learning sessions on STATISTICS (Parts 1–3 of 4) that should be covered before statistics in Python for Data Science". Part 4/4 is taught in **Building Blocks of ML & Applied AI · Class 5**. The class titles do not list sub-topics, so the split of statistics topics across Parts 1–4 below follows the standard order of an introductory statistics course.

Tip Also see **What's new 2025–26** for the Excel and SQL Server changes released after these classes were recorded, and the **Advanced Excel** and **SQL** learning sessions for the full basic-to-advanced path.

Building Blocks · Class 1

Building Blocks · Class 1: Introduction to Microsoft Excel and basic functionality

1.1 The Excel environment



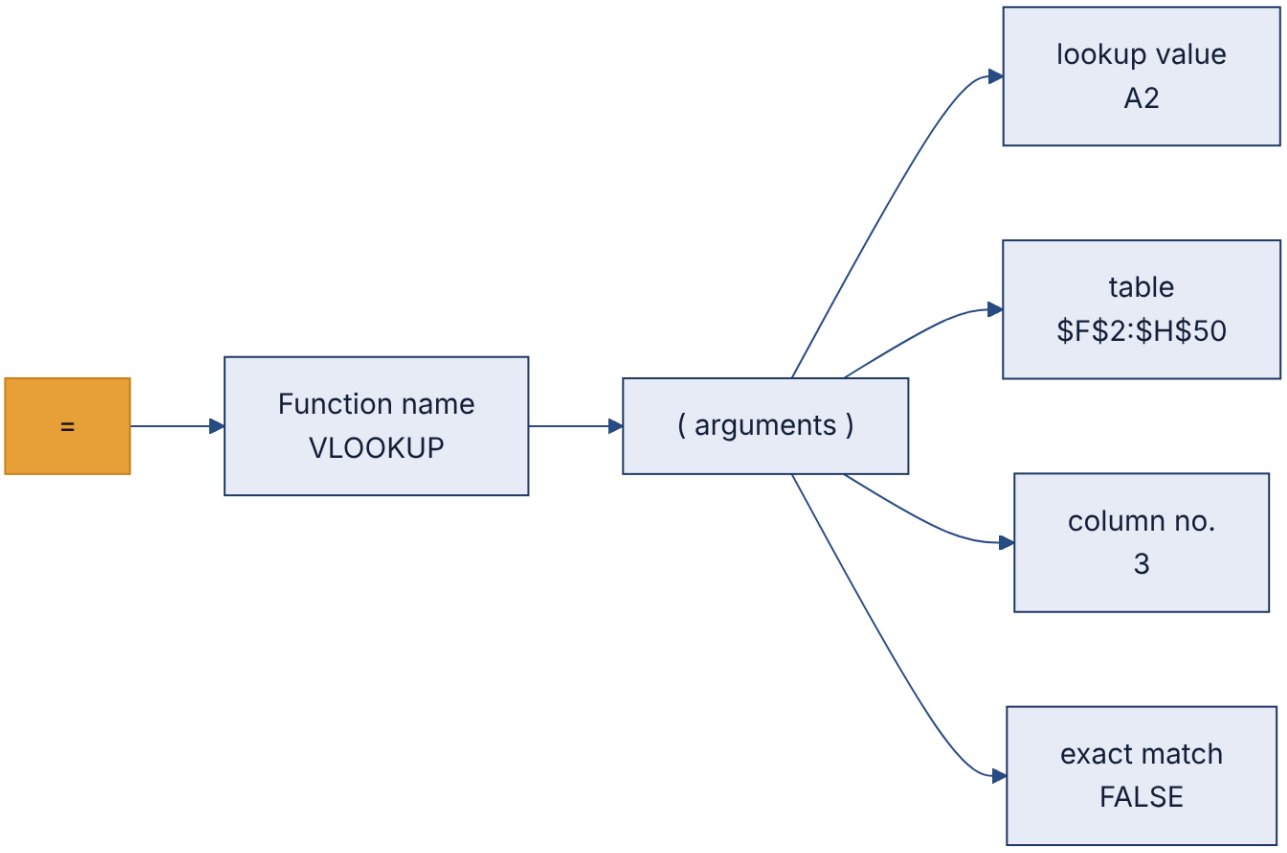
Workbook → worksheets → rows × columns → cells. A modern worksheet has 1,048,576 rows and 16,384 columns (A to XFD).

Part of the screen	What it does
Ribbon	Tabs (Home, Insert, Page Layout, Formulas, Data, Review, View) holding groups of commands
Quick Access Toolbar	Your favourite commands (Save, Undo); right-click any command → Add to QAT
Name Box	Shows the active cell address; type a name or address and press Enter to jump there
Formula Bar	Shows the real content of a cell: the formula, not just the result
Status bar	Select numbers to see instant Sum, Average and Count; right-click it to add Min/Max
Sheet tabs	Switch, rename (double-click), colour, move or copy sheets
File → Options	Calculation mode, autosave, language, add-ins (Analysis ToolPak)

1.2 Terminology

Workbook	The file (.xlsx; .xlsm if it has macros; .xlsb binary; .csv plain text, one sheet, no formatting)
Worksheet	One grid inside the workbook
Cell / address	Intersection of a column and row, e.g. C7
Range	A block of cells, e.g. A1:D20; non-adjacent ranges use a comma: A1:A5,C1:C5

Formula	Starts with `=` ; can use operators, references and functions: `=B2*C2`
Function	A built-in formula with a name and arguments: `=SUM(B2:B10)`
Argument	An input to a function, separated by commas
Relative / absolute reference	A1 shifts when copied; $A$1 stays fixed; A$1 / $A1 are mixed. Press `F4` to cycle
Table	A range converted with `Ctrl` + `T` : banded, auto-expanding, uses names like `Sales[Amount]`



Anatomy of a formula: equals sign, function name, then arguments in brackets.

1.3 Key functionalities

- **AutoFill and Flash Fill**: drag the fill handle to continue a series (Jan, Feb...); `Ctrl` + `E` Flash Fill learns a pattern from one example (first names from full names).
- **Paste Special** (`Ctrl` + `Alt` + `V`): values only, formats, transpose, multiply a range by a number.
- **Find & Replace** (`Ctrl` + `H`), Go To Special (blanks, formulas, errors).
- **Text to Columns** (Data tab): split "Gurugram, Haryana" into two columns.
- **Remove Duplicates**, **Data Validation** (drop-down lists), **Protect Sheet**.

1.4 Shortcuts

Navigate and select	
`Ctrl` + `Arrow`	Jump to the last filled cell in a direction
`Ctrl` + `Shift` + `Arrow`	Select to the edge of data
`Ctrl` + `Home` / `End`	Go to A1 / last used cell
`Ctrl` + `Space` / `Shift` + `Space`	Select column / row
`Ctrl` + `PgUp/PgDn`	Previous / next sheet
`Ctrl` + `G` or `F5`	Go To

Edit and format	
`F2`	Edit cell
`F4`	Toggle $ references / repeat last action
`Alt` + `=`	AutoSum
`Ctrl` + `1`	Format Cells
`Ctrl` + `Shift` + `$ % #`	Currency / percent / date format
`Ctrl` + `;` / `Ctrl` + `Shift` + `;`	Today's date / current time
`Alt` + `Enter`	New line inside a cell
`Ctrl` + `Shift` + `L`	Toggle filters
`Ctrl` + `` ` ``	Show formulas instead of results

1.5 Functions: four families with real examples

Example sheet: an electronics retailer's orders. Columns: A Order ID, B Order Date, C Customer, D City, E Product, F Qty, G Unit Price, H Amount.

Family	Function	Real-world formula	Result / meaning
Number	ROUND	`=ROUND(H2*18%,2)`	GST at 18%, rounded to paise
	ROUNDUP / ROUNDDOWN	`=ROUNDUP(F2/12,0)`	Cartons needed when 12 fit per carton
	INT / TRUNC	`=INT(H2/1000)`	Whole thousands
	MOD	`=MOD(A2,2)=0`	TRUE for even order IDs (alternate-row checks)
	ABS	`=ABS(Actual-Target)`	Size of a miss, ignoring sign
	CEILING / FLOOR, RAND, RANDBETWEEN	`=CEILING(G2,10)`	Price rounded up to the next ₹10

Family	Function	Real-world formula	Result / meaning
Date & Time	TODAY / NOW	`=TODAY()-B2`	Days since the order
	YEAR / MONTH / DAY	`=YEAR(B2)`	Order year for grouping
	TEXT	`=TEXT(B2,"mmm-yy")`	"Oct-26" month label
	DATE	`=DATE(2026,4,1)`	Start of the Indian financial year
	EOMONTH / EDATE	`=EOMONTH(B2,0)`	Month-end date (billing cut-off); `EDATE(B2,3)` = 3 months later
	DATEDIF	`=DATEDIF(DOB,TODAY(),"Y")`	Age in completed years (hidden function, still works)
	WEEKDAY / NETWORKDAYS / WORKDAY	`=NETWORKDAYS(B2,DeliveredOn)`	Working days to deliver (excludes weekends, optional holiday list)
Text	LEFT / RIGHT / MID	`=LEFT(A2,3)`	Region prefix "NOR" from "NOR-1042"
	LEN	`=LEN(Phone)=10`	Valid 10-digit mobile number?
	FIND / SEARCH	`=MID(Email,FIND("@",Email)+1,50)`	Email domain
	TRIM / CLEAN	`=TRIM(C2)`	Removes extra spaces from imported names
	UPPER / LOWER / PROPER	`=PROPER(C2)`	"ravi kumar" → "Ravi Kumar"
	CONCAT / & / TEXTJOIN	`=TEXTJOIN(", ",TRUE,D2:D6)`	One cell listing cities, skipping blanks
	SUBSTITUTE / REPLACE	`=SUBSTITUTE(G2,"₹","")*1`	Strip a symbol and convert to a number
	VALUE / TEXT	`=VALUE("1,250")`	Text that looks like a number → a real number

Family	Function	Real-world formula	Result / meaning
Aggregate	SUM / AVERAGE	`=SUM(H2:H500)`	Total revenue
	COUNT / COUNTA / COUNTBLANK	`=COUNTBLANK(D2:D500)`	Orders missing a city
	MIN / MAX / SMALL / LARGE	`=LARGE(H2:H500,3)`	Third-largest order
	SUMIF / SUMIFS	`=SUMIFS(H:H,D:D,"Pune",B:B,">="&DATE(2026,4,1))`	Pune revenue this financial year
	COUNTIF / COUNTIFS	`=COUNTIFS(D:D,"Delhi",H:H,">10000")`	Big Delhi orders
	AVERAGEIF(S), MEDIAN, MODE	`=AVERAGEIFS(H:H,E:E,"Laptop")`	Average laptop order value

1.6 Formatting and custom number formatting

Formatting changes how a value **looks**, never the value itself. 0.18 formatted as % shows 18% but is still 0.18 in formulas.

Custom format (Ctrl+1 → Custom)	Value	Shows as	Use
`#,##0`	1250000	1,250,000	Thousands separators
`[>=10000000]##\,##\,##\,##0;[>=100000]##\,##\,##0;##,##0`	1250000	12,50,000	Indian lakh/crore grouping
`0.0,,"M"`	1250000	1.3M	Millions in dashboards (each comma divides by 1,000)
`"₹"#,##0.00`	99.5	₹99.50	Currency
`000`	7	007	Fixed-length IDs
`dd mmm' yyyy`	09-05-1986	09 May' 1986	Readable dates
`0 "kg"`	80	80 kg	Units while staying numeric
`[Green]▲0%;[Red]▼0%;0%`	0.12 / −0.05	▲12% / ▼5%	Positive;negative;zero sections
`;;;`	anything	(hidden)	Hide values but keep them

> **Tip** A format has up to four sections separated by semicolons: **positive;negative;zero;text**. Conditions in square brackets, e.g. `[<100000]`, can replace the default sections.

1.7 Conditional formatting and custom rules

- **Highlight Cells Rules**: greater than, between, duplicate values, text contains.
- **Top/Bottom**: top 10 items, top 10%, above average.

- **Data Bars, Colour Scales, Icon Sets**: in-cell visuals for a quick heatmap.
- **Custom rule with a formula** (New Rule → Use a formula): the formula must return TRUE/FALSE for the **first** cell of the selection; lock columns with $.

```
Highlight the whole row when Amount > 50,000      select A2:H500 →  =$H2>50000
Overdue invoices (due date passed, unpaid)        =AND($E2<TODAY(),$F2="Unpaid")
Weekend orders                                     =WEEKDAY($B2,2)>5
Every other row (zebra)                            =MOD(ROW(),2)=0
Duplicate customer IDs                             =COUNTIF($A$2:$A$500,$A2)>1
```

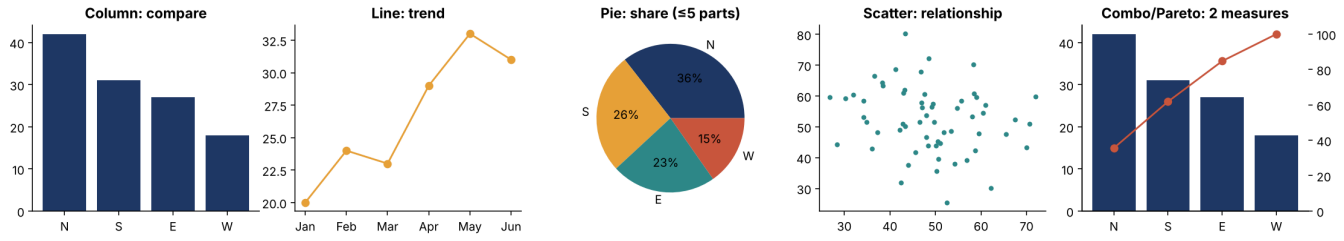
1.8 Freezing panes

View → Freeze Panes keeps headers visible while scrolling. **Freeze Top Row** or **First Column**, or select cell B2 and choose Freeze Panes to lock both row 1 and column A (everything above and left of the selected cell is frozen). Split (View → Split) shows two parts of a sheet side by side.

1.9 Sorting and filters

- Sort A→Z / Z→A on one column; **Custom Sort** for multiple levels (City A→Z, then Amount largest first) or by cell colour, or by a custom list (Jan, Feb, Mar...).
- Filter (`Ctrl` + `Shift` + `L`): pick values, Text/Number/Date filters (Top 10, Between, This Month), filter by colour.
- **Advanced Filter** (Data → Advanced): criteria range with AND (same row) / OR (different rows); copy unique records to another location.
- Use `SUBTOTAL(9,H2:H500)` or `AGGREGATE` to sum only visible (filtered) rows.

1.10 Charts: concept and creation



Pick the chart from the question: compare, trend, share, relationship, or two measures together.

1. Select the data, including headers (or click inside a Table).
2. Insert → Recommended Charts, or choose Column/Line/Pie (`Alt` + `F1` inserts a default chart instantly; `F11` on a new sheet).
3. Use the **+** button for chart elements: title, axis titles, data labels, legend.
4. Chart Design → Change Chart Type → Combo for two measures (revenue bars + margin % line on a secondary axis).

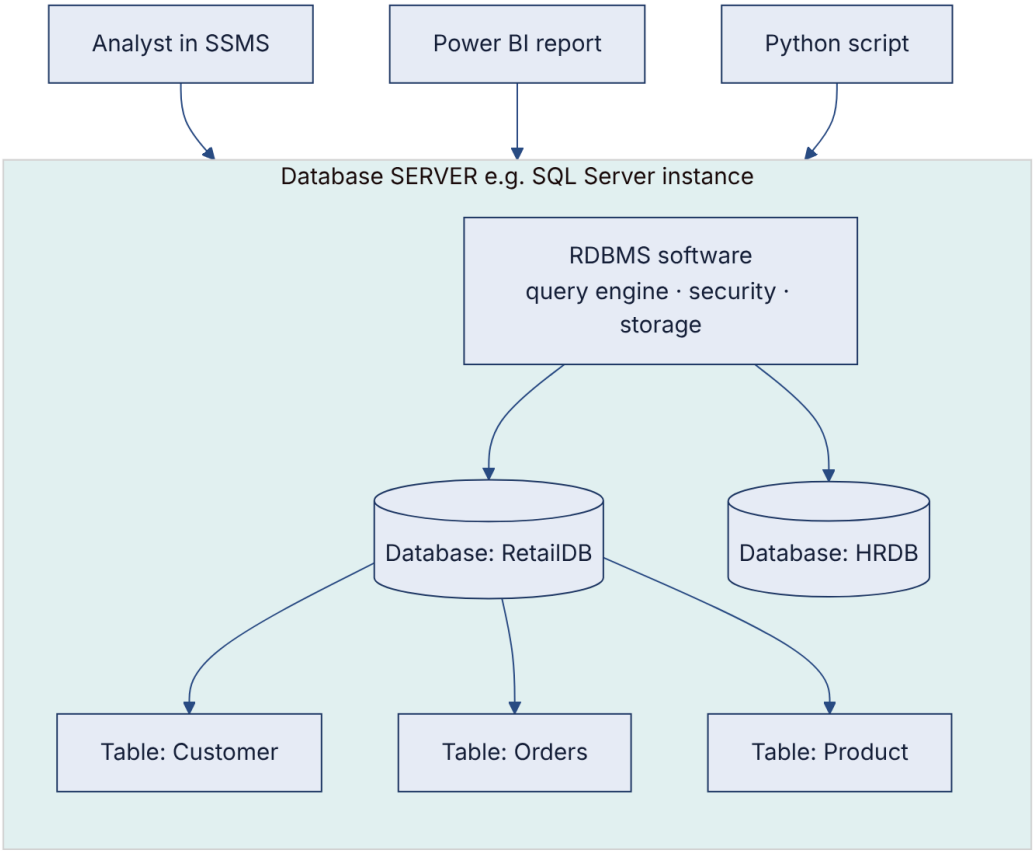
5 Format: remove clutter (gridlines, 3D), sort bars, write the insight in the title ("South leads with 34% of sales").

Tip Practise every Class 1 topic in the **Excel Basic-to-Advanced workbook**: one sheet per topic with live formulas.

Building Blocks · Class 2

Building Blocks · Class 2: Introduction to RDBMS

2.1 Server, database, DBMS, RDBMS



Users and tools connect to a server; the RDBMS software on it manages databases, and each database holds related tables.

Server	A computer (physical or cloud) that runs database software and answers requests over a network. In SQL Server one machine can host several *instances*.
Database (DB)	An organised collection of related data, e.g. RetailDB with Customer, Orders and Product tables.
DBMS	Software to create, read, update and secure data (handles storage, concurrency, backup, permissions).

RDBMS	A DBMS that stores data in **tables (relations)** of rows and columns linked by keys, and is queried with SQL. Follows ACID properties: Atomicity, Consistency, Isolation, Durability.

Excel sheet vs database table

- Excel: ~1M rows per sheet, one user editing, any value in any cell.
- Database: billions of rows, thousands of users, every column has a fixed data type and rules.

Real example

When you order on a food app, the order row is written to an Orders table, linked by customer_id to Customer and by restaurant_id to Restaurant. Analysts later query those same tables for reports.

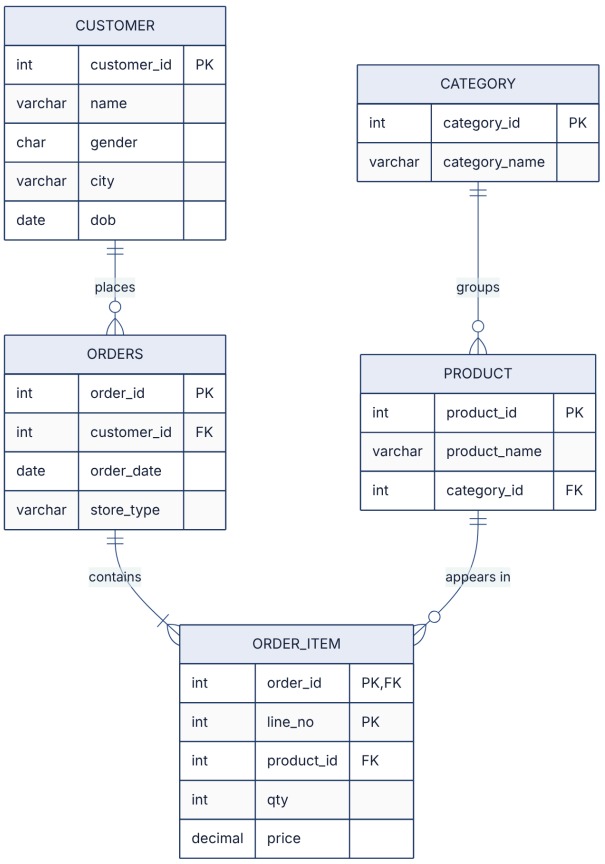
2.2 Database design and structure

- **Table** (relation/entity) → **row** (record/tuple) → **column** (field/attribute).
- **Schema**: a named container of objects (dbo.Customer, sales.Orders).
- **Normalisation**: split data so each fact is stored once. 1NF: one value per cell, no repeating groups. 2NF: every non-key column depends on the whole key. 3NF: no column depends on another non-key column (store city once in a City table, not in every order).
- **OLTP vs OLAP**: transactional systems (normalised, fast writes) vs analytics warehouses (star schemas, fast reads).

2.3 Key RDBMS vendors

Product	Vendor	Notes
SQL Server / Azure SQL	Microsoft	Used in this course; T-SQL dialect; free Developer and Express editions
MySQL	Oracle (open source)	Popular for web apps
PostgreSQL	Open source	Feature-rich; very popular in start-ups and cloud
Oracle Database	Oracle	Large enterprises, banking
SQLite	Open source	File-based, inside phones and apps
Cloud warehouses	Snowflake, Google BigQuery, Amazon Redshift, Databricks SQL, Microsoft Fabric	Analytics at scale; standard SQL with small dialect differences

2.4 Entity Relationship Diagram (ERD)



Crow's-foot ERD for a retail database. "||--o{" reads "exactly one ... zero or many".

2.5 Keys and constraints

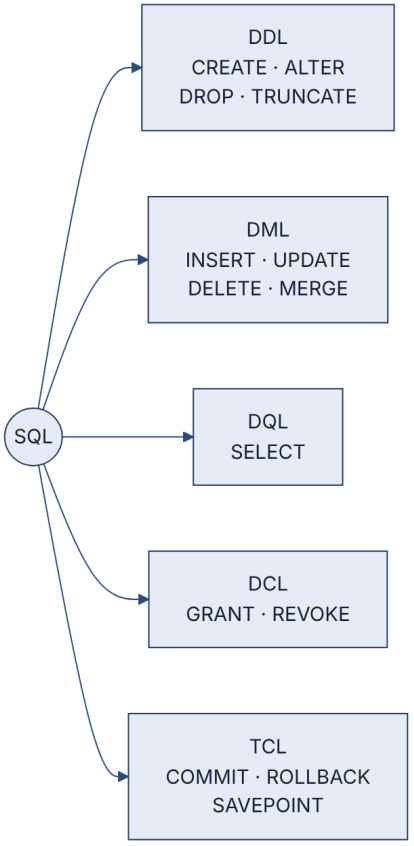
Key / constraint	Meaning	Example
Primary key (PK)	Uniquely identifies each row; not NULL; one per table	customer_id
Foreign key (FK)	References a PK in another table; enforces referential integrity	Orders.customer_id → Customer.customer_id
Candidate key	Any minimal column set that could be the PK	customer_id, email, PAN
Alternate key	A candidate key not chosen as PK	email
Composite key	PK made of 2+ columns	(order_id, line_no)
Super key	Any unique column set, even with extra columns	(customer_id, name)
Surrogate vs natural key	System-generated ID vs a real-world identifier	IDENTITY(1,1) vs PAN number
NOT NULL · UNIQUE · CHECK · DEFAULT	Column rules	`CHECK (qty > 0)`, `DEFAULT GETDATE()`

2.6 Relationships and metadata

- **One-to-one**: employee ↔ employee_passport. **One-to-many**: customer → orders (most common). **Many-to-many**: students ↔ courses, implemented with a bridge table Enrolment(student_id, course_id).
- **Metadata** is data about data: table names, column names, types, keys, row counts. Query it with `SELECT * FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'Customer';` or `EXEC sp_help 'Customer';`

2.7 SQL, SQL Server and SSMS

SQL (Structured Query Language) is the standard language for relational databases. It is **declarative**: you say *what* you want; the engine decides *how*. **T-SQL** is Microsoft's dialect (adds variables, TOP, procedures). **SSMS** (SQL Server Management Studio) is the free Windows tool for connecting to SQL Server, browsing objects in Object Explorer and running queries.



The five SQL sub-languages.

Building Blocks · Class 3

Building Blocks · Class 3: Data Definition Language

3.1 Create, select and drop a database

```
CREATE DATABASE RetailDB;           -- create
USE RetailDB;                       -- select it for the following statements
SELECT name FROM sys.databases;     -- list databases
-- DROP DATABASE RetailDB;          -- deletes the database and ALL its data (irreversible)
```

3.2 Data types

Category	Types	Use for
Whole numbers	TINYINT (0–255), SMALLINT, INT (±2.1 billion), BIGINT	IDs, counts
Exact decimals	DECIMAL(p,s) / NUMERIC, MONEY	Money: DECIMAL(10,2) = up to 99,999,999.99
Approximate	FLOAT, REAL	Scientific values (never for money)
Text	CHAR(n) fixed, VARCHAR(n) variable, NVARCHAR(n) Unicode (Hindi, emojis), VARCHAR(MAX)	Names, codes, comments
Date & time	DATE, TIME, DATETIME2, DATETIMEOFFSET (with time zone)	Order dates, timestamps
Other	BIT (0/1), UNIQUEIDENTIFIER, VARBINARY, JSON (SQL Server 2025), VECTOR (2025)	Flags, GUIDs, files, documents, AI embeddings

3.3 Create and drop tables, with keys and constraints

```
CREATE TABLE Customer (
    customer_id INT IDENTITY(1001,1) PRIMARY KEY,          -- auto number 1001, 1002...
    full_name   NVARCHAR(80) NOT NULL,
    email       VARCHAR(120) UNIQUE,
    city        VARCHAR(40)  DEFAULT 'Gurugram',
    dob         DATE CHECK (dob < '2010-01-01')
);
CREATE TABLE Orders (
    order_id    INT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date  DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
    amount      DECIMAL(10,2) CHECK (amount >= 0),
    CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);
-- DROP TABLE Orders;   (drop the child first: Customer cannot be dropped while Orders references it)
```

3.4 Insert values

```
INSERT INTO Customer (full_name, email, city, dob)
VALUES ('Asha Verma', 'asha@example.com', 'Pune', '1994-03-12'),
       ('Ravi Kumar', 'ravi@example.com', DEFAULT, '1989-11-02');     -- multi-row insert (max 1000 rows per VALUES)
INSERT INTO Orders VALUES (1, 1001, '2026-10-01', 2499.00);           -- all columns in table order
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2026-01-01';   -- insert from a query
```

3.5 Modify records: UPDATE (always with WHERE)

```sql
UPDATE Customer SET city = 'Noida' WHERE customer_id = 1002;
UPDATE Orders   SET amount = amount * 1.05 WHERE order_date >= '2026-10-01';   -- 5% price revision
-- Without WHERE every row changes. Run the same WHERE as a SELECT first to check the rows.
```

3.6 ALTER: change the table's properties

```sql
ALTER TABLE Customer ADD phone VARCHAR(15);                        -- add a column
ALTER TABLE Customer ALTER COLUMN city VARCHAR(60);               -- change type/size
ALTER TABLE Customer DROP COLUMN phone;                           -- remove a column
ALTER TABLE Orders ADD CONSTRAINT chk_amount CHECK (amount < 1000000);
ALTER TABLE Orders DROP CONSTRAINT chk_amount;
EXEC sp_rename 'Customer.full_name', 'customer_name', 'COLUMN';  -- rename a column
```

3.7 DELETE vs TRUNCATE vs DROP

	DELETE	TRUNCATE TABLE	DROP TABLE
Removes	Rows matching WHERE (or all)	All rows, keeps structure	Rows + structure + constraints
Language	DML	DDL	DDL
Identity counter	Continues	Resets to seed	Gone
Works with FK references	Yes (if rules allow)	Not if referenced by a FK	Not while referenced
Rollback in a transaction	Yes	Yes in SQL Server (inside BEGIN TRAN)	Yes in SQL Server (inside BEGIN TRAN)

3.8 Best practices and key takeaways

✔ Name consistently: singular or plural tables, snake_case columns, `pk_` / `fk_` / `chk_` constraint prefixes
✔ Choose the smallest correct type: DATE not VARCHAR for dates, DECIMAL for money
✔ Every table gets a primary key; every relationship gets a foreign key
✔ List columns in INSERT; never rely on column order
✔ Wrap risky changes in a transaction: `BEGIN TRAN … ROLLBACK / COMMIT`
✔ Keep scripts in files (and Git) so the database can be rebuilt

Building Blocks · Class 4

Building Blocks · Class 4: Data extraction with SELECT

Examples use the retail tables: Customer(customer_id, name, gender, city, dob), Orders(order_id, customer_id, order_date, store_type, amount), Product(product_id, product_name, category, price), Order_Item(order_id, product_id, qty).

4.1 Subsetting, sorting, filtering, removing duplicates

```sql
SELECT * FROM Customer;                                    -- all columns (avoid in production)
SELECT name, city FROM Customer;                           -- subset columns
SELECT TOP 10 order_id, amount FROM Orders ORDER BY amount DESC;   -- top 10 orders
SELECT DISTINCT city FROM Customer;                        -- remove duplicates
SELECT * FROM Orders WHERE store_type = 'e-Shop' AND amount > 5000;
SELECT * FROM Orders ORDER BY order_date DESC, amount ASC;         -- multi-level sort
```

4.2 Operators

Type	Operators	Example
Assignment	`=` in SET and variables	`DECLARE @fy DATE = '2026-04-01';`
Arithmetic	`+ - * / %`	`amount * 0.18 AS gst`; integer division: `7/2 = 3`, use `7/2.0`
Comparison (conditional)	`= <> != > < >= <=`	`WHERE amount <> 0`
Range / list	`BETWEEN … AND`, `IN (…)`	`WHERE city IN ('Pune','Delhi')`
Wildcards with LIKE	`%` any chars, `_` one char, `[a-c]` set, `[^a]` not	`WHERE name LIKE 'A%'`; `LIKE '_a%'`
NULL handling	`IS NULL`, `IS NOT NULL`, `ISNULL()`, `COALESCE()`	`COALESCE(phone, email, 'no contact')`
Logical	`AND OR NOT`	`WHERE NOT (city = 'Delhi' OR amount < 100)`

Tip Precedence: brackets → arithmetic → comparison → NOT → AND → OR. `WHERE a OR b AND c` means `a OR (b AND c)`; add brackets to be clear.

4.3 Sequence of operations



SQL is written SELECT-first but executed FROM-first. That is why a SELECT alias cannot be used in WHERE, but can be used in ORDER BY.

4.4 Functions

Group	Functions	Business example
Text	LEN, LEFT, RIGHT, SUBSTRING, CHARINDEX, UPPER, LOWER, TRIM, REPLACE, CONCAT, STRING_AGG	`UPPER(LEFT(city,3))` city code; `STRING_AGG(product_name, ', ')` products per order
Date & time	GETDATE, YEAR, MONTH, DATEPART, DATENAME, DATEADD, DATEDIFF, EOMONTH, FORMAT, CONVERT	`DATEDIFF(DAY, order_date, GETDATE())` days since order
Number	ROUND, CEILING, FLOOR, ABS, POWER, SQRT	`ROUND(AVG(amount), 2)`

Group	Functions	Business example
Aggregate	COUNT, COUNT(DISTINCT), SUM, AVG, MIN, MAX	`COUNT(DISTINCT customer_id)` active customers
Conversion / logic	CAST, CONVERT, TRY_CONVERT, CASE, IIF	`CASE WHEN amount >= 10000 THEN 'High' ELSE 'Normal' END`

```sql
SELECT city,
       COUNT(*)                    AS orders,
       SUM(o.amount)               AS revenue,
       ROUND(AVG(o.amount), 0)     AS avg_order_value
FROM Orders o JOIN Customer c ON o.customer_id = c.customer_id
WHERE o.order_date >= '2026-04-01'
GROUP BY city
HAVING SUM(o.amount) > 100000
ORDER BY revenue DESC;
```

4.5 Joins: extracting data from multiple tables

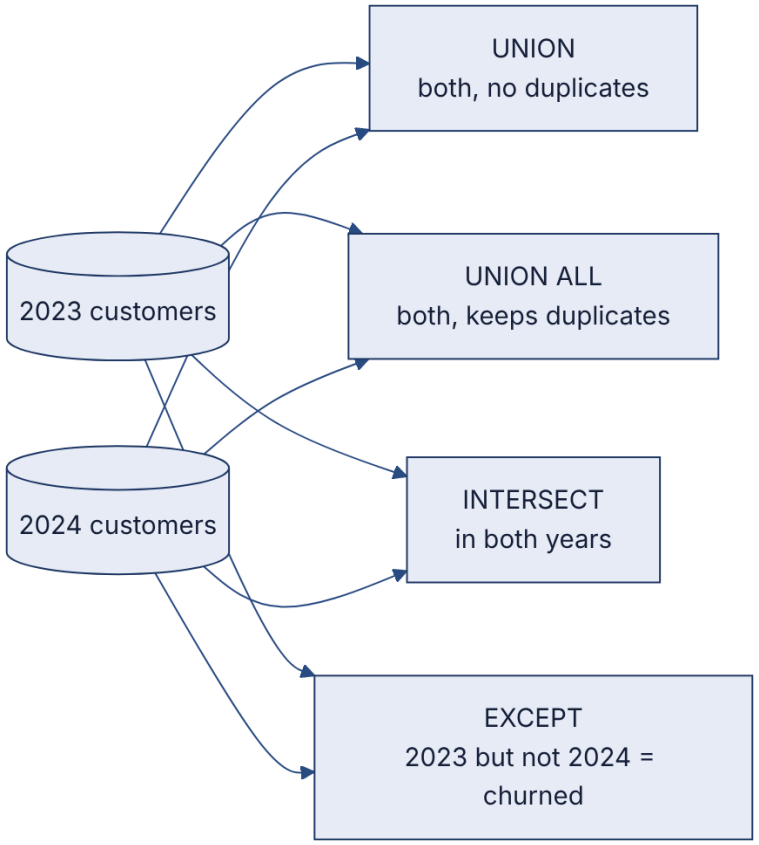


The six join types.

```sql
-- INNER: orders with their customer
SELECT o.order_id, c.name, o.amount FROM Orders o INNER JOIN Customer c ON o.customer_id =
c.customer_id;
-- LEFT: every customer, even those who never ordered (find inactive customers)
SELECT c.name FROM Customer c LEFT JOIN Orders o ON c.customer_id = o.customer_id WHERE o.order_id
IS NULL;
```

```sql
-- RIGHT: mirror of LEFT;  FULL OUTER: unmatched rows from both sides (data reconciliation)
SELECT a.id, b.id FROM System_A a FULL OUTER JOIN System_B b ON a.id = b.id WHERE a.id IS NULL OR
b.id IS NULL;
-- SELF: employee with manager name
SELECT e.name AS employee, m.name AS manager FROM Employee e LEFT JOIN Employee m ON e.manager_id =
m.emp_id;
-- CROSS: every store × every month (to build a complete calendar grid)
SELECT s.store_id, m.month_start FROM Store s CROSS JOIN Months m;
-- NATURAL JOIN joins automatically on same-named columns. It exists in MySQL/PostgreSQL/Oracle
-- but NOT in SQL Server; always write explicit ON conditions instead.
```

4.6 Appending datasets and set operators



Set operators combine rows (vertically); joins combine columns (horizontally). Both queries must have the same number of columns with compatible types.

```sql
SELECT customer_id FROM Orders_2023
UNION ALL SELECT customer_id FROM Orders_2024;      -- append (fast, keeps duplicates)
SELECT customer_id FROM Orders_2023
EXCEPT    SELECT customer_id FROM Orders_2024;      -- churned customers
```

4.7 Subqueries

```sql
-- Ordinary (nested) subquery in WHERE
SELECT * FROM Orders WHERE amount > (SELECT AVG(amount) FROM Orders);
SELECT name FROM Customer WHERE customer_id IN (SELECT customer_id FROM Orders WHERE store_type =
'Flagship store');
```

```sql
-- Inline view (derived table) in FROM
SELECT city, AVG(cust_rev) AS avg_revenue_per_customer
FROM (SELECT c.city, c.customer_id, SUM(o.amount) AS cust_rev
      FROM Customer c JOIN Orders o ON c.customer_id = o.customer_id
      GROUP BY c.city, c.customer_id) AS t
GROUP BY city;
```

4.8 Business use cases

Business question	SQL pattern
Top 5 cities by revenue this FY	JOIN + WHERE date + GROUP BY + ORDER BY + TOP 5
Customers who never ordered	LEFT JOIN ... WHERE right.key IS NULL (or NOT EXISTS)
Repeat customers (more than 3 orders)	GROUP BY customer HAVING COUNT(*) > 3
Share of revenue by store type	SUM(amount) * 100.0 / (SELECT SUM(amount) FROM Orders)
Orders above average	Subquery in WHERE
Return rate by category	SUM(CASE WHEN qty < 0 THEN 1 ELSE 0 END) * 1.0 / COUNT(*)

> **Tip** Full worked practice with real results: **SQL Retail case study** (15 questions) and **SQL Mobile case study**.

Building Blocks · Class 5

Building Blocks · Class 5: SQL recitation – installing SQL Server, SSMS and importing data



From installation to your first query on imported data.

5.1 Install SQL Server

1. Download **SQL Server Developer** (free, full features, non-production) or **Express** (free, small production) from Microsoft's SQL Server downloads page. The current release is SQL Server 2025.
2. Choose **Basic** installation (or Custom to pick the instance name). Note the instance name shown at the end, e.g. `localhost\SQLEXPRESS` or `localhost` for a default instance.
3. Install **SSMS** separately (it is a different download). Recent SSMS versions (21 and later) install through the Visual Studio Installer and add a dark theme and optional Copilot.

5.2 Connect and use the IDE

- Connect dialog: Server type *Database Engine*; Server name from step 2; Authentication *Windows Authentication*. If a certificate warning appears on a local machine, tick *Trust server certificate*.

- **Object Explorer** (left): Databases → Tables → Columns, Keys, Constraints, Indexes. Right-click a table → *Select Top 1000 Rows*.
- **Query editor**: New Query (`Ctrl` + `N`); pick the database in the toolbar drop-down or write `USE RetailDB;`; run with `F5`; run only the highlighted lines by selecting them first; `Ctrl` + `R` hides the results pane; IntelliSense refresh `Ctrl` + `Shift` + `R`.
- Results to Grid (`Ctrl` + `D`) or Text (`Ctrl` + `T`); right-click the grid → Save Results As CSV.

5.3 Import flat files (CSV / TXT)

1. Right-click the database → **Tasks → Import Flat File**.
2. Browse to `Transactions.csv`; table name defaults to the file name.
3. **Preview** the first rows. In **Modify Columns**, set data types (DATE for dates, DECIMAL for amounts), mark a primary key, and allow NULLs where needed.
4. Finish, then check: `SELECT COUNT(*) FROM Transactions;` should match the file's row count.

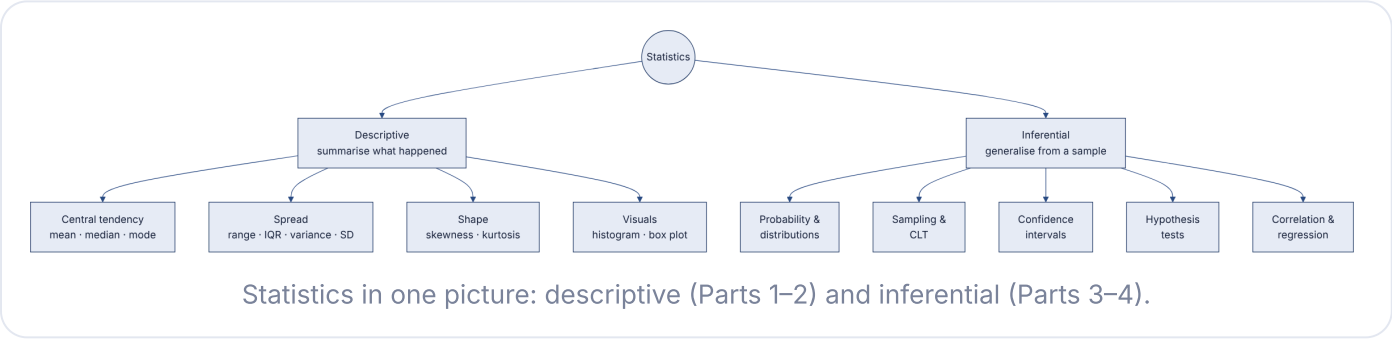
> **Common import errors:** dates in dd-mm-yyyy fail as DATE → import as VARCHAR, then convert with `TRY_CONVERT(DATE, tran_date, 105)`; amounts with commas or ₹ → import as text and clean; text longer than the default nvarchar(50) → widen the column in Modify Columns.

For Excel files use **Tasks → Import Data** (SQL Server Import and Export Wizard), or save the sheet as CSV first. For repeatable loads write `BULK INSERT`:

```
BULK INSERT dbo.Transactions
FROM 'C:\Data\Transactions.csv'
WITH (FORMAT = 'CSV', FIRSTROW = 2, FIELDTERMINATOR = ',', ROWTERMINATOR = '0x0a');
```

Building Blocks · Class 6

Building Blocks · Class 6: Introduction to Statistics (Part 1/4)



Statistics in one picture: descriptive (Parts 1–2) and inferential (Parts 3–4).

6.1 What statistics is and why analysts need it

Statistics is the science of **collecting, summarising and drawing conclusions from data** while measuring uncertainty. Example: an e-commerce firm sees average order value rise from ₹1,240 to ₹1,310 after a new checkout. Statistics answers whether that rise is real or just normal week-to-week noise.

6.2 Population vs sample, parameter vs statistic

	Population	Sample
Meaning	Every unit you care about (all 2 crore customers)	The subset you actually observe (5,000 surveyed customers)
Summary is called	Parameter	Statistic
Mean / SD symbols	μ, σ	$\bar{x}$, s
Size	N	n

6.3 Types of data and measurement scales

Scale	Type	Can you…	Example
Nominal	Categorical	Count, mode only	City, payment mode, gender
Ordinal	Categorical (ordered)	Rank, median	Rating Poor < Average < Good; education level
Interval	Numeric, no true zero	Add/subtract	Temperature in °C, calendar year
Ratio	Numeric, true zero	All maths incl. ratios	Sales, weight, age, delivery time

Numeric data is **discrete** (countable: items in cart) or **continuous** (measurable: weight). Data is also **cross-sectional** (many customers at one time), **time series** (one metric over time) or **panel** (many customers over time); and **structured** (tables) or **unstructured** (reviews, images).

6.4 Measures of central tendency

Measure	How	Excel	Best when
Mean	$\Sigma x / n$	`=AVERAGE()`	Symmetric numeric data
Median	Middle value of sorted data (average of the two middles if n is even)	`=MEDIAN()`	Skewed data: salaries, house prices, delivery times
Mode	Most frequent value	`=MODE.SNGL()`	Categories: most common shoe size
Weighted mean	$\Sigma(w \cdot x) / \Sigma w$	`=SUMPRODUCT(x,w)/SUM(w)`	Average price across products sold in different quantities
Trimmed mean	Mean after dropping top/bottom x%	`=TRIMMEAN(range,0.1)`	Judges' scores, outlier-prone data
Geometric mean	$(x_1 \cdot x_2 \cdot \ldots \cdot x_n)^{(1/n)}$	`=GEOMEAN()`	Average growth rates (CAGR)

Worked example: why the median matters

Salaries of 6 employees (₹ lakh): 4, 5, 5, 6, 7, 45. Mean = 72/6 = **12**; median = (5+6)/2 = **5.5**; mode = **5**. One senior salary pulls the mean far above what a typical employee earns, so report the median.

Building Blocks · Class 7

Building Blocks · Class 7: Introduction to Statistics (Part 2/4)

7.1 Measures of dispersion (spread)

Measure	Formula	Excel	Note
Range	max − min	`=MAX()-MIN()`	Very sensitive to outliers
Variance (sample)	$s^2 = \Sigma(x - \bar{x})^2 / (n - 1)$	`=VAR.S()`	Squared units
Standard deviation	$s = \sqrt{s^2}$	`=STDEV.S()` (population: `STDEV.P`)	Same units as data
Coefficient of variation	$s / \bar{x} \times 100\%$	`=STDEV.S()/AVERAGE()`	Compare consistency of things on different scales
IQR	Q3 − Q1	`=QUARTILE.INC(r,3)-QUARTILE.INC(r,1)`	Robust to outliers

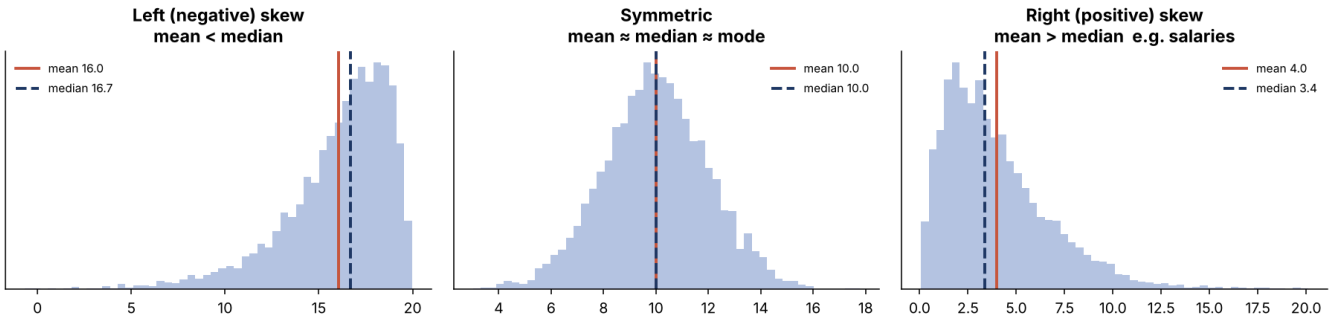
Worked example: two delivery partners

Partner A delivery times (min): 28, 30, 31, 29, 32 → mean 30, SD ≈ 1.6. Partner B: 15, 45, 20, 40, 30 → mean 30, SD ≈ 12.7. Same average, but A is far more reliable. Customers feel the spread, not the mean.

7.2 Percentiles and quartiles

The p-th percentile is the value below which p% of observations fall. Q1 = 25th, Q2 = median, Q3 = 75th percentile. "Your score is at the 90th percentile" means you beat 90% of test-takers. Excel: `=PERCENTILE.INC(range,0.9)`, `=PERCENTRANK.INC(range,x)`.

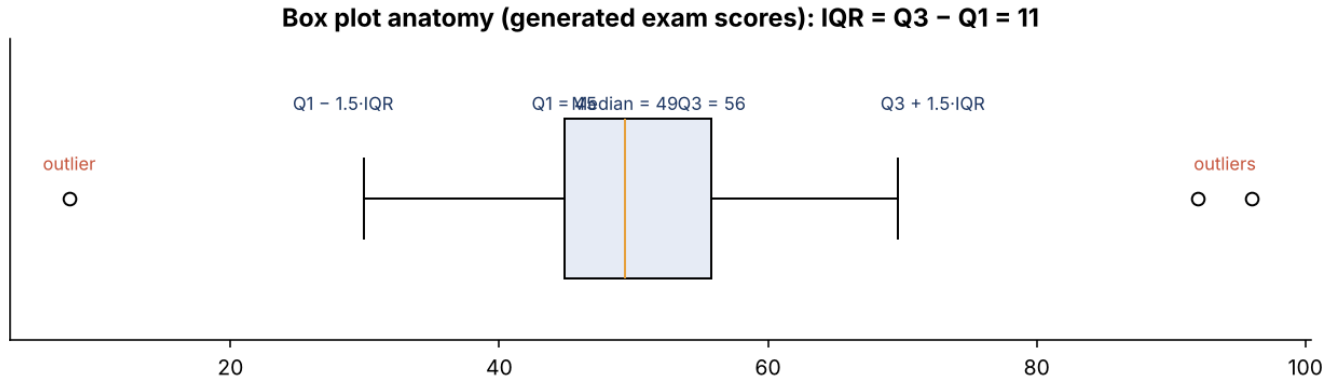
7.3 Shape: skewness and kurtosis



Generated data. In right-skewed data the long tail drags the mean above the median.

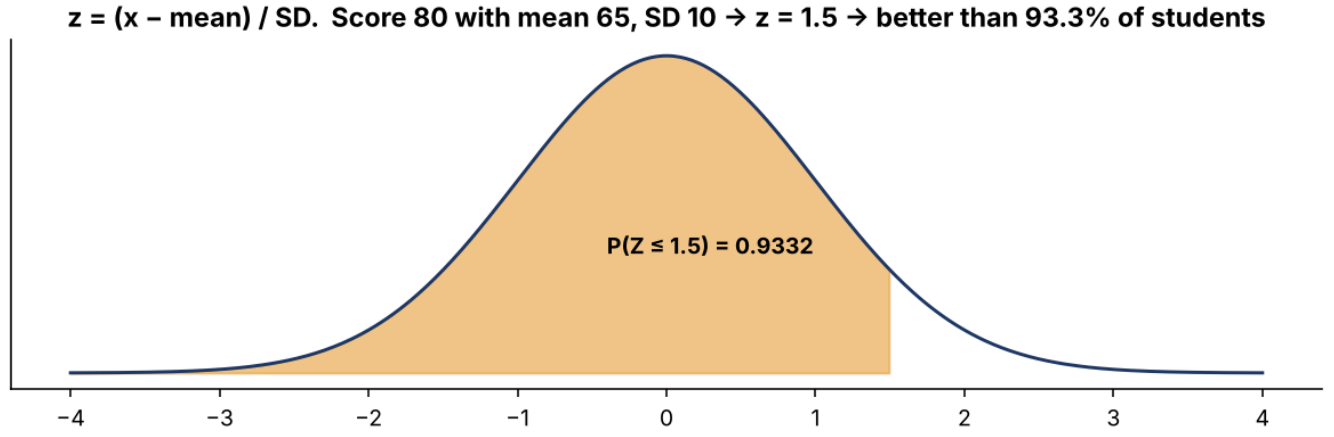
Excel: `=SKEW()` (positive = right tail) and `=KURT()` (positive = heavier tails than normal, more extreme values).

7.4 Box plot and outliers



The box spans Q1–Q3; whiskers reach the last points within 1.5 × IQR; points beyond are flagged as outliers.

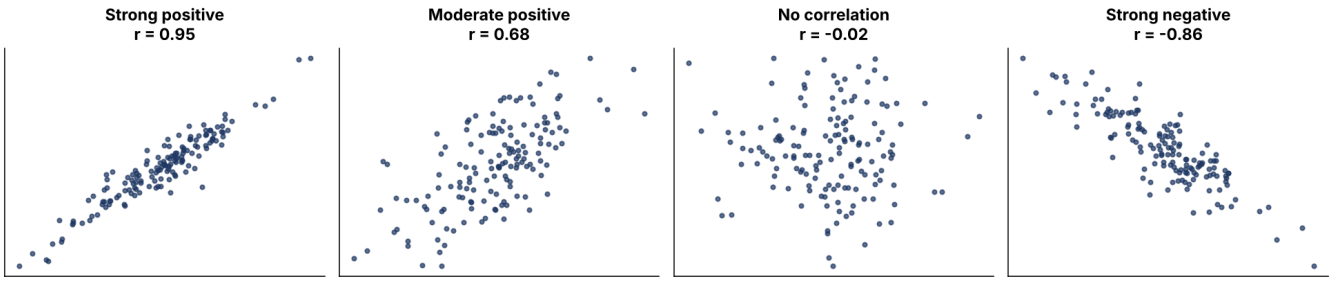
7.5 Standardisation: z-scores



A z-score says how many standard deviations a value is from the mean, so different scales can be compared.

Excel: `=STANDARDIZE(x, mean, sd)`; probability below: `=NORM.S.DIST(z,TRUE)`. Rule of thumb: $|z| > 3$ is unusual.

7.6 Relationship between two variables: covariance and correlation



Pearson's r runs from −1 to +1 and only measures straight-line association.

- Covariance: direction of joint movement, but its size depends on units. `=COVARIANCE.S()`
- Correlation r = cov / ($s_x \cdot s_y$): unit-free. `=CORREL()`. |r| > 0.7 strong, 0.3–0.7 moderate, < 0.3 weak.
- **Correlation is not causation**: ice-cream sales and AC sales both rise in summer because of heat, not because of each other.

7.7 Visualising distributions

Histogram (Insert → Statistic Chart → Histogram; set bin width), box & whisker chart, frequency table with `=FREQUENCY()` or a pivot with grouped values.

Building Blocks · Class 8

Building Blocks · Class 8: Introduction to Statistics (Part 3/4)

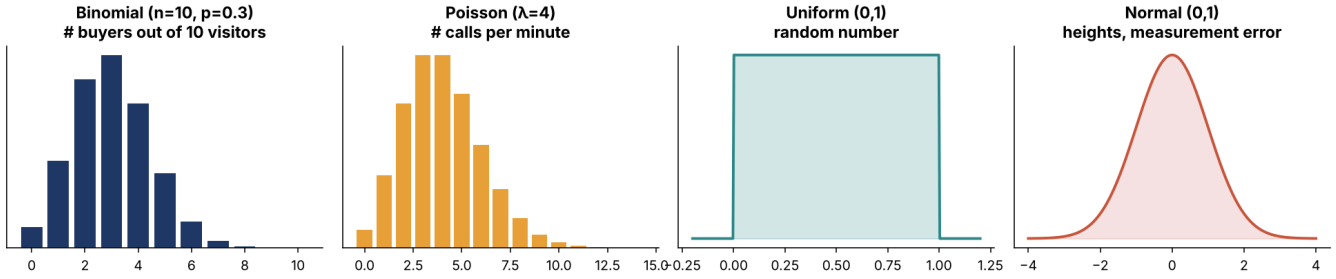
8.1 Probability basics

Experiment, outcome, event	Show an ad (experiment); click/no-click (outcomes); "user clicks" (event)
Probability	P(A) = favourable / total, between 0 and 1. Empirical: 230 clicks out of 10,000 views → P = 0.023
Complement	P(not A) = 1 − P(A)
Addition rule	P(A or B) = P(A) + P(B) − P(A and B)
Multiplication (independent)	P(A and B) = P(A) × P(B): two independent servers both failing
Conditional	P(A | B) = P(A and B) / P(B): probability of purchase given the user added to cart
Bayes' theorem	P(A | B) = P(B | A) · P(A) / P(B)

Worked Bayes example: fraud alert

1% of transactions are fraud. The rule flags 95% of frauds and wrongly flags 5% of genuine ones. If a transaction is flagged, P(fraud | flag) = 0.95×0.01 / (0.95×0.01 + 0.05×0.99) = 0.0095 / 0.059 = **16%**. Most alerts are false alarms because fraud is rare. This is why analysts look at precision, not just detection rate.

8.2 Random variables and probability distributions

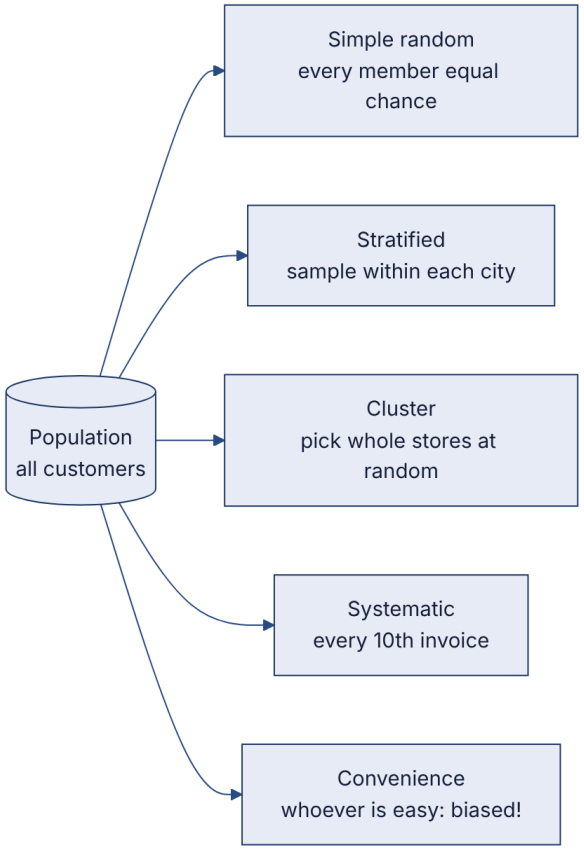


Discrete (bars) vs continuous (curves) distributions with business examples.

Distribution	Models	Excel
Bernoulli / Binomial	Number of successes in n yes/no trials	`=BINOM.DIST(k,n,p,FALSE)`
Poisson	Count of events in a fixed interval (calls per hour)	`=POISSON.DIST(k,λ,FALSE)`
Uniform	All values equally likely	`=RAND()`
Normal	Symmetric bell curve (heights, errors, sums of many effects)	`=NORM.DIST(x,μ,σ,TRUE)`
Exponential	Time between events	`=EXPON.DIST()`
t, χ^2, F	Sampling distributions used by tests (Part 4)	`T.DIST, CHISQ.DIST, F.DIST`

Expected value E[X] = Σ x·P(x): if a lottery ticket wins ₹1,000 with probability 0.001, its expected value is ₹1, so a ₹10 ticket loses ₹9 on average.

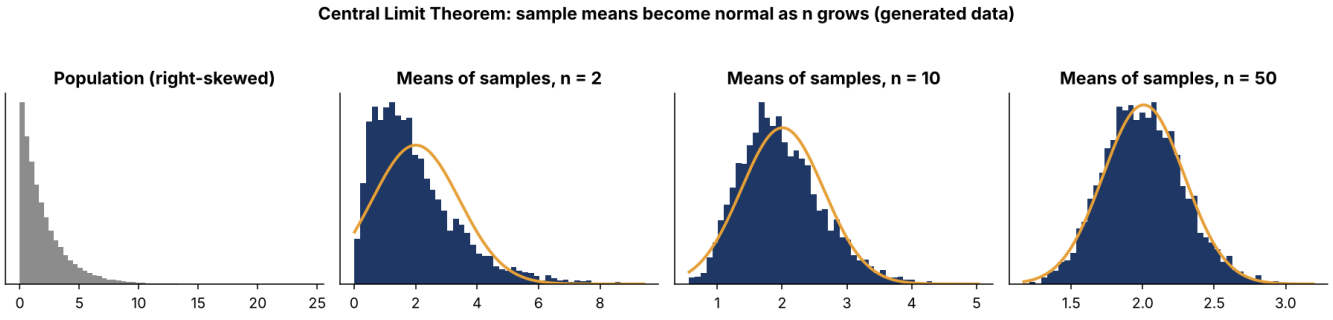
8.3 Sampling methods



Probability methods (random, stratified, cluster, systematic) allow valid inference; convenience samples are biased.

Biases to watch: selection bias (only app users surveyed), non-response bias, survivorship bias (studying only successful start-ups).

8.4 Sampling distribution and the Central Limit Theorem



Means of repeated samples form a normal curve, whatever the population shape, with spread σ/√n (the standard error).

- Bigger samples → smaller standard error → more precise estimates (4× the sample halves the error).
- The CLT is what lets us use z- and t-tests on skewed business data (Part 4).

What's new 2025–26

What changed since these classes were recorded

Excel (Microsoft 365) – new functions and features

Feature	What it does	Example
XLOOKUP / XMATCH	Lookup in any direction with a built-in "not found" value	`=XLOOKUP(A2,Products[ID],Products[Price],"Missing")`
Dynamic arrays: FILTER, SORT, SORTBY, UNIQUE, SEQUENCE	One formula spills a whole result range	`=SORT(FILTER(Sales,Sales[City]="Pune"),4,-1)`
LET / LAMBDA	Name parts of a formula; create your own reusable functions	`=LET(t,SUM(H:H),g,t*18%,t+g)`
TEXTSPLIT, TEXTBEFORE, TEXTAFTER, VSTACK, HSTACK, TAKE, DROP	Modern text and array shaping	`=TEXTBEFORE(A2,"@")`
GROUPBY, PIVOTBY, PERCENTOF	Pivot-table results from a formula	`=GROUPBY(Sales[City],Sales[Amount],SUM)`
REGEXTEST, REGEXEXTRACT, REGEXREPLACE	Pattern matching in text	`=REGEXTEST(B2,"^[6-9]\d{9}$")` valid Indian mobile
TRIMRANGE and trim references (A:.A)	Ignore empty trailing rows in whole-column references	`=SUM(H:.H)`
TRANSLATE, DETECTLANGUAGE	Translate text inside a cell	`=TRANSLATE(A2,"en","hi")`
IMPORTCSV / IMPORTTEXT (2026)	Pull a CSV file into the sheet as a dynamic array	`=IMPORTCSV("C:\data\sales.csv")`
Python in Excel (`=PY()`) + Python editor	Run pandas and charts inside cells (cloud-executed)	`=PY(xl("Sales[#All]",headers=True).describe())`
Copilot in Excel	Generate and explain formulas, fix errors, clean data, analyse text, build PivotTables and charts from a prompt	"Add a column with profit margin and highlight negatives"
`=COPILOT()` function	Calls AI inside a formula, row by row	`=COPILOT("Classify this review as Positive/Negative/Neutral",C2)`

Feature	What it does	Example
Agent Mode	Multi-step tasks (build a model or dashboard) from chat	"Create a monthly sales dashboard from this table"
Other	PivotTable auto-refresh, Show Changes history (one year), dark mode for the grid, Power Query in Excel for the web	

Availability depends on your Microsoft 365 plan, channel and region; Copilot features need a Copilot licence. Excel 2019/2021 standalone versions do not receive these.

SQL Server – newer T-SQL features

Release	Feature	Example
2022	GREATEST / LEAST	`SELECT GREATEST(q1, q2, q3, q4) AS best_quarter`
2022	DATETRUNC, DATE_BUCKET	`DATETRUNC(MONTH, order_date)` first day of the month
2022	GENERATE_SERIES	`SELECT value FROM GENERATE_SERIES(1, 12)` month numbers
2022	IS [NOT] DISTINCT FROM	NULL-safe comparison
2022	WINDOW clause, STRING_SPLIT with ordinal, APPROX_PERCENTILE_CONT	Reuse a window definition
2025	Regular expressions: REGEXP_LIKE, REGEXP_REPLACE, REGEXP_SUBSTR, REGEXP_INSTR, REGEXP_COUNT	`WHERE REGEXP_LIKE(email, '^[^@]+@[^@]+\.[a-z]{2,}$')`
2025	Native JSON data type, JSON indexes, JSON aggregates (JSON_OBJECTAGG, JSON_ARRAYAGG)	Store API responses efficiently
2025	VECTOR data type, VECTOR_DISTANCE, DiskANN vector index (preview)	Similarity search on AI embeddings
2025	Fuzzy string matching (preview), external AI model definitions, change event streaming (preview)	Match "Gurgaon" with "Gurugram"

SQL Server 2025 became generally available in November 2025. Regex and vector functions also exist in Azure SQL Database.

Practice & case study

Real-world mini case: "Mumbai Electronics" monthly review

A 6-store electronics chain shares an Orders export (Order ID, Date, Store, City, Category, Qty, Unit Price, Customer Type). The manager asks five questions. The answer to each uses only Building Blocks skills.

Question	Excel (Class 1)	SQL (Classes 2–4)	Statistics (Classes 6–8)
Revenue by store this month?	`=SUMIFS(Amount,Store,A2,Date,">="&EOMONTH(TODAY(),-1)+1)`	`GROUP BY store WHERE order_date >= DATETRUNC(MONTH, GETDATE())`	Mean and median order value per store
Which store is most consistent?	`=STDEV.S` of daily sales ÷ mean (CV)	`STDEV(daily_total)` in a derived table	Coefficient of variation
Highlight loss-making lines	Conditional formatting `=$J2<0`	`WHERE margin < 0`	Share of loss lines
Are big orders unusual?	`=STANDARDIZE(H2,AVERAGE(H:H),STDEV.S(H:H))`	`(amount - AVG(amount) OVER()) / STDEV(amount) OVER()`	$\|z\| > 3$ → outlier
Probability a walk-in buys a warranty	`=COUNTIFS(Type,"Walk-in",Warranty,"Yes")/COUNTIF(Type,"Walk-in")`	`AVG(CASE WHEN warranty='Yes' THEN 1.0 ELSE 0 END)`	Conditional probability

The **Excel workbook** in Downloads includes a "Case Study – Sales Dashboard" sheet with this scenario built out using live formulas.

More practice on the real course data: **Excel Sportsmen case**, **Excel Consumer Complaints case**, **SQL Retail case**.

Interview Q&A

Q What is the difference between a function and a formula in Excel?

A formula is anything starting with = (=A1+B1). A function is a named, built-in calculation used inside a formula (SUM, VLOOKUP).

Q Relative vs absolute reference?

A1 changes when copied; $A$1 stays fixed; A$1 and $A1 lock only the row or column. Use $ for constants such as a tax-rate cell.

Q How do you sum only visible rows after filtering?

`=SUBTOTAL(109,range)` or `=AGGREGATE(9,5,range)`. SUM includes hidden rows.

Q What is a primary key vs a foreign key?

A primary key uniquely identifies rows in its own table; a foreign key in another table points to that primary key and enforces that the referenced row exists.

Q DELETE vs TRUNCATE?

DELETE removes selected rows and fires triggers; TRUNCATE removes all rows quickly, resets identity, and cannot be used on a table referenced by a foreign key.

Q Why can't I use a column alias in WHERE?

WHERE is evaluated before SELECT, so the alias does not exist yet. Repeat the expression or wrap the query in a subquery/CTE.

Q Mean vs median: which would you report for delivery time?

Median (and the 90th percentile), because delivery times are right-skewed: a few very late orders inflate the mean.

Q What does a standard deviation tell a manager?

How much individual values typically differ from the average. Two stores with equal average sales but different SD differ in predictability.

Q Explain the Central Limit Theorem simply.

If you take many reasonably large samples and compute each sample's average, those averages form a bell curve around the true average, even if the original data is skewed. This makes confidence intervals and tests possible.

Resources

- ▶ YouTube: Simplilearn Excel full course · ▶ YouTube: Simplilearn SQL full course · ▶ YouTube: Simplilearn statistics for data science full course
- ▶ YouTube: Leila Gharani Excel functions you need to know · ▶ YouTube: Chandoo conditional formatting formulas
- ▶ YouTube: Alex The Analyst SQL basics tutorial · ▶ YouTube: install SQL Server and SSMS step by step
- ▶ YouTube: StatQuest statistics fundamentals · ▶ YouTube: Khan Academy statistics and probability
- 📘 Microsoft: What's new in Excel (Tech Community blog) · 📘 Excel functions by category
- 📘 Microsoft Learn: What's new in SQL Server 2025 · 📘 Regex support GA announcement
- 📘 Download SQL Server · 📘 Download SSMS
- 📘 Excel Campus: 17 Excel updates from 2025

COURSE HANDBOOK 02

🧮 Building Blocks of ML & Applied AI

Class 1 to Class 10: programming foundations, Statistics parts 1–4, Mathematics parts 1–5

Building Blocks of ML and Applied AI · 10 classes · modules: Programming Elements, Statistics Pre-Learning, Mathematics Foundation

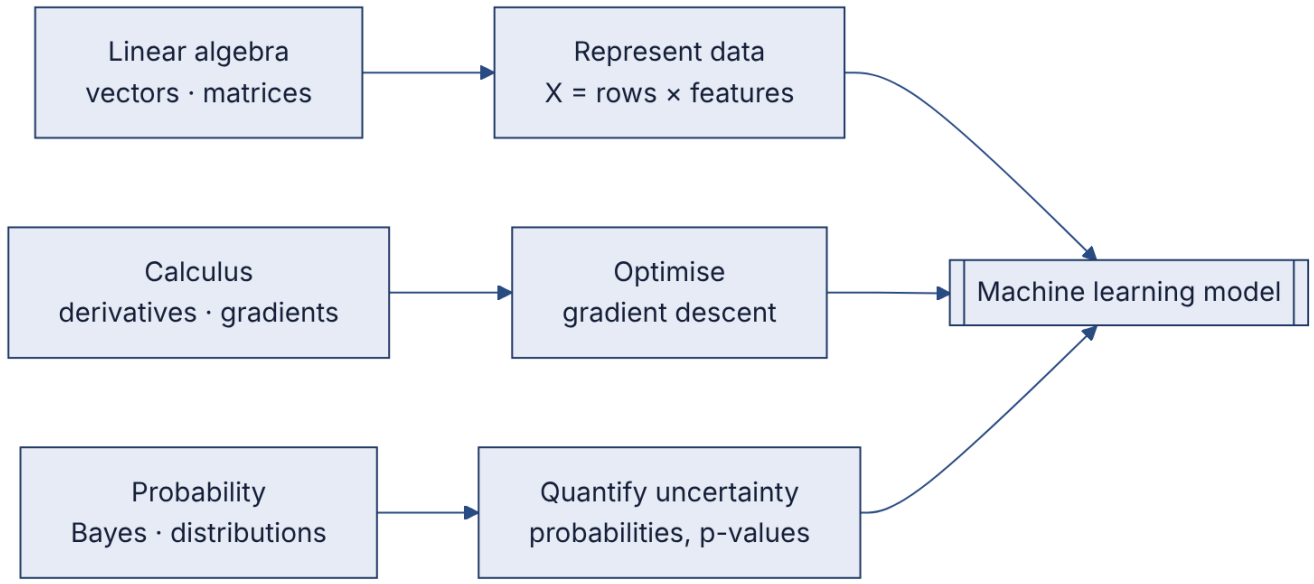
Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#m1b**

Overview

This pre-learning course prepares you for Python, statistics in Python, and the Machine Learning course. Class 1 teaches how programming works; Classes 2–5 are the four statistics pre-learning sessions; Classes 6–10 are the five mathematics sessions needed for machine learning.

Class	Topic	Prepares you for
Building Blocks ML · Class 1	Introduction to programming: programs and business use, databases, IDE, languages, high vs low level, compiler vs interpreter, syntax, variables, naming conventions, data types, operators, control flow, functions and UDFs, logic building and pseudo-code	Python for Data Science
Building Blocks ML · Class 2	Statistics Part 1/4: introduction, data types, central tendency	Python Classes 13–14, 19–21
Building Blocks ML · Class 3	Statistics Part 2/4: dispersion, shape, z-scores, correlation	EDA in Python
Building Blocks ML · Class 4	Statistics Part 3/4: probability, distributions, sampling, CLT	Python Classes 19–20
Building Blocks ML · Class 5	Statistics Part 4/4: estimation, confidence intervals, hypothesis testing, tests, regression basics	Python Class 21, ML Classes 1–3
Building Blocks ML · Class 6	Mathematics Part 1/5: algebra, functions, exponents and logarithms	Link functions, log transforms
Building Blocks ML · Class 7	Mathematics Part 2/5: vectors and matrices	How data is stored for ML
Building Blocks ML · Class 8	Mathematics Part 3/5: matrix operations, inverse, systems of equations, eigenvalues	Linear regression, PCA

Class	Topic	Prepares you for
Building Blocks ML · Class 9	Mathematics Part 4/5: calculus: derivatives, partial derivatives, gradients, optimisation	Gradient descent (ML Class 5)
Building Blocks ML · Class 10	Mathematics Part 5/5: probability for ML: sets, counting, conditional probability, Bayes, expectation	Naive Bayes, model evaluation



The three maths pillars of machine learning.

> The course lists Classes 2–5 as "Statistics Part 1/4 to 4/4" and Classes 6–10 as "Mathematics Part 1/5 to 5/5" without sub-topics. The topic split shown here follows the standard order of these pre-learning syllabi. Statistics Parts 1–3 are the same lectures as **Building Blocks Essentials Classes 6–8**, so those tabs summarise and add practice.

Building Blocks ML · Class 1

Building Blocks ML · Class 1: Introduction to programming

1.1 Computer programs and their business use

A **program** is a precise set of instructions a computer follows to turn **input** into **output**. Businesses use programs to do repeatable work at scale without errors:

Business area	Program does	Example
Banking	Validates and records transactions; flags fraud	UPI payment app checks balance, writes to ledger, sends SMS

Business area	Program does	Example
Retail / e-commerce	Pricing, inventory, recommendations	"Customers who bought this also bought..."
Operations	Scheduling, routing, forecasting	Delivery app assigns the nearest rider
Analytics	Cleans data, builds reports and models every day	Python script that emails a sales report at 9 AM

1.2 Databases and why software needs them

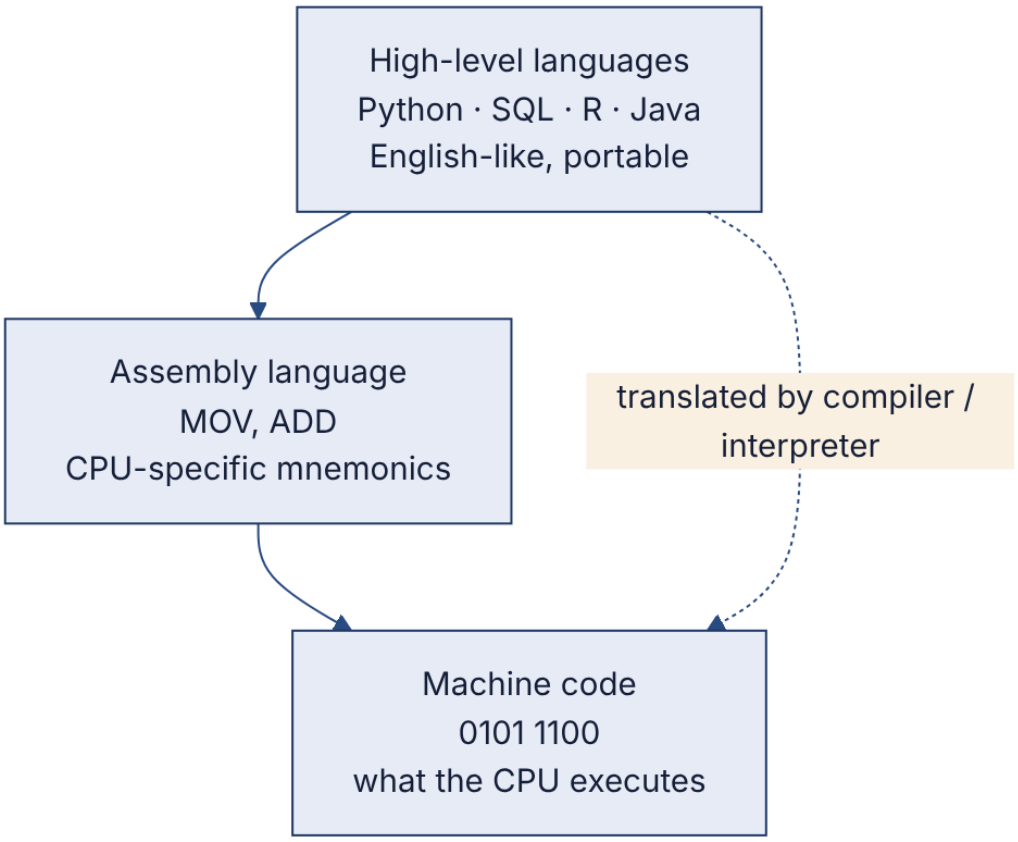
Programs forget everything when they stop. A **database** stores data permanently, lets many users read and write at once safely, enforces rules (an order must have a valid customer), and answers questions quickly with indexes. Almost every application is **front end** (screens) + **back end** (business logic program) + **database**.

1.3 IDE: Integrated Development Environment

An IDE is one application that combines a code **editor** (syntax colours, auto-complete), a way to **run** code, a **debugger** (pause and inspect), and file/project management.

Jupyter Notebook / JupyterLab	Code + output + notes in cells; the standard for data analysis (used in this course)
VS Code	Free general editor with Python and Jupyter extensions, Git, AI assistants
PyCharm, Spyder	Python-focused IDEs (Spyder looks like RStudio/MATLAB)
Google Colab	Free Jupyter in the browser, nothing to install
SSMS	The IDE for SQL Server

1.4 Programming languages; high-level vs low-level



High-level languages read like English and run anywhere; they are translated down to the machine code a CPU executes.

Language	Typical use
Python	Data science, ML, automation, web back ends
SQL	Querying databases
R	Statistics and research
Java / C#	Enterprise applications
JavaScript	Web pages and apps
C / C++	Systems, games, high performance (also inside NumPy)

1.5 Language translators: compiler vs interpreter



A compiler translates the whole program before running; an interpreter translates and runs line by line.

	Compiler	Interpreter
Translates	Entire program at once into an executable	One statement at a time while running
Errors	All reported before running	Stops at the first error it reaches
Speed	Faster execution	Slower, but instant feedback, great for exploring data
Examples	C, C++, Go, Rust	Python, R, JavaScript (Python first compiles to bytecode, then interprets it)

1.6 Why syntax rules?

Computers cannot guess intent. **Syntax** is the grammar of a language (brackets, colons, indentation, quotes). A syntax error means the translator cannot read the code at all; a **logic error** means it runs but gives the wrong answer (much harder to find).

```
if amount > 5000            # SyntaxError: missing colon
    print("High value")

total = price + quantity    # runs, but logic error: should be price * quantity
```

1.7 Programming basics

Variables

A variable is a named label pointing to a value in memory: `city = "Pune"`, `amount = 2499.0`. The value can change; the name lets you reuse it.

INC rules: Identifier Naming Conventions

Rule	Valid	Invalid
Start with a letter or underscore	`total_sales`, `_temp`	`2nd_quarter`
Only letters, digits, underscore	`sales_2026`	`sales-2026`, `sales 2026`, `sales$`
Case-sensitive	`Sales` ≠ `sales`	
Not a reserved keyword	`class_name`	`class`, `for`, `if`, `True`
Convention: meaningful snake_case for variables, CamelCase for classes, UPPER_CASE for constants	`avg_order_value`, `GST_RATE = 0.18`	`x1`, `a` (legal but unclear)

Data types

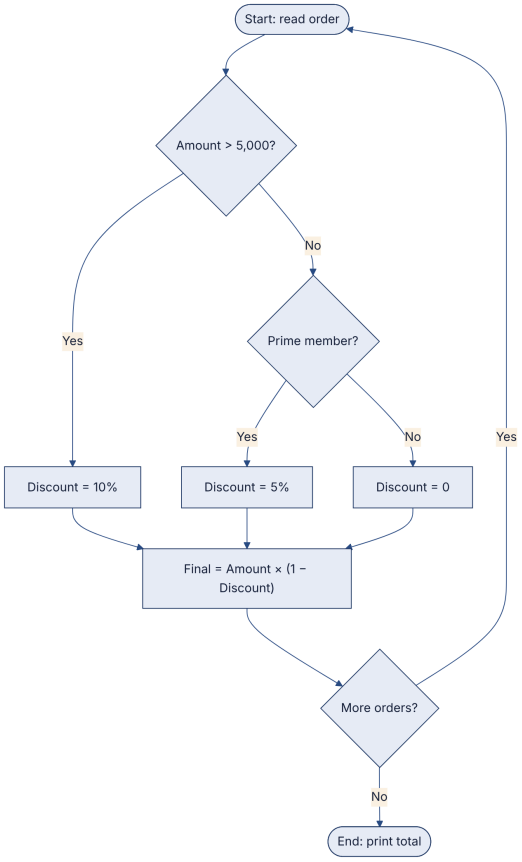
Integer	`42`	Counts, IDs
Float	`3.14`, `2499.0`	Prices, measurements
String	`"Gurugram"`	Text

Boolean	`True` / `False`	Flags, conditions
Collections	list `[1,2,3]`, tuple, dict `{"city":"Pune"}`, set	Groups of values
None / NULL	`None`	"No value"

Operators

Arithmetic	`+ - * / // % **` (// floor division, % remainder, ** power)
Comparison	`== != > < >= <=`
Logical	`and or not`
Assignment	`= += -= *=`
Membership / identity	`in`, `not in`, `is`

1.8 Control flow: conditions and loops



Flowchart of a discount rule with conditions and a loop over orders.

```
orders = [7200, 3100, 450]
prime = [False, True, False]
total = 0
for amount, is_prime in zip(orders, prime):      # loop: repeat for each order
    if amount > 5000:                             # condition
        discount = 0.10
    elif is_prime:
```

```
        discount = 0.05
    else:
        discount = 0
    total += amount * (1 - discount)
print(total)                                    # 6480 + 2945 + 450 = 9875.0

n = 0
while n < 3:                                    # while loop: repeat until condition is false
    n += 1
```

1.9 Functions and UDFs

A **function** is a named, reusable block that takes inputs (parameters) and returns an output. Built-in functions come with the language (`len` , `sum` , `round`); **UDFs** (user-defined functions) are ones you write.

```
def final_price(amount, is_prime=False):
    """Return the amount after the store's discount rule."""
    if amount > 5000:
        return amount * 0.90
    if is_prime:
        return amount * 0.95
    return amount

final_price(7200)          # 6480.0
final_price(3100, True)    # 2945.0
```

Benefits: write once, test once, reuse everywhere; easier to read and fix. Excel equivalent: `LAMBDA` ; SQL equivalent: user-defined functions and stored procedures.

1.10 Logic building and pseudo-code

Pseudo-code describes an algorithm in plain structured English before writing real code. It separates *thinking* from *syntax*.

```
PROBLEM: flag customers who have not ordered in 90 days

FOR each customer
    last_order = most recent order date for this customer
    IF no orders OR (today - last_order) > 90 days
        mark customer as "At risk"
    ELSE
        mark customer as "Active"
END FOR
OUTPUT count of At risk customers
```

1. Understand inputs and the exact output wanted.
2. Solve one example by hand.
3. Write the steps as pseudo-code / a flowchart.

4 Translate to code; test with normal and edge cases (empty list, zero, missing value).

Building Blocks ML · Class 2

Building Blocks ML · Class 2: Introduction to Statistics (Part 1/4)

Same lecture as **Building Blocks Essentials · Class 6**: what statistics is, population vs sample, data types and scales, and measures of central tendency. Revision summary:

- Descriptive statistics summarise; inferential statistics generalise from a sample to a population.
- Scales: nominal, ordinal, interval, ratio. Numeric data is discrete or continuous.
- Mean (symmetric data), median (skewed data), mode (categories), weighted mean (different quantities).

Practice

Q Classify each: PIN code, customer rating 1–5, temperature in °C, monthly spend.

PIN code: nominal (numbers used as labels). Rating: ordinal. Temperature °C: interval. Monthly spend: ratio.

Q Order values (₹): 450, 520, 480, 510, 9,800. Which average would you report and why?

Median = 510. The single ₹9,800 order pulls the mean to 2,352, which no typical customer spends.

Q A shop sells 100 units at ₹50 and 20 units at ₹200. What is the average selling price?

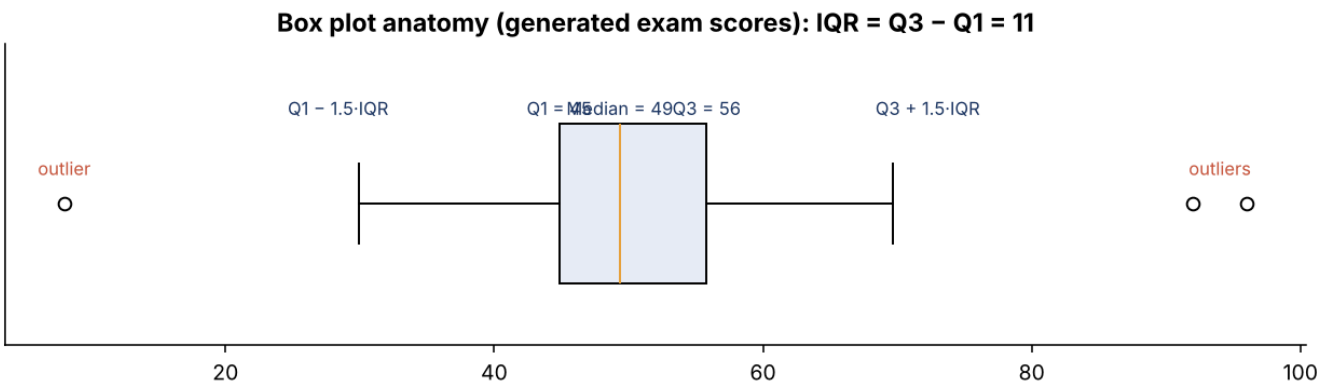
Weighted mean = (100×50 + 20×200) / 120 = 9,000 / 120 = ₹75, not the simple average of ₹125.

Python equivalents: `df["amt"].mean()`, `.median()`, `.mode()`, `np.average(x, weights=w)`.

Building Blocks ML · Class 3

Building Blocks ML · Class 3: Introduction to Statistics (Part 2/4)

Same lecture as **Building Blocks Essentials · Class 7**: range, variance, standard deviation, coefficient of variation, percentiles and quartiles, skewness and kurtosis, box plots and outliers, z-scores, covariance and correlation.



Box plot anatomy and the 1.5 × IQR outlier rule.

Practice

Q Data: 2, 4, 4, 4, 5, 5, 7, 9. Find the mean and population SD.

Mean = 40/8 = 5. Squared deviations: 9,1,1,1,0,0,4,16 = 32. Population variance = 32/8 = 4, SD = 2. (Sample SD = √(32/7) ≈ 2.14.)

Q Q1 = 40, Q3 = 60. Is 95 an outlier?

IQR = 20; upper fence = 60 + 1.5×20 = 90. 95 > 90, so yes.

Q r between ad spend and sales = 0.82. Can we say ads cause sales?

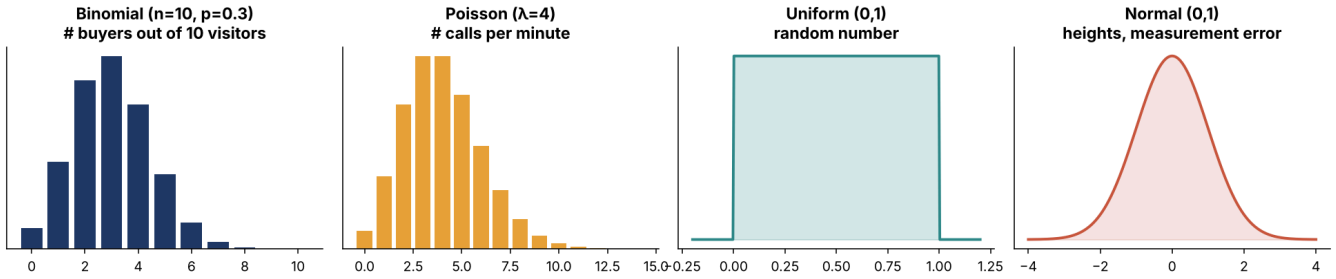
No. It is a strong positive association; causation needs an experiment (A/B test) or a careful causal analysis, because festive seasons may drive both.

Python: `df.describe()`, `df["x"].std()`, `df["x"].quantile([.25,.75])`, `df["x"].skew()`, `df.corr(numeric_only=True)`.

Building Blocks ML · Class 4

Building Blocks ML · Class 4: Introduction to Statistics (Part 3/4)

Same lecture as **Building Blocks Essentials · Class 8**: probability rules, conditional probability, Bayes, random variables, binomial/Poisson/normal distributions, expected value, sampling methods, sampling distribution and the CLT.



The distributions you will meet most often.

Practice

Q A call centre gets 4 calls per minute on average. Probability of exactly 6 calls in a minute?

Poisson: P(6) = $e^{-4} \cdot 4^6/6! \approx 0.104$. Excel `=POISSON.DIST(6,4,FALSE)`; Python `stats.poisson.pmf(6,4)`.

Q 30% of visitors buy. Probability that at least 1 of 5 visitors buys?

1 − P(none) = 1 − 0.7^5 = 1 − 0.168 = 0.832.

Q Heights ~ Normal(165, 8). What share is taller than 181 cm?

z = (181−165)/8 = 2 → about 2.3% (1 − 0.9772).

Q Population SD of order value is ₹600. What is the standard error of the mean for n = 100?

600/√100 = ₹60.

Building Blocks ML · Class 5

Building Blocks ML · Class 5: Introduction to Statistics (Part 4/4) – inferential statistics

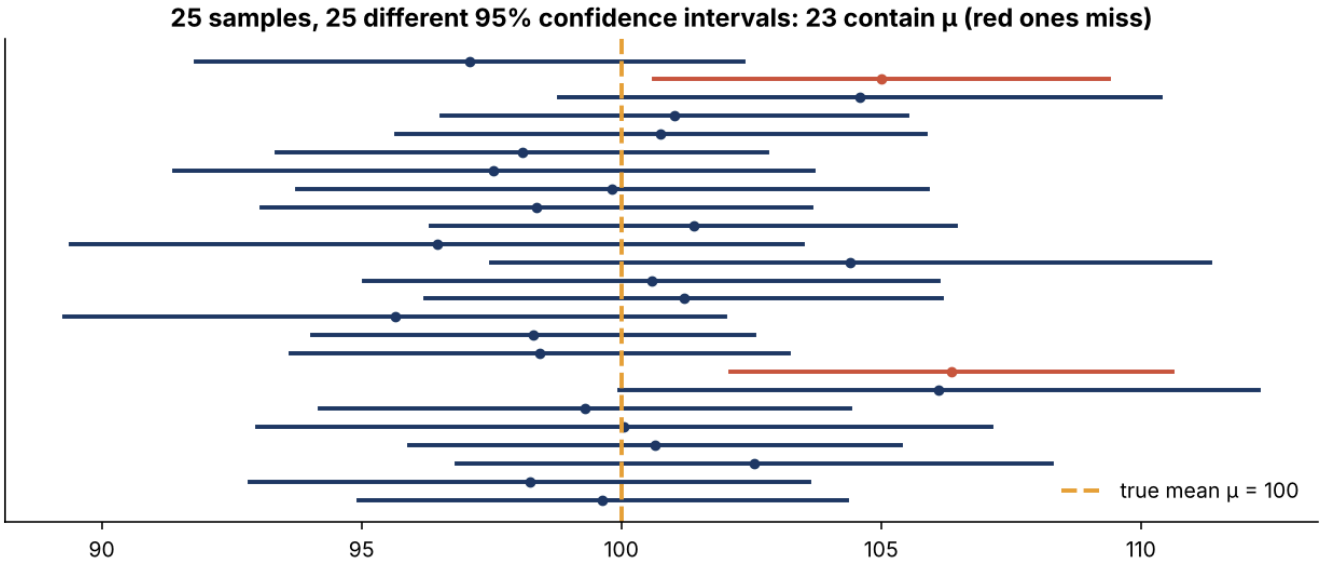
5.1 Point and interval estimation

A **point estimate** is a single best guess (sample mean $\bar{x}$ = ₹1,310). A **confidence interval** gives a range that captures the true value with a stated confidence:

```
CI = x̄ ± z* · s/√n   (z* = 1.96 for 95%; use t* for small samples)
```

Worked example

n = 400 orders, $\bar{x}$ = ₹1,310, s = ₹600. SE = 600/20 = 30. 95% CI = 1,310 ± 1.96×30 = **₹1,251 to ₹1,369**. Excel: `=CONFIDENCE.T(0.05,600,400)` returns the ± margin (≈ 59).

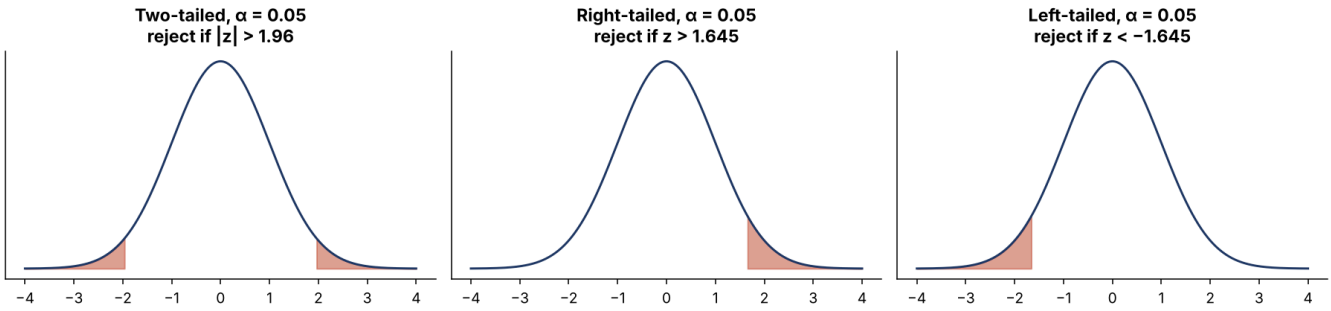


"95% confidence" describes the method: about 95 of every 100 intervals built this way contain the true mean (generated data).

Sample size for a margin E: $n = (z^* \cdot \sigma / E)^2$. To estimate average spend within ±₹50 with σ = 600: $n = (1.96 \times 600/50)^2 \approx 554$.

5.2 Hypothesis testing framework

1. H0: no change / no difference. H1: what you want evidence for.
2. Choose α (usually 0.05).
3. Pick the test; compute the statistic.
4. p-value ≤ α → reject H0; else fail to reject.
5. State the business conclusion and the effect size.



Two-tailed vs one-tailed rejection regions at α = 0.05.

Question	Test	Excel
Mean vs a target	One-sample z / t	`=T.TEST` needs two ranges; use `=T.DIST.2T(ABS(t),n-1)` on a computed t
Two independent groups	Two-sample t	`=T.TEST(A,B,2,3)` (two-tailed, unequal variance)
Before / after, same units	Paired t	`=T.TEST(Before,After,2,1)`
3+ groups	ANOVA	Data → Data Analysis → Anova: Single Factor
Two categorical variables	Chi-square	`=CHISQ.TEST(observed,expected)`
Two numeric variables	Correlation test / regression	Data Analysis → Regression

5.3 Errors

Type I (false positive, probability α): concluding a new checkout works when it does not. **Type II** (false negative, β): missing a real improvement. **Power** = 1 − β; increase it with a bigger sample.

5.4 Simple linear regression

$\hat{y} = b_0 + b_1 x$. Example: sales (₹ lakh) = 12 + 3.5 × ad spend (₹ lakh) means each extra lakh of ads is associated with ₹3.5 lakh more sales. $\mathbf{R^2}$ = share of variation in y explained by x. Excel: `=SLOPE()`, `=INTERCEPT()`, `=RSQ()`, `=FORECAST.LINEAR()`, or add a trendline to a scatter chart with "Display equation".

> **Tip** Full illustrated coverage of tests, p-values and errors: **ML Classes 1–2**; worked cases: **Hypothesis Testing case**.

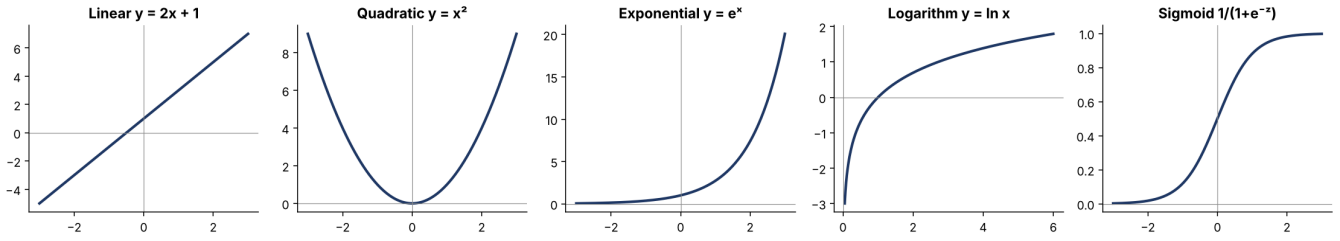
Building Blocks ML · Class 6

Building Blocks ML · Class 6: Introduction to Mathematics (Part 1/5) – algebra, functions, exponents and logs

6.1 Algebra refresher

- Variables and expressions: revenue = price × quantity → R = p·q.
- Solving linear equations: 3x + 5 = 20 → x = 5. Break-even: fixed cost ₹50,000, margin ₹250/unit → units = 50,000 / 250 = 200.
- Inequalities: 0.18x ≤ 9,000 → x ≤ 50,000.
- Summation notation: $\Sigma_{i=1}^{n} x_i = x_1 + x_2 + \ldots + x_n$ (Excel SUM). Product notation Π.

6.2 Functions and their shapes



Five shapes that appear throughout ML.

Linear y = mx + c	m = slope (change in y per unit x), c = intercept. Linear regression.
Quadratic y = ax^2 + bx + c	U-shape; squared error cost functions are quadratic, with one minimum.
Exponential y = a^{bx} / e^x	Compound growth; viral spread.
Logarithm y = log x	Inverse of exponential; compresses big ranges (income, population).
Sigmoid σ(z) = 1/(1+e^{-z})	Squashes any number to 0–1: logistic regression probabilities.

6.3 Exponents and logarithms

$a^m \cdot a^n = a^{m+n}$, $(a^m)^n = a^{mn}$, $a^0 = 1$, $a^{-n} = 1/a^n$	Exponent rules
log(ab) = log a + log b; log(a/b) = log a − log b; log(a^n) = n·log a	Log rules: turn products into sums
ln x = $\log_e$ x, e ≈ 2.718	Natural log used in ML and finance

Real-world use: CAGR

Revenue grew from ₹2 crore to ₹3.2 crore in 4 years. CAGR = (3.2/2)^(1/4) − 1 = 1.6^0.25 − 1 ≈ **12.5%**. Excel `=(B2/A2)^(1/4)-1` or `=RRI(4,2,3.2)`.

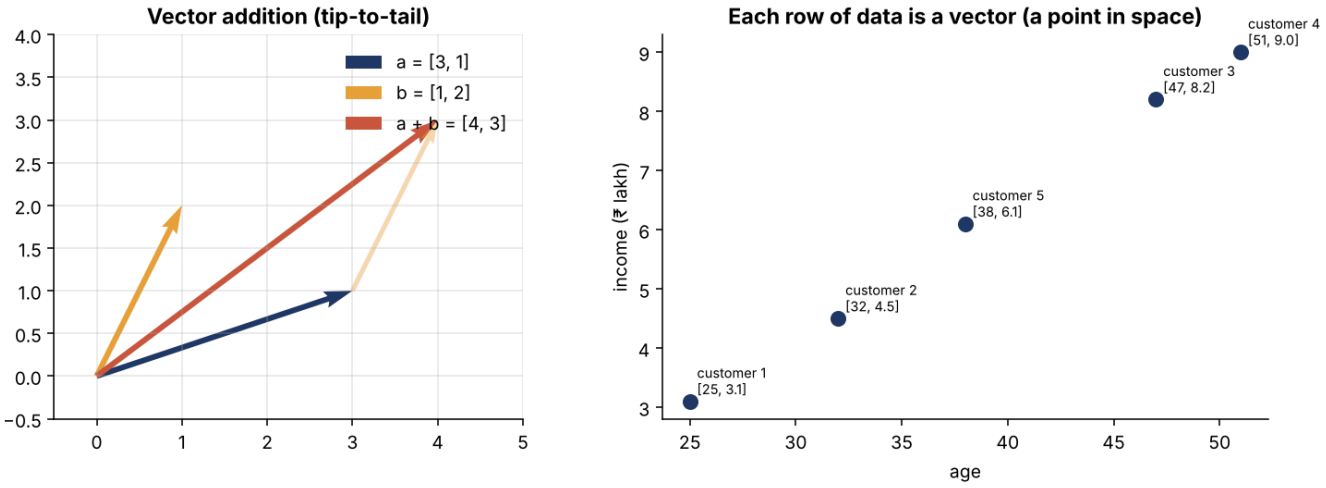
Why logs in data science: a log transform makes right-skewed data (income, sales) more symmetric; log-odds are the language of logistic regression; log-loss is a classification metric.

Building Blocks ML · Class 7

Building Blocks ML · Class 7: Introduction to Mathematics (Part 2/5) – vectors and matrices

7.1 Scalars, vectors, matrices, tensors

Scalar	A single number: 5
Vector	An ordered list: one customer = [age 32, income 4.5, visits 5]
Matrix	A 2-D grid: n customers × p features (a DataFrame without labels)
Tensor	3-D and higher: a batch of colour images = images × height × width × 3

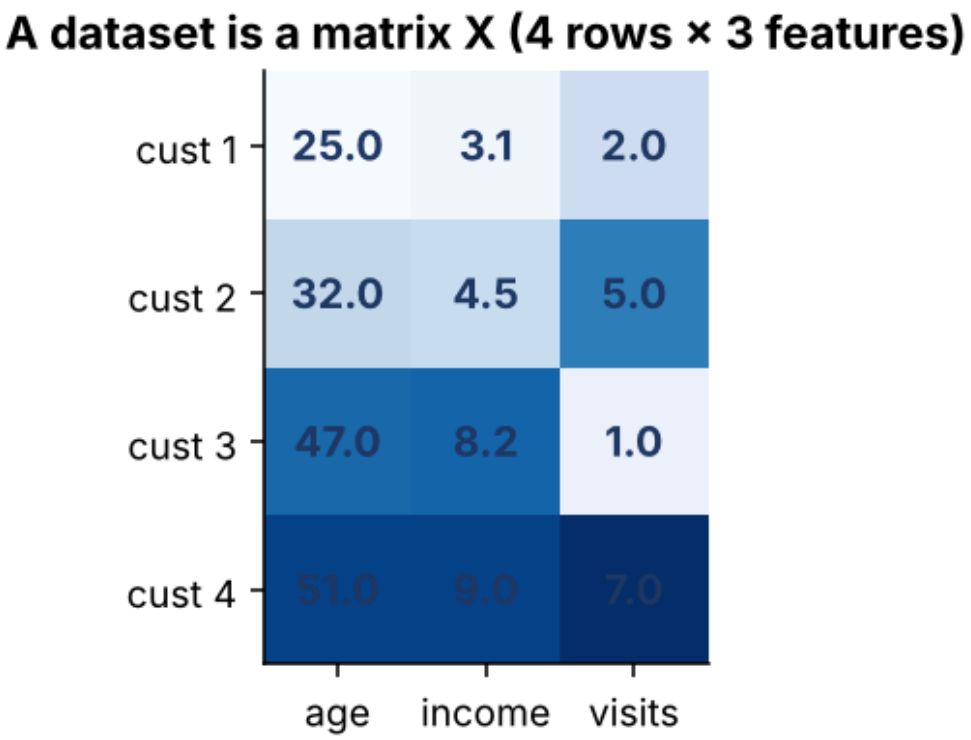


Left: adding vectors tip-to-tail. Right: each data row is a point (vector) in feature space; KNN and clustering measure distances between these points.

7.2 Vector operations

Operation	Example with a=[3,1], b=[1,2]	ML meaning
Addition	a + b = [4, 3]	Combining effects
Scalar multiplication	2a = [6, 2]	Scaling features
Dot product	a·b = 3×1 + 1×2 = 5	A linear model's prediction is a dot product: ŷ = w·x + b
Magnitude (norm)	‖a‖ = √(9+1) ≈ 3.16	Size of a vector; regularisation penalties
Euclidean distance	‖a − b‖ = √(4+1) ≈ 2.24	KNN, k-means
Cosine similarity	a·b / (‖a‖‖b‖) = 5/(3.16×2.24) ≈ 0.71	Text and embedding similarity (search, RAG)

7.3 Matrices



A dataset with 4 rows and 3 features is a 4 × 3 matrix X.

- Shape (rows × columns); element a_{ij} is row i, column j.
- Special matrices: square, identity I (1s on the diagonal), diagonal, symmetric ($A = A^T$), zero.
- Transpose A^T flips rows and columns.
- Addition and scalar multiplication work element by element (same shape required).

```
import numpy as np
X = np.array([[25, 3.1, 2], [32, 4.5, 5], [47, 8.2, 1], [51, 9.0, 7]])
X.shape        # (4, 3)
X.T.shape      # (3, 4)
a, b = np.array([3, 1]), np.array([1, 2])
a @ b          # 5  dot product
np.linalg.norm(a - b)   # 2.236 distance
```

Building Blocks ML · Class 8

Building Blocks ML · Class 8: Introduction to Mathematics (Part 3/5) – matrix operations

8.1 Matrix multiplication

(m × n) · (n × p) → (m × p). Inner sizes must match. Each output cell = dot product of a row of the first with a column of the second. Order matters: AB ≠ BA in general.

Real-world example: billing

Quantities Q (2 customers × 3 products) = [[2,1,0],[0,3,1]]; prices P (3 × 1) = [[500],[200],[1000]]. Bill = Q·P = [[2×500 + 1×200 + 0], [0 + 3×200 + 1×1000]] = **[[1,200],[1,600]]**. Excel: `=MMULT(Q,P)`. All predictions of a linear model at once: ŷ = Xw.

8.2 Determinant and inverse

- For A = [[a,b],[c,d]], det(A) = ad − bc. If det = 0 the matrix is **singular** (no inverse): its columns are redundant (perfect multicollinearity).
- Inverse A^{-1} satisfies $A \cdot A^{-1} = I$. For 2×2: $A^{-1} = (1/\det) \cdot [[d,-b],[-c,a]]$. Excel `=MINVERSE()`, `=MDETERM()`.

8.3 Systems of linear equations

2x + y = 1,100 and x + 3y = 1,800 (two baskets' totals) → A·[x,y] = b → [x,y] = A^{-1}b = [300, 500]: item prices ₹300 and ₹500.

Linear regression's exact solution is the same idea: $\mathbf{w = (X^T X)^{-1} X^T y}$ (the "normal equation").

```
A = np.array([[2, 1], [1, 3]]); b = np.array([1100, 1800])
np.linalg.solve(A, b)          # array([300., 500.])
np.linalg.det(A)               # 5.0
np.linalg.inv(A)
```

8.4 Eigenvalues and eigenvectors (intuition)

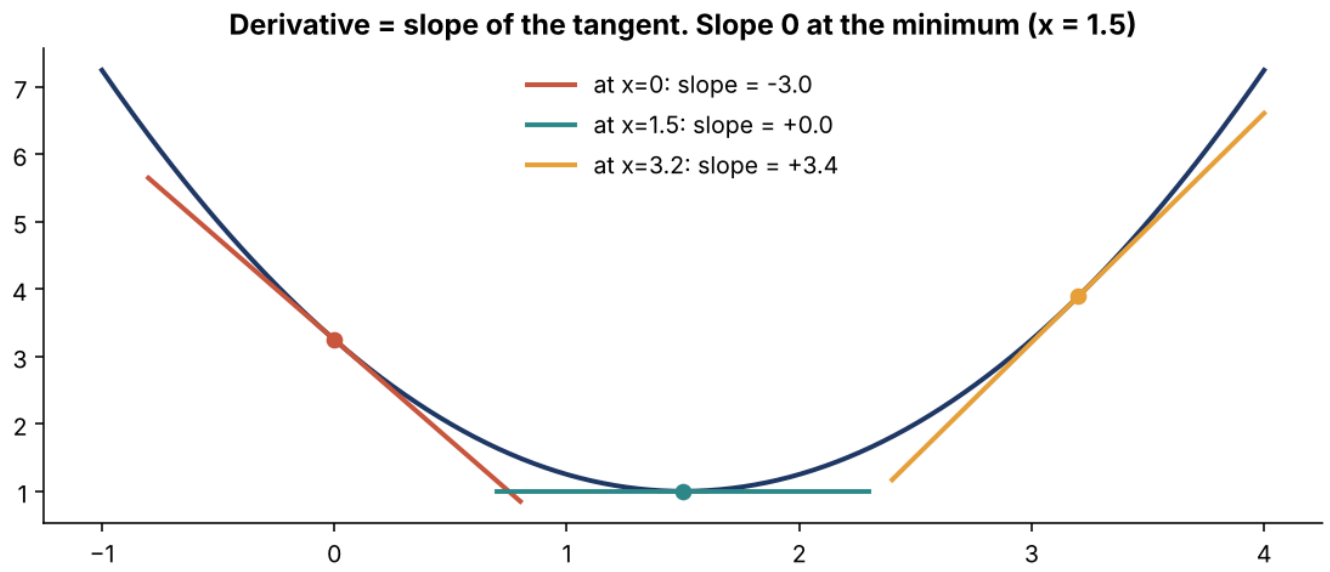
For a square matrix A, an eigenvector v keeps its direction when multiplied: Av = λv; λ is how much it is stretched. In data, the eigenvectors of the covariance matrix point along the directions of greatest spread, which is exactly what **PCA** uses to compress many correlated features into a few components.

Building Blocks ML · Class 9

Building Blocks ML · Class 9: Introduction to Mathematics (Part 4/5) – calculus for optimisation

9.1 Limits and the idea of a derivative

A derivative is the instantaneous rate of change: the slope of a curve at a point. f′(x) = lim(h→0) [f(x+h) − f(x)] / h.



Negative slope left of the minimum, zero at the minimum, positive to the right. Gradient descent always steps against the slope.

9.2 Differentiation rules

Rule	Formula	Example
Constant	$d/dx(c) = 0$	$d/dx(7) = 0$
Power	$d/dx(x^n) = n \cdot x^{n-1}$	$d/dx(x^3) = 3x^2$
Constant multiple / sum	$(c \cdot f)' = c \cdot f'$; $(f + g)' = f' + g'$	$d/dx(4x^2 + 3x) = 8x + 3$
Product	$(fg)' = f'g + fg'$	Revenue = $p \cdot q(p)$
Chain	$d/dx\ f(g(x)) = f'(g(x)) \cdot g'(x)$	$d/dx\ (2x+1)^2 = 2(2x+1) \cdot 2$
Exponential / log	$(e^x)' = e^x$; $(\ln x)' = 1/x$	Used in logistic regression
Sigmoid	$\sigma'(z) = \sigma(z)(1 - \sigma(z))$	Back-propagation in neural nets

9.3 Maxima, minima and optimisation

Business example: best price

Demand $q = 1{,}000 - 20p$. Revenue $R = p \cdot q = 1{,}000p - 20p^2$. $R' = 1{,}000 - 40p = 0 \rightarrow$ **p = ₹25**, q = 500, revenue ₹12,500. $R'' = -40 < 0$ confirms a maximum.

9.4 Partial derivatives and the gradient

With several inputs, a partial derivative $\partial f/\partial x$ changes one input while holding the others fixed. The **gradient** ∇f is the vector of all partials; it points uphill. For the regression cost $J(b_0,b_1) = (1/n)\Sigma(b_0 + b_1x - y)^2$:

$$\partial J/\partial b_0 = (2/n)\Sigma(\hat{y} - y) \qquad \partial J/\partial b_1 = (2/n)\Sigma(\hat{y} - y)\cdot x$$

Gradient descent update: $b \leftarrow b - \alpha\cdot\nabla J$. See it run on real numbers in **ML Class 5**.

9.5 Integrals (one idea)

An integral is the area under a curve: probabilities for continuous distributions are areas under the density curve ($P(Z \le 1.5) = 0.933$).

Building Blocks ML · Class 10

Building Blocks ML · Class 10: Introduction to Mathematics (Part 5/5) – probability and sets for ML

10.1 Sets

Union $A \cup B$ (either), intersection $A \cap B$ (both), complement A^c, difference $A - B$. Venn diagrams. Same ideas as SQL UNION / INTERSECT / EXCEPT and pandas merges.

10.2 Counting: permutations and combinations

Factorial n!	5! = 120 ways to order 5 items
Permutations nPr = n!/(n−r)!	Order matters: ways to pick 1st, 2nd, 3rd prize from 10 = 720
Combinations nCr = n!/(r!(n−r)!)	Order doesn't matter: ways to choose 3 features out of 10 = 120 (Excel `=COMBIN(10,3)`)

10.3 Conditional probability, independence, Bayes

$P(A|B) = P(A\cap B)/P(B)$. Independent if $P(A|B) = P(A)$. Bayes: $P(A|B) = P(B|A)P(A)/P(B)$, the basis of **Naive Bayes** spam filters: P(spam | "free offer") from how often "free offer" appears in spam vs genuine mail.

10.4 Random variables, expectation, variance

- $E[X] = \Sigma\, x\cdot P(x)$ (discrete) — expected profit of a campaign.
- $Var(X) = E[X^2] - (E[X])^2$.
- Joint and marginal distributions; covariance between two random variables.

Real-world example: expected value of a decision

A retention offer costs ₹200. If offered, 30% of at-risk customers stay, each worth ₹2,000 → expected gain = 0.3×2,000 − 200 = **₹400 per customer**. Positive, so offer it. Churn models supply the probabilities.

10.5 Likelihood and maximum likelihood (intuition)

Likelihood asks: given these parameters, how probable is the data we actually saw? Logistic regression picks coefficients that make the observed 0/1 outcomes most likely; minimising **log-loss** is the same thing.

Interview Q&A

Q Compiler vs interpreter?

A compiler translates the whole program into machine code before running (fast execution, all errors upfront); an interpreter translates and runs line by line (instant feedback, stops at the first error). Python is interpreted (via bytecode).

Q What is pseudo-code and why use it?

Plain-language step-by-step logic written before coding. It lets you design and check the algorithm without worrying about syntax.

Q What is a confidence interval?

A range built from sample data that, by the method used, captures the true population value a stated share of the time (e.g. 95%).

Q Why is the dot product important in ML?

A linear model's prediction, a neural-network neuron's input, and cosine similarity between embeddings are all dot products.

Q What does a zero determinant mean for regression?

X^TX is not invertible because some features are exact linear combinations of others (perfect multicollinearity), so coefficients cannot be uniquely estimated.

Q Why do we need derivatives in ML?

To find which direction to change parameters to reduce error; gradient descent uses the gradient of the cost function.

Q Explain Bayes' theorem with an example.

It updates a prior belief with evidence. With 1% fraud and a test that is 95% sensitive with a 5% false-alarm rate, only about 16% of flagged transactions are fraud.

Resources

- ▶ YouTube: Simplilearn programming basics for beginners · ▶ YouTube: CS50 lecture 0 scratch computational thinking

- ▶ YouTube: Simplilearn statistics for data science · ▶ YouTube: StatQuest confidence intervals clearly explained
- ▶ YouTube: 3Blue1Brown essence of linear algebra · ▶ YouTube: 3Blue1Brown essence of calculus
- ▶ YouTube: Khan Academy linear algebra · ▶ YouTube: Khan Academy probability and combinatorics
- ▶ YouTube: Simplilearn mathematics for machine learning
- 📘 Mathematics for Machine Learning (free book, Deisenroth et al.)
- 📘 Python official tutorial

COURSE HANDBOOK 03

🐍 Python for Data Science

Core Python, NumPy, pandas, visualisation, statistics and first ML models

Python for Data Science 2026 (24 classes + 2 tests)

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#m2**

Overview

Python turns repeatable analysis into code. This course moves from core syntax to pandas data wrangling, charts, statistical testing and the first two ML models (linear and logistic regression).

Class	Topics	Jump to
1	Introduction to Python, Jupyter Notebook	Core Python
2	Type conversion, print, f-strings, format, input	Core Python
3	if / elif / else, while loops	Core Python
4	for loops, OOP basics, string methods, indexing and slicing	Core Python
5	Lists	Data structures
6	Dictionaries, tuples, sets	Data structures
7	User-defined functions, lambda, map, filter	Functions & files
8	File handling, modules	Functions & files
9–10	NumPy arrays and operations; reading data with pandas	NumPy
11–12	DataFrames: subsetting, reordering, renaming, dropping, sorting, filtering	pandas
13–14	Type conversion, univariate analysis, aggregates, missing values	pandas
15	Joins, groupby, pivot_table	pandas
16–17	Visualisation; working with dates	Visualisation
18	Datetime, outliers, connecting to SQL	Dates, outliers, SQL
19	Subplots, probability distributions, normal distribution, CLT intro	Statistics
20	Sampling, CLT, introduction to hypothesis testing	Statistics
21	z-test, t-test, chi-square, ANOVA	Statistics
22	Introduction to machine learning	Regression
23	Linear regression	Regression
24	Logistic regression	Regression

> **Tip** Case studies for this course (Retail, Credit Card, Insurance, Visualisation, Hypothesis Testing, E-commerce capstone and the two exercise notebooks) are fully solved on the real data under **Case Studies**.

Core Python

Jupyter basics

A notebook mixes **code cells** and **markdown cells**. `Shift` + `Enter` runs a cell; `Esc` then `A` / `B` inserts a cell above/below; `Esc` `M` makes markdown. The last expression in a cell is displayed automatically.

Variables, types and conversion (Class 2)

```
age = 34                # int
price = 99.5            # float
name = "Asha"           # str
active = True           # bool
type(price)             # <class 'float'>
int("42") + 1           # 43   (str -> int)
float("3.5"); str(10)   # 3.5, '10'
int(7.9)                # 7    (truncates, does not round)
print(f"{name} is {age} and paid ₹{price:,.2f}")   # f-string with format spec
print("{} scored {:.1f}%".format(name, 87.456))     # str.format
city = input("City? ")   # always returns a string
```

Operators and precedence

`**` → `* / // %` → `+ -` → comparisons → `not` → `and` → `or`. Example from the exercise: `5 + 4 * 9 % (3 + 1) / 6 - 1` = 4.0, because 36 % 4 = 0.

Conditions and loops (Classes 3–4)

```
score = 72
if score >= 80:
    grade = "A"
elif score >= 60:
    grade = "B"
else:
    grade = "C"

n = 5
while n > 0:          # repeat until the condition is False
    n -= 1

for i in range(1, 11, 2):   # 1,3,5,7,9  (start, stop-exclusive, step)
    if i == 7: break        # leave the loop
    if i == 3: continue     # skip to next iteration
    print(i)
```

Strings: indexing, slicing, methods (Class 4)

```
s = "Data Science"
s[0], s[-1]          # 'D', 'e'
s[0:4], s[5:], s[::-1]   # 'Data', 'Science', 'ecneicS ataD'
s.lower(), s.upper(), s.title()
s.replace("Data", "Big"), s.split(" ")      # 'Big Science', ['Data','Science']
"  hi ".strip(), s.find("S"), s.count("a"), s.startswith("Da")
```

OOP in one example

```
class Account:
    def __init__(self, owner, balance=0):   # constructor
        self.owner, self.balance = owner, balance
    def deposit(self, amt):                 # method
        self.balance += amt
        return self.balance
acc = Account("Asha"); acc.deposit(500)     # 500
```

Tip Everything in Python is an object. `"abc".upper()` is calling the `upper` method of a string object, which is why pandas code reads as `df.groupby(...).sum()`.

Data structures

	list	tuple	dict	set
Syntax	`[1, 2, 2]`	`(1, 2)`	`{"a": 1}`	`{1, 2}`
Ordered	Yes	Yes	Yes (insertion)	No
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Keys unique	No
Use for	Sequences you change	Fixed records, dict keys	Lookups by key	Unique values, membership

```
# Lists (Class 5)
nums = [5, 7, 22, 97]
nums.append(54); nums.extend([1, 2]); nums.insert(0, 99)
nums.remove(22); last = nums.pop(); nums.sort(reverse=True)
squares = [x**2 for x in nums if x % 2]      # list comprehension with a filter

# Dict, tuple, set (Class 6)
emp = {"name": "Ravi", "dept": "Ops"}
emp["salary"] = 50000; emp.get("age", "n/a")
for k, v in emp.items(): print(k, v)
point = (3, 4); x, y = point                  # tuple unpacking
a, b = {1, 2, 3}, {2, 3, 4}
a & b, a | b, a - b                           # intersection, union, difference
```

Functions & files

User-defined functions (Class 7)

```
def describe(*nums, decimals=2):           # *args collects positional values
    """Return basic descriptives."""
    mean = sum(nums) / len(nums)
    return {"count": len(nums), "mean": round(mean, decimals), "max": max(nums)}

def area(shape, **dims):                    # **kwargs collects named values
    if shape == "square": return dims["side"] ** 2
    if shape == "circle": return 3.1416 * dims["radius"] ** 2

square = lambda x: x * x                    # anonymous one-line function
list(map(square, [1, 2, 3]))                # [1, 4, 9]
list(filter(lambda x: x % 2, [1, 2, 3, 4]))  # [1, 3]
```

Tip Scope: variables created inside a function are local. Return values instead of printing them, so the result can be reused.

Files and modules (Class 8)

```
with open("notes.txt", "w") as f:     # 'w' write, 'a' append, 'r' read
    f.write("line 1\n")
with open("notes.txt") as f:
    lines = f.readlines()

import math, os
from datetime import date
import numpy as np                     # alias convention
```

The `with` block closes the file automatically, even if an error happens.

Errors

```
try:
    value = int("abc")
except ValueError as e:
    print("Not a number:", e)
finally:
    print("always runs")
```

NumPy

NumPy arrays hold one data type and support **vectorised** maths: an operation applies to every element without a Python loop, which is 10–100× faster.

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
```

```
a.shape, a.ndim, a.dtype        # (2, 3), 2, int64
np.zeros((2, 3)); np.ones(3); np.arange(0, 10, 2); np.linspace(0, 1, 5)
np.random.seed(1); np.random.randint(1, 10, size=5); np.random.normal(0, 1, 100)
a * 10; a + a; a.T              # element-wise maths and transpose
a[0, 1]; a[:, 1]; a[a > 3]      # indexing, column slice, boolean mask
a.sum(axis=0); a.mean(axis=1)   # axis=0 down columns, axis=1 across rows
a.reshape(3, 2); np.where(a > 3, "high", "low")
a @ a.T                          # matrix multiplication
```

Tip Broadcasting: NumPy stretches smaller arrays to match shapes, so `a - a.mean(axis=0)` subtracts each column's mean from that column.

pandas

Reading and inspecting (Classes 10–11)

```
import pandas as pd
df = pd.read_csv("Transactions.csv")            # also read_excel, read_sql, read_json
df.head(); df.tail(); df.shape; df.info(); df.describe()
df.dtypes; df.columns; df["Store_type"].value_counts(); df.nunique()
```

Selecting, filtering, sorting (Classes 11–12)

```
df["Qty"]                        # Series
df[["cust_id", "total_amt"]]     # DataFrame
df.loc[df["total_amt"] > 0, ["cust_id", "Qty"]]   # label-based + boolean mask
df.iloc[0:5, 0:3]                # position-based
df.query("Store_type == 'e-Shop' and Qty > 2")
df[df["city"].isin(["Pune", "Delhi"])]
df.rename(columns={"Qty": "quantity"}); df.drop(columns=["Tax"])
df.sort_values(["Store_type", "total_amt"], ascending=[True, False])
df = df[["cust_id", "tran_date", "total_amt"]]    # reorder columns
```

Cleaning (Classes 13–14)

```
df["amt"] = df["amt"].str.replace("$", "", regex=False).astype(float)
df["date"] = pd.to_datetime(df["date"], dayfirst=True)
df.isna().sum()
df["age"] = df["age"].fillna(df["age"].mean())         # continuous -> mean/median
df["city"] = df["city"].fillna(df["city"].mode()[0])   # categorical -> mode
df.dropna(subset=["cust_id"]); df.drop_duplicates("cust_id", keep="last")
```

Aggregating, joining, reshaping (Class 15)

```
df.groupby("Store_type")["total_amt"].agg(["sum", "mean", "count"])
df.groupby(["city", "gender"], as_index=False).agg(spend=("total_amt", "sum"), orders=("tran_id",
"nunique"))
pd.merge(tran, cust, how="left", left_on="cust_id", right_on="customer_Id")   #
```

```
inner/left/right/outer
pd.concat([df_2015, df_2016], ignore_index=True)                                  # stack rows
df.pivot_table(index="city", columns="Store_type", values="total_amt", aggfunc="sum", fill_value=0)
wide.melt(id_vars="id", var_name="year", value_name="sales")                      # wide -> long
s.unstack(); df.stack()
```



> **Tip** After every merge, check row counts. If rows increase unexpectedly, the right-hand key was not unique (a many-to-many join).

apply and lambda

```
df["band"] = df["age"].apply(lambda a: "Youth" if a < 30 else "Adult")
df["band"] = pd.cut(df["age"], [0, 18, 30, 60, 120], labels=["Child", "Youth", "Adult", "Senior"])
df["flag"] = np.where((df["type"] == "Injury") & (df["police"] == "No"), 1, 0)
```

Visualisation

Which chart for which question

Question	Chart	Code
Distribution of one number	Histogram, box plot	`df["amt"].plot.hist(bins=30)`
Compare categories	Bar (horizontal if labels are long)	`s.plot.barh()`
Trend over time	Line	`ts.plot(marker="o")`
Relationship of two numbers	Scatter	`df.plot.scatter("x", "y")`
Part of a whole (few parts)	Pie / donut, or 100% stacked bar	`s.plot.pie(autopct="%1.1f%%")`
Two categories × value	Grouped bar, heatmap	`sns.heatmap(pivot, annot=True)`

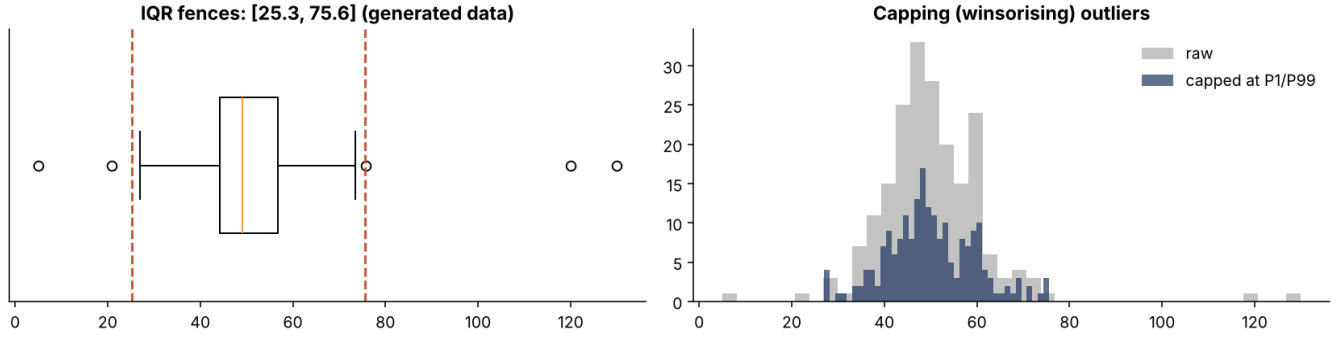
```
import matplotlib.pyplot as plt, seaborn as sns
fig, ax = plt.subplots(1, 2, figsize=(12, 4))        # subplots grid (Class 19)
ax[0].plot(months, revenue, "r:o", label="Revenue")  # colour red, dotted, circle markers
ax[0].set(title="Revenue", xlabel="Month", ylabel="₹"); ax[0].legend(loc="lower right")
sns.boxplot(data=df, x="segment", y="claim_amount", ax=ax[1])
plt.tight_layout(); plt.savefig("chart.png", dpi=150)
```

Dates, outliers, SQL

Dates (Classes 17–18)

```
df["dt"] = pd.to_datetime(df["dt"], format="%d-%m-%Y")      # explicit format is safer
df["dt"].dt.year, df["dt"].dt.month, df["dt"].dt.day_name(), df["dt"].dt.quarter
df["dt"].dt.to_period("M")                                   # month bucket for grouping
(df["resolved"] - df["received"]).dt.days                    # durations
df.set_index("dt").resample("W")["sales"].sum()              # weekly totals
pd.Timestamp("2018-10-01") - pd.Timedelta(days=20)
pd.to_datetime("07Sep59", format="%d%b%y")                   # 2059! subtract 100 years if needed
```

Outliers



IQR rule: values below Q1 − 1.5·IQR or above Q3 + 1.5·IQR are flagged. Capping at the 1st/99th percentile keeps the rows but limits their pull.

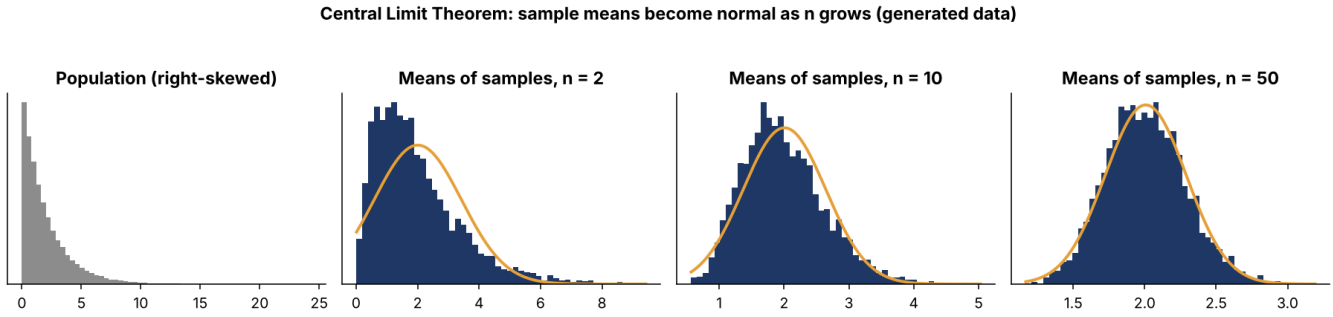
```
q1, q3 = df["amt"].quantile([.25, .75]); iqr = q3 - q1
mask = df["amt"].between(q1 - 1.5 * iqr, q3 + 1.5 * iqr)
df["amt_capped"] = df["amt"].clip(df["amt"].quantile(.01), df["amt"].quantile(.99))
z = (df["amt"] - df["amt"].mean()) / df["amt"].std(); df[z.abs() > 3]   # z-score rule
```

Connecting to SQL

```
import sqlalchemy as sa
engine = sa.create_engine("mssql+pyodbc://@SERVER/RetailDB?
driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes")
df = pd.read_sql("SELECT * FROM Transactions WHERE Qty > 0", engine)
df.to_sql("clean_transactions", engine, if_exists="replace", index=False)
```

Statistics in Python

Distributions and the CLT (Classes 19–20)



Even from a skewed population, the averages of samples form a normal curve. Its spread is the standard error, σ/√n.

```python
from scipy import stats
stats.norm.cdf(1.96)            # 0.975  area to the left
stats.norm.ppf(0.95)            # 1.645  z for 95% one-tailed
stats.binom.pmf(3, n=10, p=.2)  # P(exactly 3 successes)
stats.poisson.pmf(2, mu=4)      # P(2 events when 4 expected)
sample = df["amt"].sample(100, random_state=1)   # simple random sampling
df.groupby("city", group_keys=False).apply(lambda g: g.sample(frac=.1))   # stratified
```

Choosing a test (Class 21)

Question	Data	Test	scipy
Is the mean equal to a target?	1 numeric sample	One-sample t (z if σ known, n large)	`ttest_1samp(x, 10000)`
Do two groups differ?	Numeric × 2 independent groups	Two-sample t (Welch)	`ttest_ind(a, b, equal_var=False)`
Before vs after on the same units?	Paired numeric	Paired t	`ttest_rel(before, after)`
Do 3+ groups differ?	Numeric × 3+ groups	One-way ANOVA	`f_oneway(g1, g2, g3)`
Are two categories related?	Category × category counts	Chi-square independence	`chi2_contingency(pd.crosstab(a, b))`
Are two numbers related?	Numeric × numeric	Pearson (or Spearman for ranks)	`pearsonr(x, y)`

Hypothesis testing is covered in depth, with pictures, in ML Classes 1–2.

Regression models

Machine learning in one line (Class 22)

Learn a function *f* from past examples (X → y) so it predicts y for new X. **Regression** predicts a number; **classification** predicts a class.

Linear regression (Class 23)

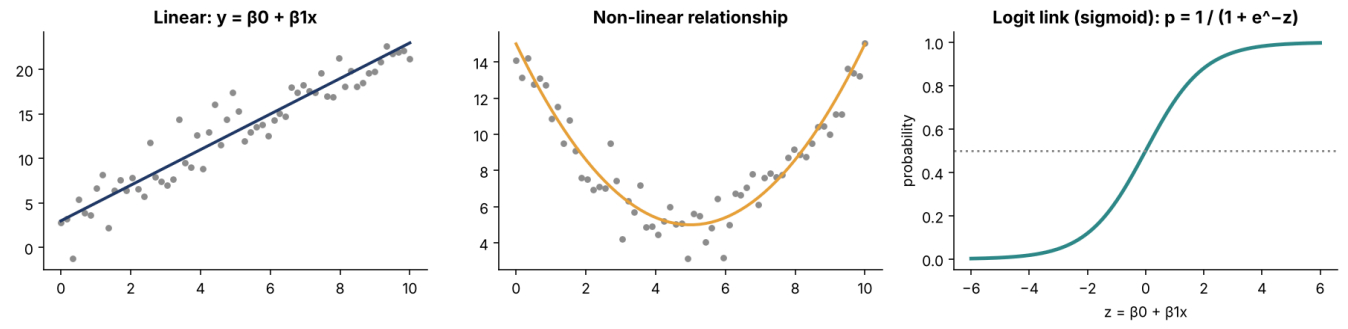
$\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p$. The βs are chosen to minimise the sum of squared errors (OLS).

```
import statsmodels.formula.api as smf
m = smf.ols("sales ~ tv + radio + C(region)", data=df).fit()
print(m.summary())          # coefficients, p-values, R², adjusted R²

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=.3, random_state=42)
lr = LinearRegression().fit(X_tr, y_tr); pred = lr.predict(X_te)
r2_score(y_te, pred), mean_absolute_error(y_te, pred), mean_squared_error(y_te, pred) ** .5
```

Assumption	Check	If violated
Linearity	Residuals vs fitted plot shows no pattern	Transform x (log, square) or use a non-linear model
Independence of errors	Durbin–Watson ≈ 2	Add lag features (time series)
Constant variance (homoscedasticity)	No funnel in residual plot	Log-transform y
Normal errors	Q-Q plot of residuals	Transform y; large n is forgiving
No multicollinearity	VIF < 5 (or 10)	Drop or combine correlated features

Logistic regression (Class 24)



Logistic regression feeds a linear score through the sigmoid to get a probability between 0 and 1.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score
```

```
clf = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)
prob = clf.predict_proba(X_te)[:, 1]; pred = (prob >= 0.5).astype(int)
print(classification_report(y_te, pred)); roc_auc_score(y_te, prob)
np.exp(clf.coef_)        # odds ratios: how much the odds multiply per unit increase
```

Metrics (confusion matrix, precision, recall, ROC/AUC) are explained with pictures in ML Class 7.

Interview Q&A

Q List vs tuple?

Lists are mutable and used for collections that change; tuples are immutable, slightly faster, and can be dictionary keys.

Q loc vs iloc?

`loc` selects by label (and accepts boolean masks); `iloc` selects by integer position. `loc` slices include the end label; `iloc` slices exclude the end position.

Q How do you handle missing values?

First find why they are missing. Then drop rows (if few and random), impute (mean/median for numbers, mode for categories, or model-based), or add a "was missing" flag. Never impute the target.

Q merge vs concat vs join?

`merge` joins on key columns (like SQL JOIN); `concat` stacks frames along rows or columns; `join` is a shortcut that merges on the index.

Q What is a p-value?

The probability of seeing data at least this extreme if the null hypothesis were true. Small p (below α, often 0.05) means the data is unlikely under H0, so we reject H0. It is not the probability that H0 is true.

Q R^2 vs adjusted R^2?

R^2 is the share of variance explained and never falls when you add features. Adjusted R^2 penalises extra features, so it only rises if a new feature genuinely helps.

Q Why is logistic regression called regression if it classifies?

It regresses the log-odds of the outcome on the features linearly; classification only happens when you apply a cut-off to the predicted probability.

Resources

- ▶ YouTube: Corey Schafer Python tutorial for beginners, clear core Python.
- ▶ YouTube: Corey Schafer pandas tutorial and ▶ YouTube: Keith Galli complete pandas tutorial, hands-on pandas.
- ▶ YouTube: codebasics numpy tutorial and ▶ YouTube: codebasics matplotlib tutorial.
- ▶ YouTube: StatQuest linear regression clearly explained and ▶ YouTube: StatQuest logistic regression.
- 📘 pandas user guide · 📘 10 minutes to pandas
- 📘 SciPy stats reference · 📘 scikit-learn user guide

COURSE HANDBOOK 04

📊 Data Visualization & Analytics

Advanced Excel, SQL Server and AI-assisted analytics (Power BI has its own section)

Data Visualization & Analytics 2026 (BT DVA APR 2026, 24 classes + tests)

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#m3**

Overview

DVA is the analyst toolkit: Excel for quick analysis and dashboards, SQL for querying databases, Power BI for interactive reporting, and newer AI tools that write formulas and queries for you. Power BI (Classes 18–21) is covered in its own **Power BI** section.

Class	Topics
1	Orientation
2	Excel basics, data validation, formatting, custom and conditional formatting
3	Aggregate functions (SUM, COUNT, COUNTA, COUNTBLANK, AVERAGE, MIN, MAX, SMALL, LARGE) and text functions (LEFT, RIGHT, MID, CONCAT, TEXTJOIN, LOWER, UPPER, PROPER, LEN, TRIM, FIND...)
4	IF, IFS, nested IF; array functions; sort and filter; remove duplicates; UNIQUE; SUMPRODUCT
5	Reverse lookup, INDEX/MATCH, CHOOSE, VLOOKUP + CHOOSE; pivot tables (cache, areas, filters, grand totals)
6	Pivot refresh, grouping, bins, subtotals, MoM/YoY, calculated fields; Video Game Sales case
8	Named ranges (static/dynamic); thermometer, Pareto, population pyramid and waterfall charts; Superstore case
9	Dashboards, protection, Power Pivot, Power Query, Data Analysis ToolPak, Evaluate Formula
10	SQL basics, ER diagrams, relationships, keys; DDL/DML/DQL/DCL/TCL
11	Data types, variables, operators, comments; CREATE, ALTER, DROP, TRUNCATE; DML
12	Constraints; DELETE vs DROP vs TRUNCATE; aliases; WHERE, ORDER BY, IN, LIKE, IS NULL
13	GROUP BY / HAVING; aggregate, text, date and number functions; DATEPART; CASE WHEN; joins
14	Joins case; UNION, UNION ALL, INTERSECT, EXCEPT; subqueries
15	OFFSET/FETCH; window functions: OVER, PARTITION BY, RANK, DENSE_RANK, ROW_NUMBER, aggregate windows
16	LEAD/LAG, EXISTS, correlated subqueries
17	CTEs, views, stored procedures
18	ROLLUP, CUBE, COALESCE; start of Power BI
19–21	Power BI (see **Power BI**)

Class	Topics
22	Unpivot; GenAI introduction, prompt engineering; AI vs GenAI vs agentic AI; AI in SQL; Docker; Metabase; MinusX
23	Metabase and MinusX
24	Gemini in Excel, Copilot for Excel, Frontier Office Agent

Assessments: DVA MCQ/SQL test, DVA case study test, integrated DVA case study, DVA viva, integrated MCQ.

Excel: formulas

Logic (Class 4)

```
=IF(C2>=80,"A",IF(C2>=60,"B","C"))              nested IF
=IFS(C2>=80,"A", C2>=60,"B", TRUE,"C")           IFS: first TRUE wins; TRUE acts as "else"
=IF(AND(B2="Male",C2>30),"Yes","No")             AND / OR inside IF
=IFERROR(VLOOKUP(A2,Tbl,2,0),"Not found")        trap errors
```

Lookups (Class 5)

Need	Formula
Classic lookup (result to the right)	`=VLOOKUP(A2, Products!$A:$D, 3, FALSE)`
Horizontal table	`=HLOOKUP(J2, LOCATION!$B$2:$M$3, 2, FALSE)`
Any direction (reverse lookup)	`=INDEX(Sheet!$A:$A, MATCH(R2, Sheet!$B:$B, 0))`
Two-way lookup	`=INDEX($B$2:$M$20, MATCH(name,$A$2:$A$20,0), MATCH(month,$B$1:$M$1,0))`
VLOOKUP leftwards with CHOOSE	`=VLOOKUP(R2, CHOOSE({1,2}, Sheet!$B:$B, Sheet!$A:$A), 2, 0)`
Modern (365)	`=XLOOKUP(A2, Ids, Names, "Not found")`

Arrays, dynamic arrays, SUMPRODUCT

```
=UNIQUE(B2:B500)                        distinct list (spills)
=SORT(FILTER(A2:D500, C2:C500="East"), 4, -1)
=SUMPRODUCT((B2:B500="East")*(D2:D500))  conditional sum without SUMIFS
=SUMIFS(D:D, B:B, "East", C:C, ">=2024-01-01")
=COUNTIFS(K:K, $G5, I:I, H$4)           cross-tab counts
=TEXTJOIN(", ", TRUE, UNIQUE(B2:B50))
```

Text cleaning (Class 3)

```
=PROPER(TRIM(A2))                       tidy names
=LEFT(A2, FIND(" ", A2) - 1)            text before first space
=MID(A2, FIND("@", A2) + 1, 100)        domain from email
```

```
=SUBSTITUTE(A2, "$", "") * 1              strip a symbol and convert to number
=TEXT(A2, "000")   =TEXT(B2, "dd mmm' yyyy")
```

Excel: pivots & charts

Pivot tables (Classes 5–6)

- The pivot reads from a **pivot cache**, so changes to source data need **Refresh** (Alt+F5). Format the source as a Table so new rows are included.
- Four areas: **Filters, Columns, Rows, Values**. Drag a field twice into Values to show count and %.
- **Group** dates into Years/Quarters/Months; group numbers into **bins** (0–100, 100–200...).
- **Show Values As**: % of Grand Total, % of Row, Difference From (previous) for MoM/YoY, Running Total.
- **Calculated field**: Profit % = Profit / Sales (operates on sums).
- Design tab: Report Layout → Tabular, Repeat item labels, Subtotals off, Grand Totals off.
- Slicers and timelines filter several pivots at once (Report Connections).

Special charts (Class 8)

Chart	Purpose	How
Pareto	Find the vital few (80/20)	Sort desc, cumulative % on a secondary-axis line; or Insert → Histogram → Pareto
Waterfall	Bridge from start to end value	Insert → Waterfall; set totals as "Set as total"
Thermometer	Progress to target	Clustered column of target and actual, 100% overlap, actual on top
Population pyramid	Two groups by age band	Make one series negative, bar chart, overlap 100%, number format `0;0`

Named ranges

```
Static:  Formulas → Define Name → SalesData = Sheet1!$A$1:$D$500
Dynamic: =OFFSET(Sheet1!$A$1,0,0,COUNTA(Sheet1!$A:$A),4)   (grows with data)
Better:  convert to a Table and use structured references: =SUM(Sales[Amount])
```

Dashboards and the power tools (Class 9)

- **Dashboard**: one sheet, KPIs at the top, 3–5 charts, slicers on the side, gridlines off, consistent colours.
- **Protection**: Review → Protect Sheet (lock formulas), Protect Workbook (structure).
- **Power Query**: repeatable cleaning: import → remove columns → change types → unpivot → merge → Close & Load. Click Refresh next month and every step reruns.
- **Power Pivot**: data model with relationships across tables and DAX measures (same engine as Power BI).
- **Analysis ToolPak**: descriptive statistics, histogram, regression, t-test in a dialog.
- **Evaluate Formula** (Formulas tab) steps through a formula to debug it.

SQL: core

Data types and DDL (Classes 10–12)

```sql
CREATE TABLE Orders (
    order_id    INT IDENTITY(1,1) PRIMARY KEY,
    customer_id INT NOT NULL FOREIGN KEY REFERENCES Customer(customer_id),
    order_date  DATE NOT NULL,
    amount      DECIMAL(10,2) CHECK (amount >= 0),
    channel     VARCHAR(20) DEFAULT 'Web'
);
ALTER TABLE Orders ADD discount DECIMAL(5,2);
ALTER TABLE Orders ALTER COLUMN channel VARCHAR(30);
DECLARE @cutoff DATE = '2024-01-01';          -- variable
SELECT * FROM Orders WHERE order_date >= @cutoff;  -- single-line comment
/* multi-line
   comment */
```

	DELETE	TRUNCATE	DROP
Removes	Chosen rows (WHERE)	All rows	Table + data
Type	DML	DDL	DDL
Identity reset	No	Yes	n/a
Triggers fire	Yes	No	No
Speed	Slower (row-logged)	Fast	Fast

Aggregation and CASE (Class 13)

```sql
SELECT  c.city,
        COUNT(DISTINCT o.order_id)                AS orders,
        SUM(o.amount)                             AS revenue,
        AVG(o.amount)                             AS aov,
        SUM(CASE WHEN o.amount > 1000 THEN 1 ELSE 0 END) AS big_orders
FROM    Orders o
JOIN    Customer c ON o.customer_id = c.customer_id
WHERE   YEAR(o.order_date) = 2024                 -- filters rows (before grouping)
GROUP BY c.city
HAVING  SUM(o.amount) > 50000                     -- filters groups (after grouping)
ORDER BY revenue DESC;
```

Useful functions

Date	`GETDATE()`, `DATEPART(QUARTER, d)`, `DATENAME(WEEKDAY, d)`, `DATEADD(DAY, -30, d)`, `DATEDIFF(MONTH, a, b)`, `EOMONTH(d)`, `FORMAT(d,'yyyy-MM')`
Text	`LEN`, `LEFT`, `RIGHT`, `SUBSTRING`, `CHARINDEX`, `REPLACE`, `UPPER`, `LOWER`, `TRIM`, `CONCAT`, `STRING_AGG`
Number	`ROUND`, `CEILING`, `FLOOR`, `ABS`, `POWER`, `SQRT`

Conversion	`CAST(x AS DATE)`, `CONVERT(DATE, '28-02-2014', 105)`, `TRY_CONVERT`, `ISNULL(x,0)`, `COALESCE(a,b,c)`

Joins and set operators (Classes 13–14)



Also: CROSS JOIN (every combination) and SELF JOIN (a table joined to itself, such as employee → manager).

```sql
SELECT name FROM Customers_2023
UNION        SELECT name FROM Customers_2024;   -- distinct rows from both
-- UNION ALL keeps duplicates (faster); INTERSECT = in both; EXCEPT = in first only

-- Subqueries
SELECT * FROM Orders WHERE amount > (SELECT AVG(amount) FROM Orders);                -- scalar
SELECT * FROM Customer WHERE customer_id IN (SELECT customer_id FROM Orders);         -- list
SELECT city, AVG(rev) FROM (SELECT customer_id, city, SUM(amount) rev FROM ... GROUP BY ...) t
GROUP BY city;  -- derived table
```

SQL: advanced

Window functions (Class 15)

A window function computes across related rows **without collapsing them**, unlike GROUP BY.

```sql
SELECT customer_id, order_date, amount,
       ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date)        AS order_seq,
       RANK()       OVER (ORDER BY amount DESC)                                AS rnk,        --
1,2,2,4
       DENSE_RANK() OVER (ORDER BY amount DESC)                                AS dense_rnk,  --
1,2,2,3
       SUM(amount)  OVER (PARTITION BY customer_id)                            AS cust_total,
       SUM(amount)  OVER (PARTITION BY customer_id ORDER BY order_date
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)    AS running_total,
       AVG(amount)  OVER (ORDER BY order_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS ma_3,
       LAG(amount)  OVER (PARTITION BY customer_id ORDER BY order_date)        AS prev_amount,   --
Class 16
       LEAD(order_date) OVER (PARTITION BY customer_id ORDER BY order_date)    AS next_order
FROM Orders;

-- Top N per group
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (PARTITION BY city ORDER BY revenue DESC) rk FROM CityCustomer
) t WHERE rk <= 3;
```

```sql
-- Paging
SELECT * FROM Orders ORDER BY order_date OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;
```

EXISTS and correlated subqueries (Class 16)

```sql
-- customers who ordered at least once (stops at first match: efficient)
SELECT c.* FROM Customer c WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.customer_id =
c.customer_id);
-- correlated: inner query runs per outer row
SELECT o.* FROM Orders o
WHERE o.amount > (SELECT AVG(o2.amount) FROM Orders o2 WHERE o2.customer_id = o.customer_id);
```

CTEs, views, stored procedures (Class 17)

```sql
WITH monthly AS (
    SELECT FORMAT(order_date,'yyyy-MM') ym, SUM(amount) rev FROM Orders GROUP BY
FORMAT(order_date,'yyyy-MM')
)
SELECT ym, rev, rev - LAG(rev) OVER (ORDER BY ym) AS mom_change,
       100.0 * (rev - LAG(rev) OVER (ORDER BY ym)) / LAG(rev) OVER (ORDER BY ym) AS mom_pct
FROM monthly;

CREATE VIEW vw_city_revenue AS
SELECT c.city, SUM(o.amount) revenue FROM Orders o JOIN Customer c ON o.customer_id = c.customer_id
GROUP BY c.city;

CREATE PROCEDURE usp_top_customers @n INT, @year INT AS
BEGIN
    SELECT TOP (@n) customer_id, SUM(amount) spend
    FROM Orders WHERE YEAR(order_date) = @year
    GROUP BY customer_id ORDER BY spend DESC;
END;
EXEC usp_top_customers @n = 10, @year = 2024;
```

ROLLUP, CUBE, COALESCE (Class 18)

```sql
SELECT COALESCE(region,'All regions') region, COALESCE(category,'All categories') category,
SUM(sales) sales
FROM Sales GROUP BY ROLLUP (region, category);   -- subtotals per region + grand total
-- CUBE(region, category) adds subtotals for every combination (also per category)
```

Unpivot (Class 22)

```sql
SELECT product, month, units
FROM MonthlySales UNPIVOT (units FOR month IN ([Jan],[Feb],[Mar])) u;
```

AI for analytics

AI vs GenAI vs agentic AI (Class 22)

	What it does	Example in analytics
AI / ML	Learns patterns to predict or classify	Churn model, demand forecast
Generative AI	Creates new text, code, images from a prompt	Writes a SQL query or DAX measure from a sentence
Agentic AI	Plans multi-step tasks and uses tools to act	Pulls data, builds the chart, writes the summary, emails it

Prompting for analysis

Role + context + task + format + constraints. Example:

```
You are a SQL Server analyst. Tables: Orders(order_id, customer_id, order_date, amount), Customer(customer_id, city). Write ONE T-SQL query returning the top 3 cities by 2024 revenue with each city's share of total revenue (%), rounded to 1 decimal. Explain each clause in one line.
```

Tip Always check AI-written queries against a small manual calculation. Watch for wrong join keys and for filters placed in HAVING instead of WHERE.

Tools covered (Classes 22–24)

- **Docker**: runs an app in an isolated container; used to start Metabase locally with one command (`docker run -d -p 3000:3000 metabase/metabase`).
- **Metabase**: open-source BI. Connect a database, ask questions with a point-and-click builder or SQL, and pin results to dashboards.
- **MinusX**: an AI assistant that works inside tools such as Metabase and Jupyter to write and run queries from plain-English requests.
- **Gemini in Sheets/Excel and Copilot for Excel**: generate formulas, explain them, create pivots and charts, and summarise data from a prompt.
- **Office agents**: multi-step assistants that work across files (for example, build a workbook, then draft a report from it).

Interview Q&A

Q WHERE vs HAVING?

WHERE filters rows before grouping and cannot use aggregates. HAVING filters groups after GROUP BY and can use aggregates such as SUM().

Q RANK vs DENSE_RANK vs ROW_NUMBER?

For values 100, 90, 90, 80: ROW_NUMBER gives 1,2,3,4; RANK gives 1,2,2,4; DENSE_RANK gives 1,2,2,3.

Q Find the second highest salary.

```
SELECT MAX(salary) FROM Emp WHERE salary < (SELECT MAX(salary) FROM Emp);
-- or: SELECT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) r FROM Emp)
t WHERE r = 2;
```

Q CTE vs temp table vs view?

A CTE exists only for one statement and improves readability. A temp table (#t) stores data for the session and can be indexed. A view is a saved query stored in the database.

Q Why use INDEX/MATCH over VLOOKUP?

It can look left, does not break when columns are inserted, and only reads the two columns it needs.

Q How do you remove duplicates in SQL while keeping one row?

```
WITH d AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY email ORDER BY created DESC) rn FROM
Users)
DELETE FROM d WHERE rn > 1;
```

Q What does a pivot cache mean for you?

Pivots read a snapshot of the source, so new or changed data appears only after Refresh. Pivots on the same source share a cache.

Resources

- ▶ YouTube: Leila Gharani XLOOKUP INDEX MATCH and ▶ YouTube: Leila Gharani Excel dashboard.
- ▶ YouTube: Chandoo pivot tables tutorial and ▶ YouTube: Chandoo Power Query tutorial.

- ▶ YouTube: Alex The Analyst SQL tutorial, beginner to advanced SQL.
- ▶ YouTube: techTFQ SQL window functions and ▶ YouTube: techTFQ SQL interview questions, great for practice.
- 📘 Microsoft Learn: SELECT (T-SQL) · 📘 Window functions (OVER clause)
- 📘 Metabase documentation
- Practice: 📘 LeetCode SQL 50 · 📘 HackerRank SQL

COURSE HANDBOOK 05

🤖 ML, Text Mining & Intro to AI

Illustrated notes for Classes 1–7 (completed), plus a preview of what comes next

Machine Learning using Python (Online) Oct 2026 · trainer Sandeep Ravula · 26 classes · completed through Class 7

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#ml**

Overview

This course has two modules: **Machine Learning with Python** and **Intro to AI and Neural Networks**. Classes 1–7 are complete. These notes follow the class topics in order and add worked pictures and code so you can revise without rewatching 3-hour recordings.

How these notes were made: written from the official class topic list for each session plus standard references. The class videos themselves were not transcribed, so if your trainer used a specific example, add your own notes next to the matching section. Charts use either generated data (labelled) or scikit-learn's built-in breast-cancer dataset so every number shown is real output.

Class	Date	Topics	Notes
1	Sat 4 Oct	Hypothesis testing, CLT, parametric vs non-parametric tests, case study introduction	open
2	Fri 10 Oct	Tests from the case study; p-value, alpha, critical value; one- and two-tailed tests	open
3	Sat 11 Oct	Concluding hypothesis testing; introduction to ML; types of business problems; business → statistical problem; types of ML systems	open
4	Fri 17 Oct	Analytics / ML project life cycle; linear vs non-linear models; linear equation; link functions	open
5	Sat 18 Oct	Gradient descent for linear regression in Python	open
6	Fri 24 Oct	Pre-processing (missing values, outliers, categorical encoding); overfitting/underfitting; segmentation intro (heuristic, RFM)	open
7	Sat 25 Oct	KNN classifier and regressor; GridSearchCV; Bankloans case study; classification metrics	open
8	Fri 31 Oct, 2 PM	Next class	preview

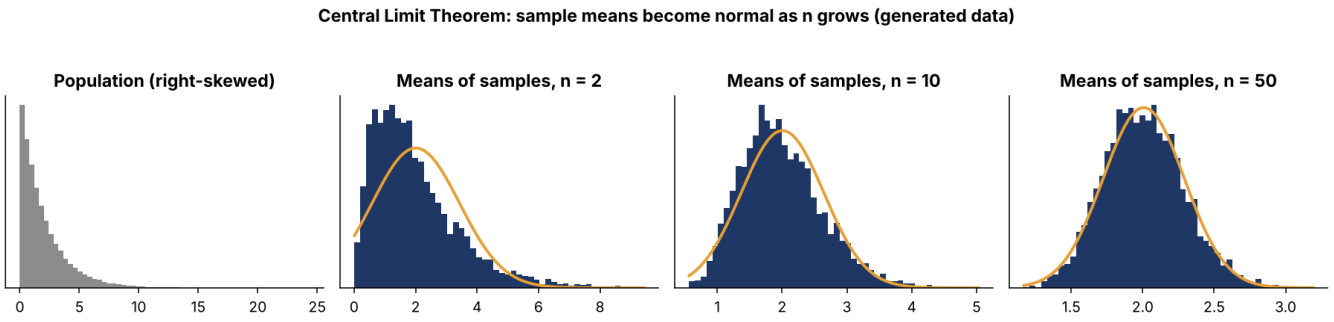
Tip A 7-day revision plan: Day 1 hypothesis-test picker table · Day 2 redo the Insurance case hypothesis questions · Day 3 ML problem types · Day 4 run the gradient-descent code and change α · Day 5 pre-processing pipeline · Day 6 KNN + GridSearchCV · Day 7 explain every metric from one confusion matrix.

Class 1–2: Hypothesis testing

Why we test hypotheses

We almost never see the whole population. We see a **sample** and must decide whether a pattern in it is real or just sampling noise. Hypothesis testing is a formal way to make that call while controlling how often we are fooled.

The Central Limit Theorem (CLT)



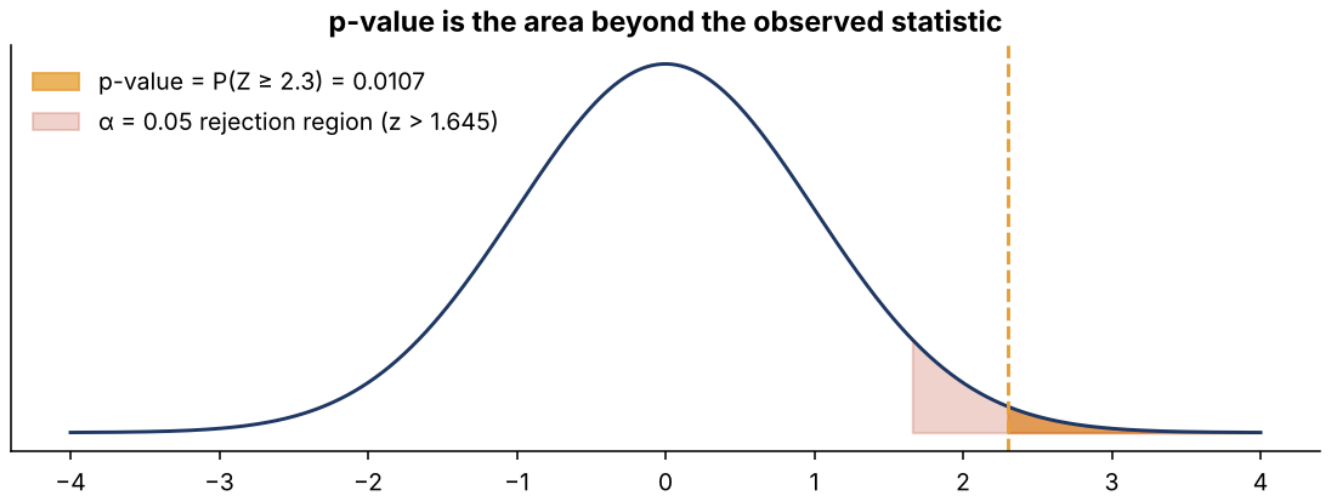
Population: right-skewed (exponential, σ = 2). Means of samples of size 2, 10 and 50 become more normal and narrower. Standard errors measured: 1.42, 0.64 and 0.28, matching σ/√n = 1.41, 0.63 and 0.28.

- For a large enough sample (rule of thumb n ≥ 30), the **sampling distribution of the mean** is approximately normal, whatever the population's shape.
- Its centre is the population mean μ; its spread, the **standard error**, is σ/√n.
- This is why z- and t-tests work on non-normal business data such as order values.

The 6-step recipe

1. State H0 (no effect / no difference / equals a value) and H1 (what you want evidence for).
2. Choose α, the risk of a false alarm you accept (usually 0.05).
3. Pick the test from the data type and question (table below).
4. Compute the test statistic (z, t, χ^2, F) from the sample.
5. Get the p-value (or compare the statistic with the critical value).
6. Decide ($p < \alpha \rightarrow$ reject H0) and translate into business language.

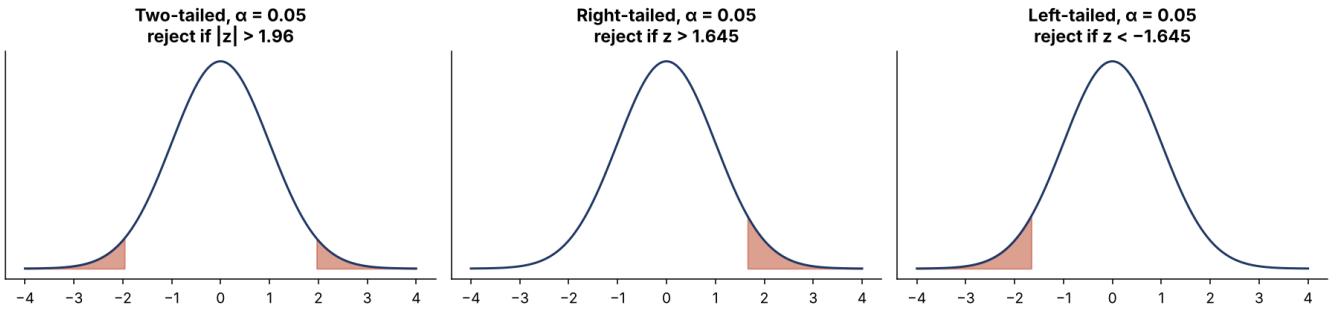
p-value, α and critical value



If the observed z is 2.3, the p-value for a right-tailed test is the area beyond 2.3 (0.0107). It is below α = 0.05, so H0 is rejected.

α (significance level)	Chosen before the test. The probability of rejecting H0 when it is actually true.
p-value	Computed from the data. The probability, assuming H0 is true, of getting a result at least as extreme as the one observed.
Critical value	The cut-off on the statistic's scale that matches α: 1.96 (two-tailed, 5%), 1.645 (one-tailed, 5%), 2.576 (two-tailed, 1%).
Decision rule	p ≤ α ⟺ |statistic| ≥ critical value → reject H0. Both rules always agree.

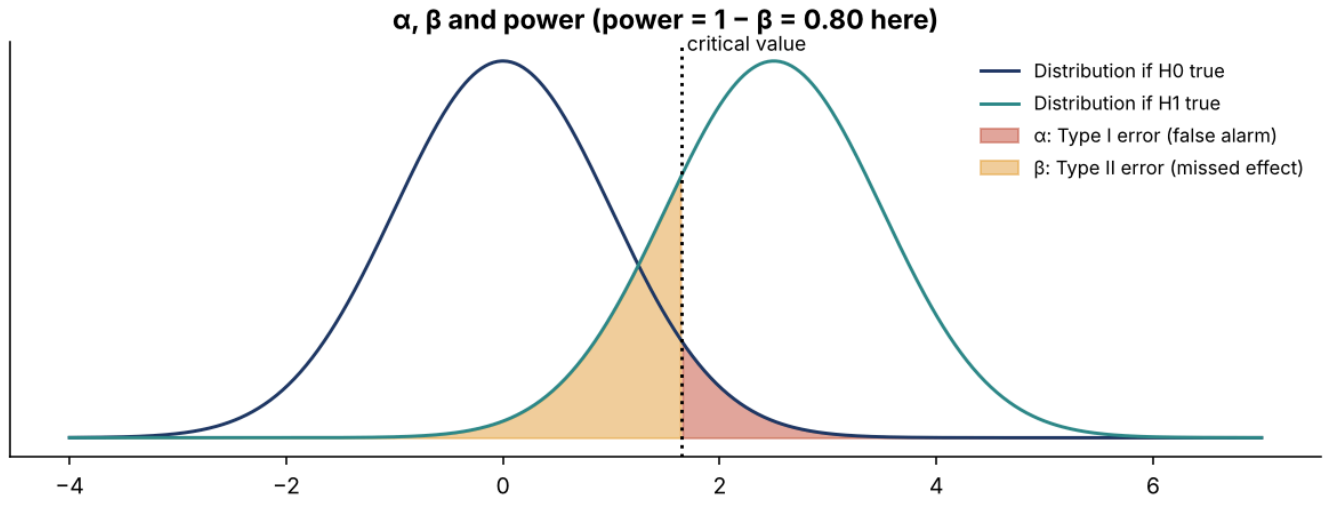
One-tailed vs two-tailed



Red areas are the rejection regions for α = 0.05.

- **Two-tailed** (H1: μ ≠ μ0): you care about a difference in either direction. "Is the average claim different from $10,000?"
- **Right-tailed** (H1: μ > μ0): "Has the average claim *risen* above $10,000?" (Insurance case Q18).
- **Left-tailed** (H1: μ < μ0): "Did the new process *reduce* delivery time?"
- scipy returns two-tailed p-values; for a one-tailed test halve it, but only when the statistic points in the H1 direction.

Errors and power



Shrinking α moves the critical line right: fewer false alarms (Type I) but more missed effects (Type II). A larger sample narrows both curves and raises power.

	H0 actually true	H0 actually false
Reject H0	Type I error (α), false positive	Correct (power = 1 − β)
Do not reject H0	Correct	Type II error (β), false negative

Parametric vs non-parametric tests

Situation	Parametric (assumes normality / equal variance)	Non-parametric alternative (ranks, fewer assumptions)
1 sample vs a value	One-sample t / z	Wilcoxon signed-rank
2 independent groups	Two-sample t (Welch if variances differ)	Mann–Whitney U
2 paired measurements	Paired t	Wilcoxon signed-rank
3+ groups	One-way ANOVA	Kruskal–Wallis
Association of 2 numbers	Pearson r	Spearman ρ
Association of 2 categories	Chi-square test of independence (works on counts; needs expected count ≥ 5 in most cells)	

Assumption checks: `stats.shapiro(x)` (normality, p > 0.05 means no evidence against normal) and `stats.levene(a, b)` (equal variances).

Choosing the right test: decision guide

What is the outcome?

Numeric
How many groups?

1 vs target → *one-sample t*
2 independent → *two-sample t*
2 paired → *paired t*
3+ → *ANOVA*

Categorical
Compared with?

another category → *chi-square independence*
expected proportions → *chi-square goodness of fit*
one proportion → *z-test for proportion*

Two numeric variables
Relationship?

linear → *Pearson*
monotonic / ranks / outliers → *Spearman*

Python template

```
from scipy import stats
# 1-sample, right-tailed: has the mean claim risen above 10,000?
t, p_two = stats.ttest_1samp(claims_2018, 10000)
p = p_two / 2 if t > 0 else 1 - p_two / 2
# 2 independent groups
stats.ttest_ind(male, female, equal_var=False)
# paired: same 12 orders priced by two experts
stats.ttest_rel(barry, mary)
# 3+ groups
stats.f_oneway(*[g["days"] for _, g in df.groupby("priority")])
# categories
chi2, p, dof, expected = stats.chi2_contingency(pd.crosstab(df["age_group"], df["segment"]))
# non-parametric
stats.mannwhitneyu(a, b); stats.kruskal(g1, g2, g3); stats.spearmanr(x, y)
```

Worked examples from your case studies

Question	Test	Result on the real data
Do Mary and Barry quote differently for the same 12 orders?	Paired t	t = 2.52, p = 0.028 → reject H0: Barry quotes higher on average (124.3 vs 114.8)
Did re-engineering change the critical-incident rate?	Two-sample t (Welch)	CI fell from 53.9% to 23.4%, p = 0.046 → significant at 5%
Is the 2018 mean claim above $10,000?	One-sample t, right-tailed	mean 12,743, t = 4.48, p < 0.001 → significant rise
Do high-priority jobs finish faster?	One-way ANOVA	F = 1.81, p = 0.164 → no evidence the priority system works

Full code and outputs: **Hypothesis Testing case** and **Insurance case Q16–20**.

Common mistakes

- Saying "accept H0". We only *fail to reject* it; absence of evidence is not proof of no effect.
- Using an independent t-test on paired data.
- Running many tests and reporting only the significant one (multiple-testing problem). Correct with Bonferroni (α / number of tests).

- Confusing statistical significance with business importance. With huge samples, tiny differences become "significant"; report the effect size too.

Class 3: Intro to ML

What machine learning is

Traditional programming: **rules + data → answers**. Machine learning: **data + answers → rules** (a model), which then gives answers for new data.

Historical data with outcomes → **Learning algorithm** → **Model f(x)** → **Predictions for new cases**

Types of business problems

Business question	Problem type	Output	Typical models
How much will this customer spend next month?	Regression	Number	Linear regression, KNN regressor, trees, boosting
Will this applicant default?	Classification	Class / probability	Logistic regression, KNN, trees, random forest
Which customers are similar?	Segmentation / clustering	Group labels	Heuristic rules, RFM, k-means, hierarchical
What sells together?	Association	Rules (A → B)	Apriori, market-basket analysis
How many orders next week?	Forecasting	Future values	Moving averages, ARIMA, Prophet
Is this transaction unusual?	Anomaly detection	Score / flag	z-score rules, isolation forest
What are customers complaining about?	Text mining	Topics, sentiment	Bag-of-words, TF-IDF, sentiment models

Converting a business problem into a statistical problem

1. **Business goal**: "Reduce loan losses."
2. **Decision** the model supports: approve or reject an application.
3. **Target variable (y)**: default = 1 if the customer missed 90+ days within 12 months, else 0.
4. **Unit of analysis**: one row per application. **Features (X)**: age, income, debt-to-income, employment years, past defaults.
5. **Problem type**: binary classification. **Success metric**: recall on defaulters at an acceptable precision, plus the money saved.

Tip Most project failures start at step 3: a vague target. Write the exact definition of y, including the time window, before touching any model.

Types of ML systems

Supervised

Learns from labelled examples (X, y). Regression and classification.

Loan default, price prediction

Unsupervised

No labels; finds structure. Clustering, dimensionality reduction, association.

Customer segments, PCA

Reinforcement

An agent learns by trial and reward over time.

Bidding, robotics, game play

Other ways to classify ML systems	
Batch (offline) vs online learning	Retrain on the full dataset periodically, or update continuously as new data streams in.
Instance-based vs model-based	KNN memorises training points and compares new ones (instance-based); regression learns parameters and discards the data (model-based).
Parametric vs non-parametric models	Fixed number of parameters (linear regression) vs complexity growing with data (KNN, trees).
Semi-supervised / self-supervised	A few labels plus many unlabelled rows; or labels created from the data itself (how LLMs are pre-trained).

Class 4: Lifecycle & models

Analytics / ML project life cycle

1. Business understanding
Goal, decision, success metric, constraints

2. Data understanding
Sources, data audit, EDA, quality issues

3. Data preparation
Clean, impute, treat outliers, encode, feature engineering

4. Modelling
Split train/test, try models, tune hyper-parameters

5. Evaluation
Metrics on unseen data, business sense-check

6. Deployment & monitoring
API / batch scoring, drift checks, retraining (see ML Ops)

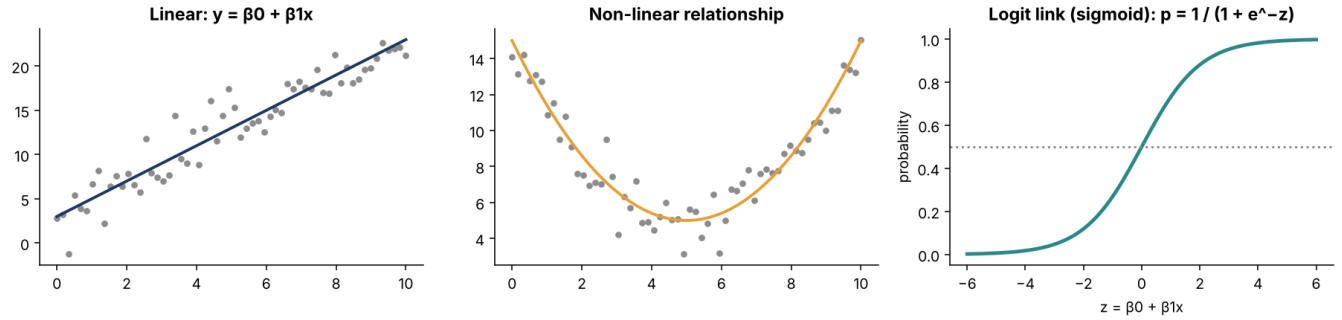
This is the CRISP-DM cycle. Arrows loop back: evaluation often sends you back to data preparation.

The linear equation

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p$$

- **β_0 (intercept)**: predicted y when every x is 0.
- **β_j (coefficient)**: change in y for a one-unit increase in x_j, holding the other features fixed.
- "Linear" means linear in the **parameters**. $\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$ is still a linear model (a polynomial feature), which is why linear regression can fit curves.

Linear vs non-linear models



Left: a straight-line relationship. Middle: a curved one that a straight line cannot capture. Right: the sigmoid link used by logistic regression.

	Linear models	Non-linear models
Examples	Linear and logistic regression	KNN, decision trees, random forest, boosting, neural networks
Strengths	Fast, interpretable coefficients, little data needed	Capture interactions and curves automatically
Weaknesses	Miss non-linear patterns unless you engineer features	Need more data, harder to explain, can overfit

Link functions (generalised linear models)

A link function connects the linear score $z = \beta_0 + \beta_1 x \ldots$ to the kind of target you have.

Target	Link	Model	Inverse (prediction)
Continuous number	Identity: $z = \mu$	Linear regression	$\hat{y} = z$
Yes/No (probability)	Logit: $z = \ln(p / (1 - p))$	Logistic regression	$p = 1 / (1 + e^{-z})$
Counts (claims, calls)	Log: $z = \ln(\mu)$	Poisson regression	$\hat{y} = e^z$

> **Tip** Odds = p/(1 − p). If p = 0.8, odds = 4 ("4 to 1"). A logistic coefficient of 0.7 multiplies the odds by e^0.7 ≈ 2.0 per unit increase in that feature.

Class 5: Gradient descent

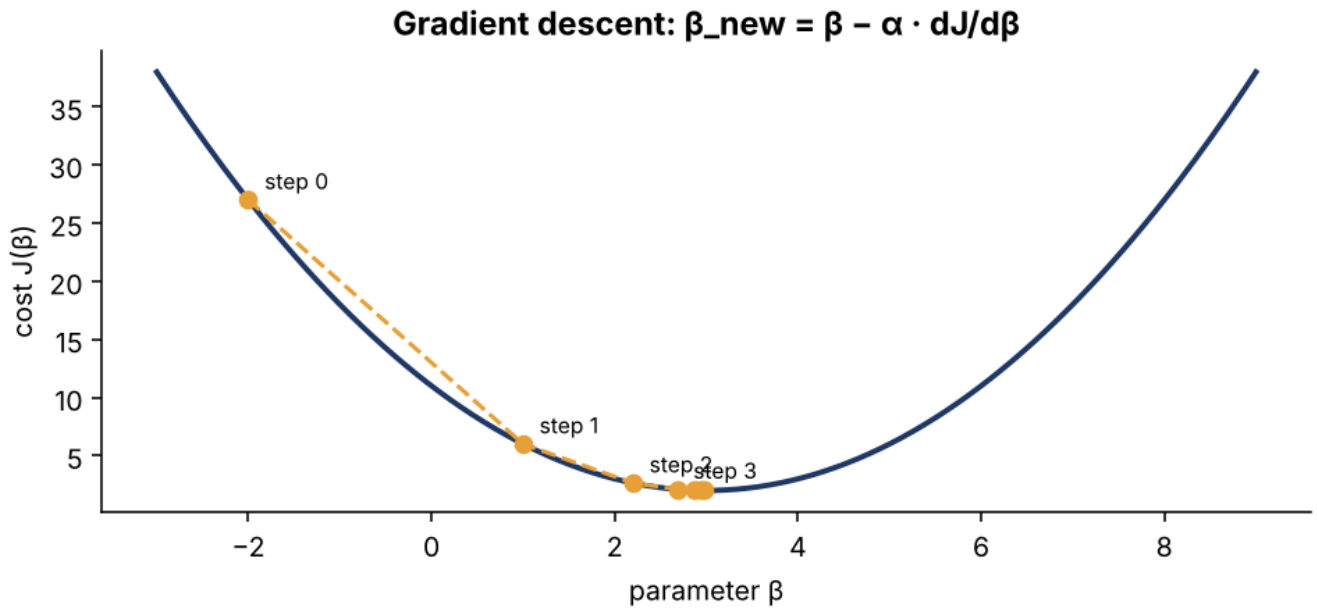
The problem gradient descent solves

For linear regression we want β_0, β_1 that minimise the cost (mean squared error):

$$J(\beta_0, \beta_1) = (1/n) \Sigma (\beta_0 + \beta_1 x_i - y_i)^2$$

Gradient descent starts with a guess and repeatedly steps **downhill**, opposite the gradient:

$$\beta_0 \leftarrow \beta_0 - \alpha \cdot (2/n) \Sigma(\hat{y}_i - y_i) \qquad \beta_1 \leftarrow \beta_1 - \alpha \cdot (2/n) \Sigma(\hat{y}_i - y_i) \cdot x_i$$



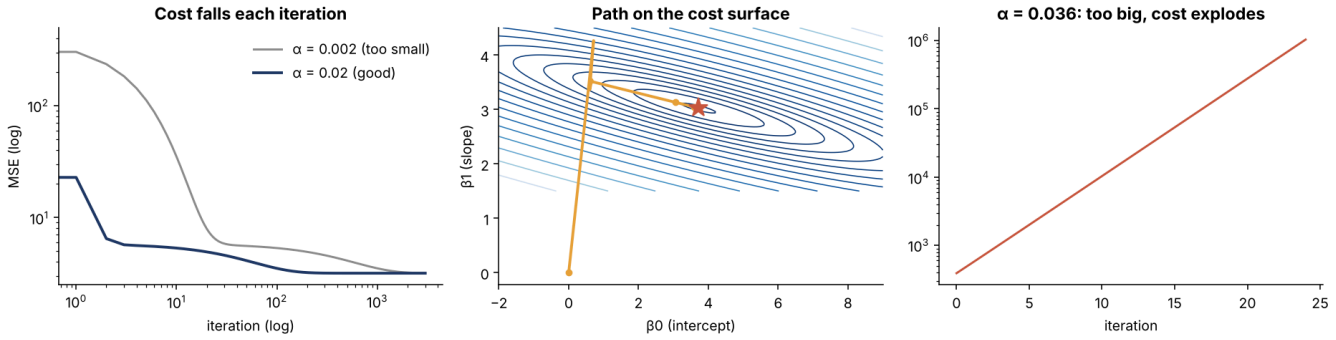
One parameter: big steps where the slope is steep, small steps near the bottom.

From-scratch implementation

```python
import numpy as np
rng = np.random.default_rng(42)
X = rng.uniform(0, 10, 100)
Y = 4 + 3 * X + rng.normal(0, 2, 100)          # true line: y = 4 + 3x (+ noise)

def gradient_descent(X, Y, lr=0.02, iters=3000):
    b0, b1 = 0.0, 0.0
    for _ in range(iters):
        err = (b0 + b1 * X) - Y                # prediction error
        b0 -= lr * 2 * err.mean()              # dJ/db0
        b1 -= lr * 2 * (err * X).mean()        # dJ/db1
    return b0, b1

b0, b1 = gradient_descent(X, Y)
print(round(b0, 3), round(b1, 3))              # 3.723  3.025
print(np.polyfit(X, Y, 1)[::-1].round(3))      # [3.723 3.025]  closed-form OLS agrees
```



Left: cost per iteration for a small and a good learning rate (log scales). Middle: the path of (β_0, β_1) on the cost surface toward the OLS solution (★). Right: a learning rate just a little too large makes the cost explode.

Learning rate α and convergence

- **Too small**: correct but slow (grey line).
- **Too large**: overshoots and diverges (red panel); here α = 0.036 explodes while 0.02 converges.
- **Feature scaling** (standardising X) makes the cost surface rounder, so one α works for all parameters and convergence is much faster.
- Stop when the cost changes less than a tolerance, or after a fixed number of iterations.

Flavours

Batch GD	Uses all rows per step: stable, slow on big data.
Stochastic GD (SGD)	One row per step: noisy, fast, can escape shallow dips.
Mini-batch GD	32–512 rows per step: the standard in deep learning.

Tip Why learn this if `LinearRegression()` solves it directly? Because logistic regression, neural networks and boosting have no closed-form answer; they are all trained with gradient-based optimisation.

Class 6: Pre-processing & fit

Pre-processing pipeline

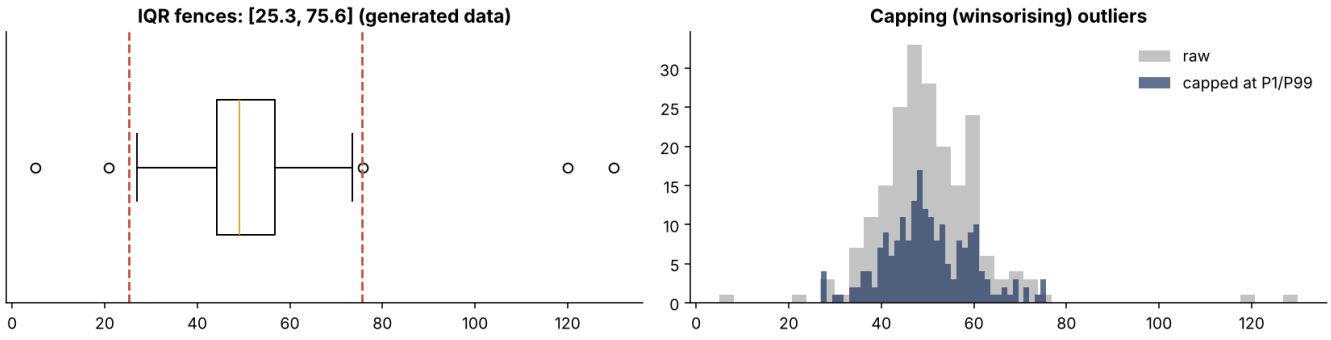
Data audit → Missing values → Outliers → Encoding → Scaling → Train/test split

Tip Split first, then **fit** imputers, scalers and encoders on the training set only and apply them to the test set. Fitting on all the data leaks test information into training (data leakage).

Missing value imputation

Method	When	Code
Drop rows	Very few rows, missing at random	`df.dropna(subset=["income"])`
Drop column	Mostly empty (for example > 40–50%) and not critical	`df.drop(columns=["col"])`
Mean / median	Numeric; median if skewed	`SimpleImputer(strategy="median")`
Mode / "Missing" label	Categorical	`SimpleImputer(strategy="most_frequent")`
Group-wise	Value depends on a group	`df.groupby("city")["income"].transform(lambda s: s.fillna(s.median()))`
Model-based	Strong relationships between columns	`KNNImputer(n_neighbors=5)`
Missing flag	Missingness itself is informative	`df["income_missing"] = df["income"].isna().astype(int)`

Outlier treatment



Generated data: Q1 = 44.1, Q3 = 56.7, fences at 25.3 and 75.6. Three planted extreme values sit outside the fences. Capping keeps the rows but limits their influence.

- **Detect**: box plot / IQR fences, z-score above 3, domain rules (age > 100).
- **Treat**: correct data-entry errors; cap (winsorise) at P1/P99; log-transform skewed variables; or keep them if they are real and important (fraud).
- KNN and linear regression are sensitive to outliers; tree models are much less so.

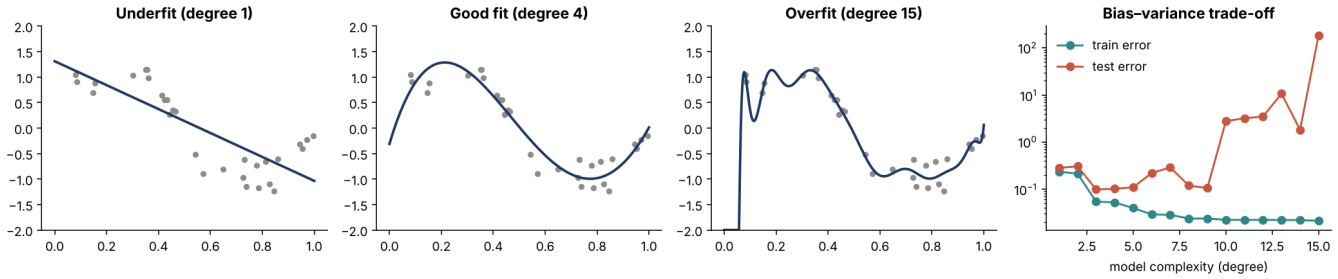
Categorical encoding

Method	Example	Use for
One-hot (dummy)	city → city_Delhi, city_Pune (0/1)	Nominal categories. `pd.get_dummies(df, columns=["city"], drop_first=True)`
Label / ordinal	Low=0, Medium=1, High=2	Ordered categories. `OrdinalEncoder(categories=[["Low","Medium","High"]])`

Method	Example	Use for
Frequency / target encoding	city → share of rows or mean of y	High-cardinality columns (hundreds of values); fit on train only

> **Tip** `drop_first=True` avoids the dummy-variable trap (perfect multicollinearity) in linear and logistic regression.

Overfitting and underfitting



Generated data from a sine curve. Degree 1 is too simple (high bias); degree 15 memorises noise (high variance) and its test error shoots up. The sweet spot is where test error is lowest.

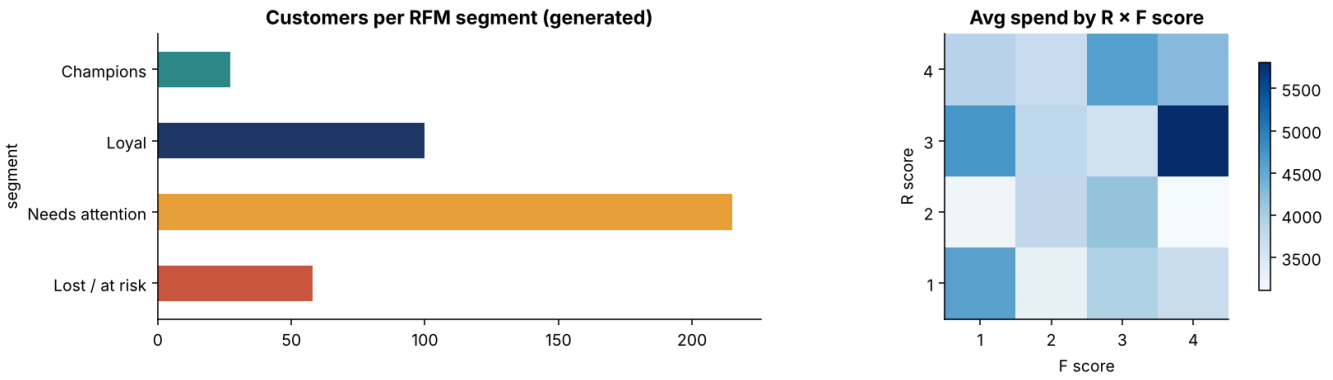
	Underfitting	Overfitting
Symptom	Poor on train and test	Great on train, poor on test
Cause	Model too simple, missing features	Model too complex, too little data, leakage
Fix	Add features, more flexible model, less regularisation	More data, simpler model, regularisation, cross-validation, feature selection; for KNN raise K

Segmentation: heuristic and RFM

Heuristic segmentation uses business rules ("salaried, age 25–35, spend > ₹50k = Young Affluent"). It is quick and explainable but relies on judgement. **RFM** is a data-driven heuristic:

Recency	Days since last purchase (lower is better)
Frequency	Number of purchases
Monetary	Total spend

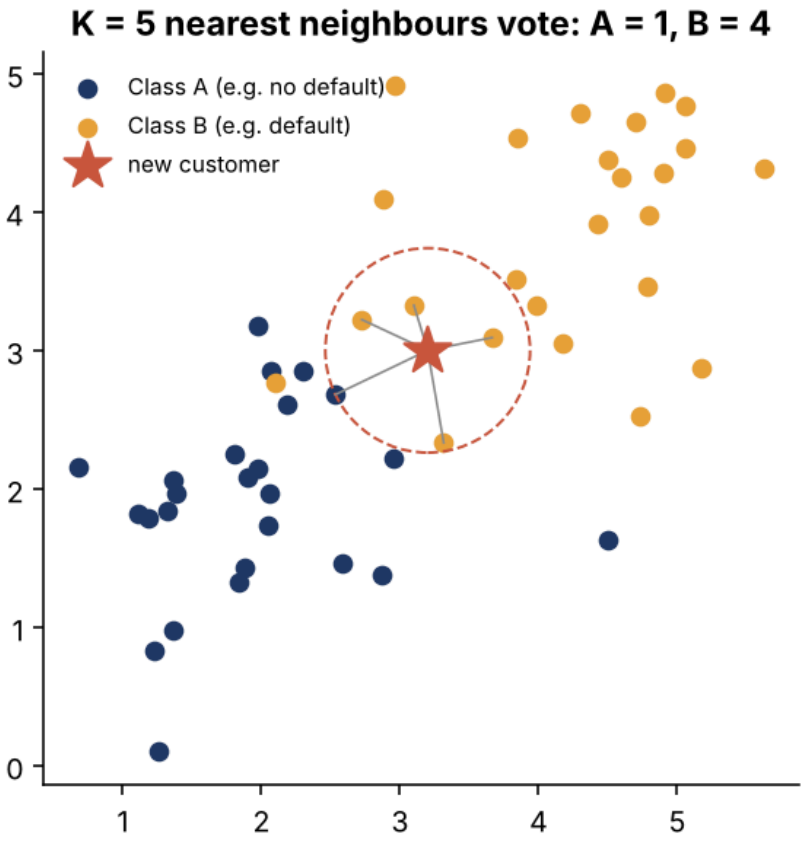
```
snap = tx["date"].max() + pd.Timedelta(days=1)
rfm = tx.groupby("cust_id").agg(recency=("date", lambda d: (snap - d.max()).days),
                                frequency=("txn_id", "nunique"), monetary=("amount", "sum"))
rfm["R"] = pd.qcut(rfm["recency"], 4, labels=[4, 3, 2, 1]).astype(int)          # recent = 4
rfm["F"] = pd.qcut(rfm["frequency"].rank(method="first"), 4, labels=[1, 2, 3, 4]).astype(int)
rfm["M"] = pd.qcut(rfm["monetary"], 4, labels=[1, 2, 3, 4]).astype(int)
rfm["score"] = rfm[["R", "F", "M"]].sum(axis=1)                                 # 3 .. 12
rfm["segment"] = pd.cut(rfm["score"], [2, 5, 8, 10, 12], labels=["Lost / at risk", "Needs
attention", "Loyal", "Champions"])
```



Generated customers. Right: average spend is highest where both R and F scores are high, so those customers deserve retention offers first.

Class 7: KNN & metrics

K-Nearest Neighbours



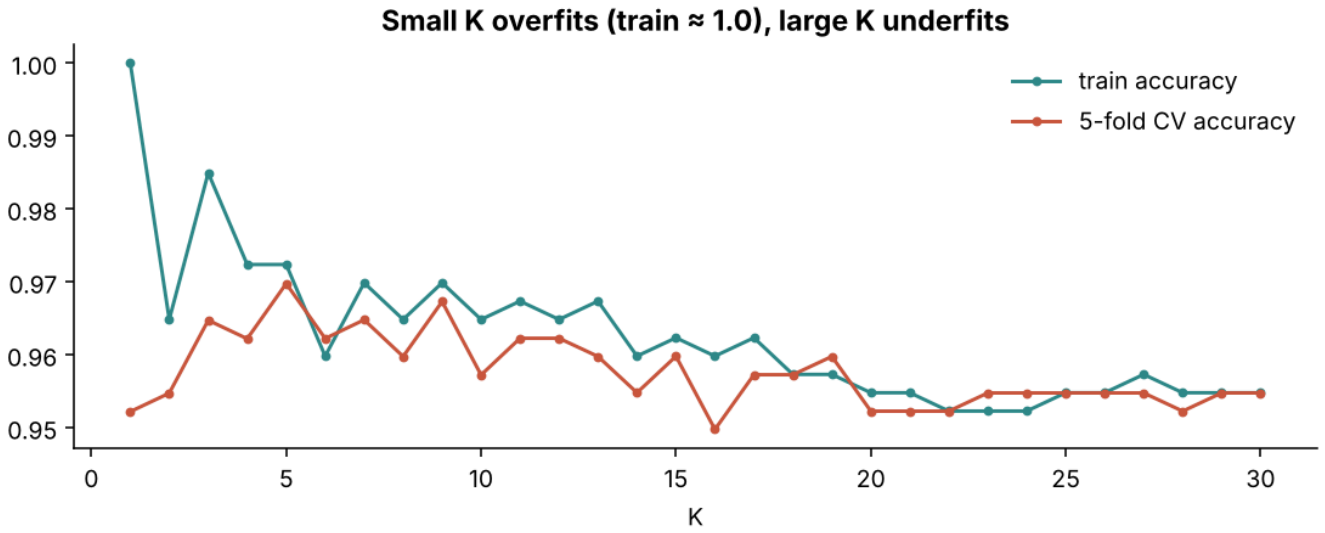
Generated points. A new customer is classified by a vote of its 5 closest neighbours.

1. Choose K and a distance measure.
2. Compute the distance from the new point to every training point.
3. Take the K closest.
4. **Classifier**: majority vote (or probability = share of votes). **Regressor**: average of the neighbours' y values.

Euclidean (p = 2)	$\sqrt{\Sigma(a_i - b_i)^2}$, straight-line distance
Manhattan (p = 1)	$\Sigma\lvert a_i - b_i \rvert$, grid distance; more robust with many features
weights="distance"	Closer neighbours get a bigger vote

Always scale features for KNN. Distance is dominated by the largest-scale column: income in lakhs would drown out age in years. Use `StandardScaler` inside a Pipeline.

Choosing K



Breast-cancer data (scikit-learn). K = 1 scores 100% on train (memorisation) but lower in cross-validation. Very large K smooths too much. Pick K where CV accuracy peaks; odd K avoids ties in binary problems.

GridSearchCV

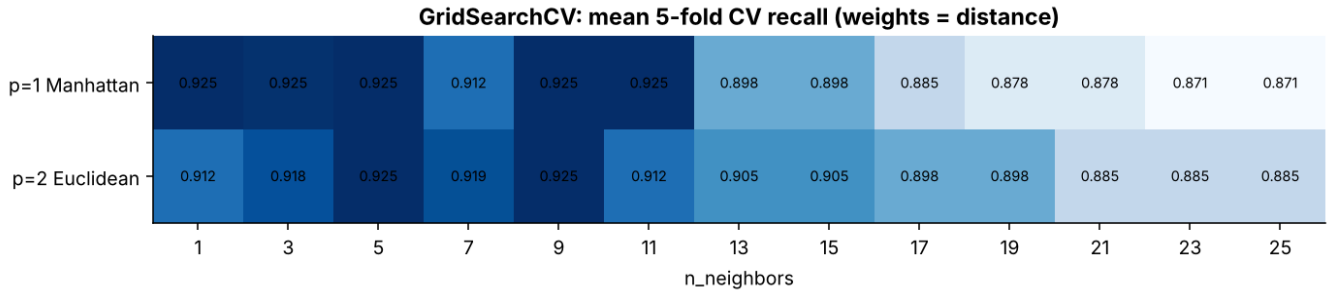
Tries every combination of hyper-parameters with k-fold cross-validation on the training set and keeps the best.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score

data = load_breast_cancer(); X, y = data.data, 1 - data.target        # 1 = malignant (positive class)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

pipe = Pipeline([("scale", StandardScaler()), ("knn", KNeighborsClassifier())])
grid = {"knn__n_neighbors": range(1, 26, 2), "knn__weights": ["uniform", "distance"], "knn__p": [1, 2]}
gs = GridSearchCV(pipe, grid, cv=5, scoring="recall", n_jobs=-1).fit(X_tr, y_tr)
print(gs.best_params_)        # {'knn__n_neighbors': 9, 'knn__p': 1, 'knn__weights': 'uniform'}
print(round(gs.best_score_, 4))   # 0.9253  mean CV recall
```

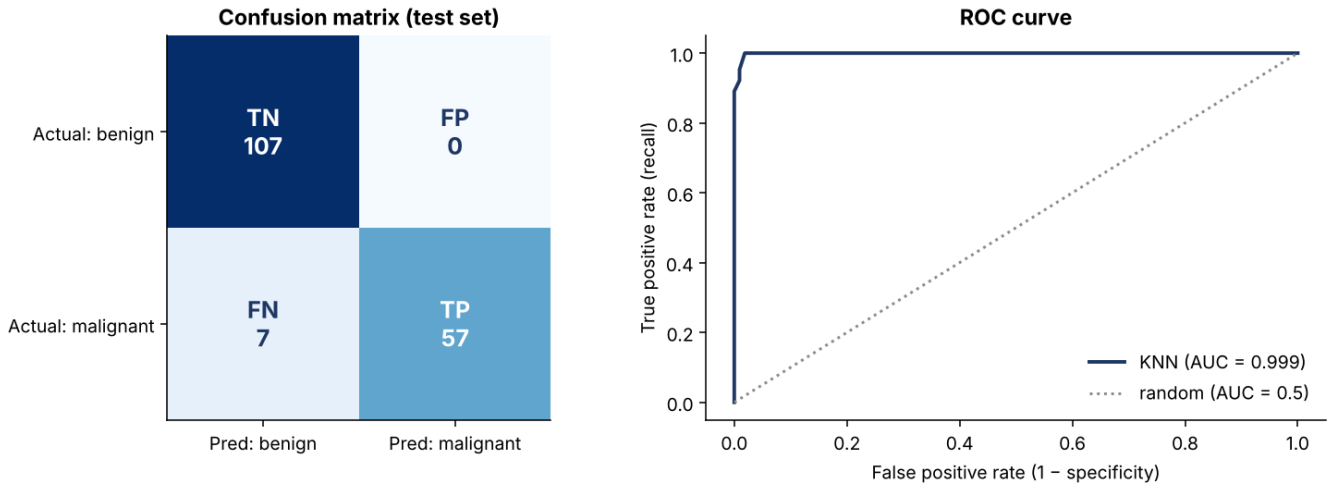
```
best = gs.best_estimator_
pred = best.predict(X_te); prob = best.predict_proba(X_te)[:, 1]
print(confusion_matrix(y_te, pred))     # [[107   0]
                                        #  [  7  57]]
print(classification_report(y_te, pred, target_names=["benign", "malignant"]))
print(roc_auc_score(y_te, prob))        # 0.9989
```



Cross-validated recall for each K and distance metric. Scoring is set to recall because missing a malignant case is the costly error.

Tip Regression version: `KNeighborsRegressor` with `scoring="neg_root_mean_squared_error"`; evaluate with RMSE, MAE and R^2.

Classification metrics from one confusion matrix

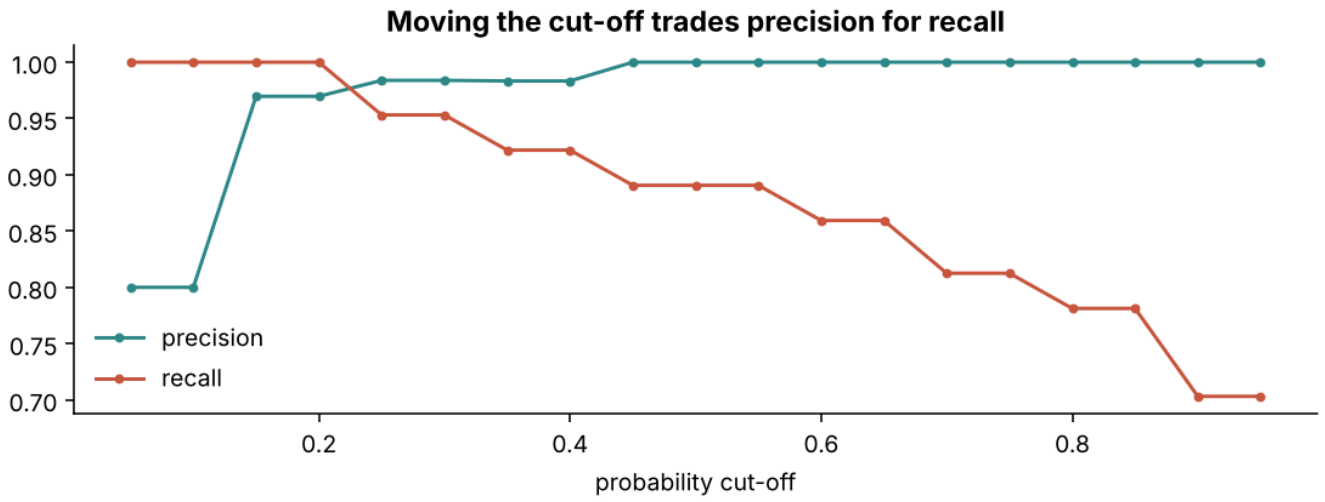


Test set of 171 cases: TN = 107, FP = 0, FN = 7, TP = 57.

Metric	Formula	Value here	Question it answers
Accuracy	(TP + TN) / all	(57 + 107) / 171 = **0.959**	How often is the model right overall? Misleading when classes are imbalanced.
Precision	TP / (TP + FP)	57 / 57 = **1.000**	When it says "positive", how often is it right?
Recall (sensitivity, TPR)	TP / (TP + FN)	57 / 64 = **0.891**	Of all actual positives, how many did it catch?

Metric	Formula	Value here	Question it answers
Specificity (TNR)	TN / (TN + FP)	107 / 107 = **1.000**	Of all actual negatives, how many did it clear?
F1 score	2·P·R / (P + R)	**0.942**	Single balance of precision and recall
ROC-AUC	Area under TPR vs FPR curve	**0.999**	How well does the score rank positives above negatives, across all cut-offs?

Choosing the cut-off



Lowering the probability cut-off catches more positives (recall up) at the cost of more false alarms (precision down).

Business case	Costlier error	Optimise
Loan default (Bankloans)	Approving a defaulter (FN)	Recall on defaulters, then precision
Spam filter	Blocking a real email (FP)	Precision
Disease screening	Missing a sick patient (FN)	Recall
Marketing campaign with limited budget	Wasting offers (FP)	Precision in the top decile (lift)

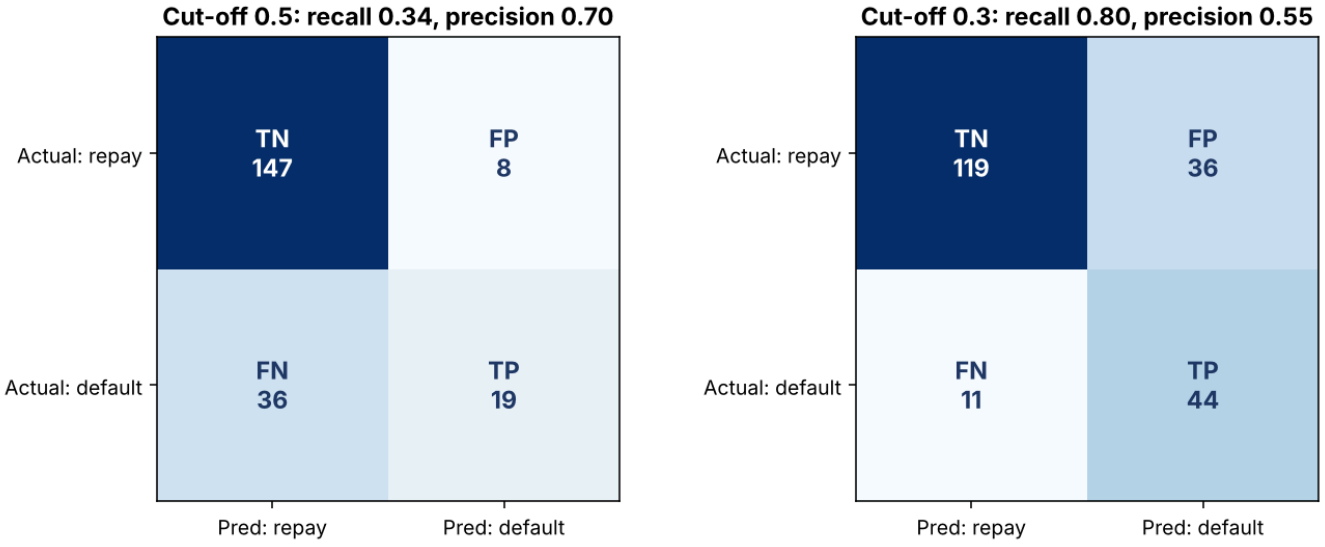
Bankloans case study, solved on the course file

`bankloans.csv` ships with the ML Ops module materials (it is the same dataset used for the Flask banking app). It has 850 applicants: 700 with a known `default` value (26.1% defaulted) and 150 new applicants with a blank default to be scored. Features: age, ed (education level), employ (years with employer), address (years at address), income, debtinc (debt-to-income %), creddebt and othdebt (credit-card and other debt).

```
bank = pd.read_csv("bankloans.csv")
known, new = bank[bank["default"].notna()], bank[bank["default"].isna()]       # 700 / 150
cap = lambda s: s.clip(s.quantile(.01), s.quantile(.99))                        # outlier capping
(as in class)
```

```
X = known.drop(columns="default").apply(cap); y = known["default"].astype(int)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

pipe = Pipeline([("scale", StandardScaler()), ("knn", KNeighborsClassifier())])
gs = GridSearchCV(pipe, {"knn__n_neighbors": range(3, 40, 2), "knn__weights": ["uniform",
"distance"]},
                  cv=5, scoring="roc_auc").fit(X_tr, y_tr)
gs.best_params_        # {'knn__n_neighbors': 35, 'knn__weights': 'distance'}
gs.best_score_         # 0.811 CV AUC  (test AUC 0.835)
prob = gs.best_estimator_.predict_proba(X_te)[:, 1]
pred_05 = (prob >= 0.5).astype(int);  pred_03 = (prob >= 0.3).astype(int)
```



Test set of 210 applicants (55 defaulters). The default 0.5 cut-off catches only 19 of 55 defaulters (recall 0.35). Lowering it to 0.3 catches 44 (recall 0.80) at the cost of more false alarms (precision 0.70 → 0.55).

Cut-off	Accuracy	Precision	Recall	F1
0.5	0.790	0.704	0.345	0.463
0.3	0.776	0.550	0.800	0.652

Business reading: accuracy barely changes, but at 0.3 the bank catches 80% of future defaulters instead of 35%. Because a bad loan usually costs far more than a rejected good customer, the lower cut-off is the better business choice. Scoring the 150 new applicants at 0.3 flags 31.3% of them for review.

> **Tip** In the ML Ops module the same data trains a random forest (with RFE feature selection) that is pickled and served through Flask. See **ML Ops → Banking app**.

Coming next

Preview only: these topics have not been taught yet. Typical order for the rest of the ML and Intro-to-AI modules is below; confirm with each class title as it is published.

Supervised models

- Regularised regression: Ridge, Lasso, ElasticNet
- Decision trees (Gini, entropy, pruning)
- Ensembles: bagging, random forest, gradient boosting, XGBoost
- Naive Bayes, SVM
- Imbalanced data: class weights, SMOTE

Unsupervised and text

- K-means, hierarchical clustering, silhouette score
- PCA for dimensionality reduction
- Text mining: tokenisation, stop words, stemming/lemmatisation, bag-of-words, TF-IDF, sentiment, topic models
- Time series forecasting

Intro to AI and neural networks

- Perceptron, activation functions, back-propagation
- Keras / TensorFlow basics
- CNNs (images) and RNN/LSTM (sequences)
- Transformers and the link to LLMs

How to prepare

- Revise Class 5 (gradient descent): neural nets use the same idea.
- Revise Class 6 (overfitting): trees and boosting are tuned against it.
- Practise `Pipeline` + `GridSearchCV`: every later model plugs into it.

Interview Q&A

Q Explain the p-value to a business stakeholder.

"If there were truly no difference, a result this large would happen only about p×100 times in 100 by chance. Because that is rare (below 5%), we treat the difference as real."

Q Type I vs Type II error with an example?

In fraud detection, Type I is blocking a genuine transaction (false alarm); Type II is letting a fraud through (miss). Lowering α reduces Type I but increases Type II unless you collect more data.

Q When would you use a non-parametric test?

Small samples that are clearly non-normal, ordinal data (ratings), or heavy outliers. Example: Mann–Whitney U instead of a two-sample t-test.

Q Why must features be scaled for KNN but not for decision trees?

KNN uses distances, so large-scale features dominate. Trees split one feature at a time on thresholds, so scale does not change the splits.

Q What does a small K vs a large K do in KNN?

Small K follows individual points: low bias, high variance (overfits). Large K averages many points: high bias, low variance (underfits). Choose K by cross-validation.

Q Why is accuracy a poor metric for imbalanced data?

If 2% of loans default, predicting "no default" for everyone is 98% accurate and useless. Use recall, precision, F1, PR-AUC or ROC-AUC instead.

Q What is data leakage?

When information unavailable at prediction time sneaks into training, such as scaling on the full dataset before splitting, or using a feature recorded after the outcome. It produces great test scores that collapse in production.

Q What does the learning rate do in gradient descent?

It sets the step size. Too small converges slowly; too large overshoots and can diverge. Scaling features lets a single learning rate work well.

Resources

- ▶ YouTube: StatQuest hypothesis testing and the null hypothesis · ▶ YouTube: StatQuest p-values what they are and how to interpret them
- ▶ YouTube: StatQuest central limit theorem clearly explained · ▶ YouTube: StatQuest gradient descent step by step
- ▶ YouTube: StatQuest k-nearest neighbors clearly explained · ▶ YouTube: StatQuest ROC and AUC clearly explained · ▶ YouTube: StatQuest confusion matrix
- ▶ YouTube: StatQuest bias and variance · ▶ YouTube: StatQuest cross validation
- ▶ YouTube: Krish Naik KNN algorithm and ▶ YouTube: codebasics machine learning tutorial python, applied walkthroughs.
- ▶ YouTube: Andrew Ng machine learning specialization gradient descent, the classic explanation.
- 📘 scikit-learn: nearest neighbors · 📘 GridSearchCV · 📘 Model evaluation metrics
- 📘 SciPy statistical tests

COURSE HANDBOOK 06

🚀 ML Ops & Model Deployment

Take a trained model to production: Git, Conda, Flask, Heroku and Azure

ML Ops & Model Deployment (self-paced, 6 classes)

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#m5**

Overview

A model in a notebook helps nobody. ML Ops is the set of practices that gets a model into a product **reliably**, keeps it **working**, and makes it **reproducible**. It combines machine learning, DevOps and data engineering.

Class	Topic	Jump to
1	Introduction to MLOps	MLOps concepts
2	Version control with Git	Git
3	Conda environments and Flask	Conda & Flask
4	Banking application with Flask	Banking app
5	Deploying the app to Heroku	Deploy
6	Deploying the app to Azure	Deploy

Module materials used for these notes: the course slide deck "Machine Learning Operations – MLOps", the e-book "Practical MLOps", and the `bankin_app_flask` project (application.py, the HTML form, the training notebook and bankloans.csv).

MLOps concepts

Three artifacts, three pipelines

1. Data engineering

- Ingestion (CSV, databases, Spark, HDFS)
- Exploration and validation (profiling, schema checks)
- Wrangling / cleaning
- Labelling
- Splitting into train, validation and test

2. Model engineering

- Training (feature engineering, hyper-parameter tuning)
- Evaluation against the business objective
- Testing on a hold-out set ("model acceptance test")
- Packaging (pickle, joblib, ONNX, PMML)

3. Deployment (code engineering)

- Serving the model (REST API, batch job)
- Performance monitoring on live data
- Logging every prediction request
- Triggering re-training

DevOps vs MLOps

	DevOps	MLOps
What changes	Code	Code + data + model
Continuous Integration	Test code and components	Also validate data, schemas and models
Continuous Delivery	Ship a software package	Ship a *training pipeline* that produces and deploys a prediction service
Continuous Training	n/a	Automatically retrain and redeploy models
Failure mode	Bugs	Bugs plus silent decay as data drifts (training–serving skew)
Team	Developers + operations	Data scientists + ML engineers + data engineers + DevOps

MLOps maturity levels

Level 0: Manual

Notebook-driven. A data scientist hands a pickled model to engineers. Rare releases, no CI/CD, no monitoring. Fine for models that rarely change.

Level 1: ML pipeline automation

The whole training pipeline is deployed and runs automatically (continuous training) on triggers: schedule, new data, performance drop or data drift. Needs data and model validation, a feature store and metadata tracking.

Level 2: CI/CD pipeline automation

Pipelines themselves are built, tested and deployed automatically. Components: source control, test and build services, deployment services, model registry, feature store. For teams retraining daily or hourly.

Key terms

Experiment tracking	Log parameters, metrics, data versions and artifacts for every training run so runs can be compared and reproduced (MLflow, Weights & Biases).
Model registry	Versioned store of trained models with stages (staging, production, archived).
Feature store	One place that defines, stores and serves features so training and serving use identical logic.
Data drift	Input distribution changes (customers get younger, incomes rise).
Concept drift	The relationship between X and y changes (a recession changes who defaults).
Training–serving skew	Features computed differently in training and in production.
Canary / A-B deployment	Send a small share of traffic to the new model first, compare, then roll out.

Git

Why version control

Git records every change to your code, lets several people work in parallel on **branches**, and lets you return to any earlier version. GitHub (or Azure Repos, GitLab) hosts the shared copy; Heroku and Azure deploy straight from it.

Working directory → **git add → Staging area** → **git commit → Local repo** →

git push → Remote (GitHub)

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git init                          # start tracking a folder
git status                        # what changed?
git add application.py templates/   # stage files  (git add . = everything)
git commit -m "Add Flask app for loan default model"
git remote add origin https://github.com/<user>/bank-app.git
git push -u origin main
git clone https://github.com/<user>/bank-app.git   # copy a repo
git pull                          # get latest changes
git log --oneline                 # history
git checkout -b feature-ui        # new branch
git merge feature-ui              # merge it back (from main)
git diff                          # line-by-line changes
```

Tip Add a `.gitignore` for `__pycache__/`, `.ipynb_checkpoints/`, `.env` and large raw data. Never commit passwords or API keys.

Conda & Flask

Conda environments (Class 3)

Each project gets its own isolated set of packages, so a model trained with scikit-learn 1.0 keeps working even if another project needs 1.5.

```
conda create -n bankapp python=3.10 -y
conda activate bankapp
pip install flask pandas scikit-learn gunicorn
conda list                        # what is installed (the project's conda_info.txt / package_info.txt)
pip freeze > requirements.txt     # pin versions for deployment
conda env export > environment.yml  # full conda spec
conda deactivate
```

Pin the same scikit-learn version for training and serving. A pickled model often fails to load, or behaves differently, under another version.

Flask in five minutes

Flask is a lightweight Python web framework. A **route** maps a URL to a Python function; the function returns a page or data.

```
from flask import Flask, request, jsonify, render_template
app = Flask(__name__)

@app.route("/")                          # GET http://127.0.0.1:5000/
def home():
    return render_template("index.html") # file in templates/

@app.route("/api/predict", methods=["POST"])
def predict_api():
    data = request.get_json()            # {"employ": 10, "income": 50, ...}
    return jsonify({"default": int(model.predict(pd.DataFrame([data]))[0])})

if __name__ == "__main__":
    app.run(debug=True)                  # dev server only; use gunicorn in production
```

GET	Ask for something (open a page)
POST	Send data (submit a form, call the prediction API)
templates/	HTML files rendered with Jinja (`{{ prediction }}`)
static/	CSS, JS, images

Banking app walkthrough

The project (Class 4)

```
bankin_app_flask/
├── Banking Application Model.ipynb   train + pickle the model
├── bankloans.csv                     850 applicants (700 with known default)
├── RF_model_example.pkl              the saved random forest
├── application.py                    Flask app that serves it
├── templates/index.html              input form
└── requirements.txt                  packages for the server
```

Step 1: train and save the model (notebook)

```
bankloans = pd.read_csv("bankloans.csv")
people_with_info = bankloans[bankloans.default.notnull()]      # 700 rows for training
people_with_info = people_with_info.apply(lambda x: x.clip(x.quantile(.01), x.quantile(.99)))   #
cap outliers
# feature selection: Recursive Feature Elimination with a random forest
rfe = RFE(estimator=RandomForestClassifier(), n_features_to_select=5).fit(x_train, y_train)
# chosen features: employ, income, debtinc, creddebt, othdebt
cv_rf = GridSearchCV(RandomForestClassifier(oob_score=True, random_state=1024),
                     {"n_estimators": [10, 20, 40, 80, 100], "max_depth": [1, 2, 3]},
```

```
cv=5).fit(x_train, y_train)
pickle.dump(cv_rf.best_estimator_, open("RF_model_example.pkl", "wb"))
```

Step 2: serve it (application.py from the course)

```
import pickle
import pandas as pd
from flask import Flask, render_template, request

model = pickle.load(open('RF_model_example.pkl', 'rb'))   # load once at start-up
app = Flask(__name__)

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'GET':
        return render_template('index.html')               # show the form
    else:
        form_inputs = pd.DataFrame(request.form.to_dict(), index=[0])   # form fields -> 1-row
frame
        prediction = model.predict(form_inputs.astype(float))
        return str(prediction)

if __name__ == "__main__":
    app.run(debug=True)
```

Step 3: the form (templates/index.html)

```
<title>Bank application</title>
<form method="post">
    employ:   <input type="text" name='employ'>
    income:   <input type="text" name='income'>
    debtinc:  <input type="text" name='debtinc'>
    creddebt: <input type="text" name='creddebt'>
    othdebt:  <input type="text" name='othdebt'>
    <input type="submit">
</form>
```

> **The input names must match the training column names and order** (employ, income, debtinc, creddebt, othdebt). If the form sends columns in a different order, the model silently predicts on the wrong features.

Run it locally

```
conda activate bankapp
python application.py          # open http://127.0.0.1:5000
```

Improvements worth making

- Return a friendly page: "Likely to default (probability 0.72)" using `predict_proba` and a business cut-off (see ML Class 7 for why 0.5 may be wrong).

- Validate inputs (no blanks, sensible ranges) before predicting.
- Log every request and prediction for monitoring.
- Add a JSON `/api/predict` route so other systems can call the model.

Deploy: Heroku & Azure

Production checklist

✔ `requirements.txt` with pinned versions, including `gunicorn`
✔ Model file committed (or downloaded at start-up)
✔ No `debug=True` in production
✔ Port and secrets read from environment variables

Heroku (Class 5)

```
Procfile                      (no extension)
web: gunicorn application:app  # module_name:flask_object
```

```
heroku login
heroku create bank-default-app
git push heroku main          # Heroku builds from requirements.txt and starts the Procfile
heroku logs --tail            # debug start-up errors
heroku open
```

Tip Heroku ended its free plan in November 2022, so deployments now need a paid dyno. Render, Railway and PythonAnywhere offer similar git-push deployment with free or low-cost tiers; the Procfile/gunicorn pattern is the same.

Azure App Service (Class 6)

1. Azure Portal → Create a resource → **Web App**. Runtime stack: Python 3.x; OS: Linux; pick a resource group, region and pricing plan.
2. Deployment Center → Source: **GitHub** → choose repository and branch. Azure creates a GitHub Actions workflow that builds and deploys on every push.
3. Configuration → General settings → Startup command: `gunicorn --bind=0.0.0.0 --timeout 600 application:app`. (Azure auto-detects `application.py` / `app.py` with an `app` object.)
4. Browse to `https://<app-name>.azurewebsites.net`. Check Log stream if it fails to start.

```
# CLI alternative from the project folder
az login
```

```
az webapp up --name bank-default-app --runtime "PYTHON:3.10" --sku B1
```

Beyond a single app

Docker	Package app + environment into an image that runs identically anywhere
Batch scoring	Score a whole table nightly and write results to a database (often simpler than an API)
Managed ML platforms	Azure ML, AWS SageMaker, Google Vertex AI: registry, endpoints, monitoring built in
MLflow	Open-source experiment tracking, model registry and serving

Interview Q&A

Q What is MLOps and why is it needed?

Practices that combine ML, DevOps and data engineering to deploy and maintain models reliably. It is needed because models depend on data that changes, so they decay silently without versioning, monitoring and automated retraining.

Q How is CI/CD different for ML?

CI also validates data and models, not only code. CD deploys a training pipeline that produces a prediction service. Continuous training (CT) is new: retraining automatically on triggers such as drift.

Q Data drift vs concept drift?

Data drift: the distribution of inputs changes. Concept drift: the link between inputs and the target changes. Both lower accuracy; detect them by monitoring input statistics and live performance.

Q Why use a virtual environment?

To isolate each project's package versions so a model trained with one version can be loaded and served with exactly the same versions.

Q Why gunicorn instead of app.run()?

Flask's built-in server is single-threaded and meant for development. Gunicorn is a production WSGI server that runs multiple workers and handles concurrent requests.

Q How would you monitor a deployed credit-risk model?

Log inputs and predictions; track input distributions against training (PSI), approval and default rates over time, and the model's precision/recall once outcomes arrive; alert and retrain when thresholds are crossed.

Resources

- ▶ YouTube: Krish Naik deploy machine learning model using flask and ▶ YouTube: Krish Naik end to end ML project deployment
- ▶ YouTube: freeCodeCamp git and github for beginners crash course
- ▶ YouTube: Corey Schafer Flask tutorial
- ▶ YouTube: deploy flask app to azure app service
- 📘 Flask quickstart · 📘 Conda: managing environments
- 📘 Azure: Quickstart deploy a Python web app · 📘 Heroku Python getting started
- 📘 Google Cloud: MLOps continuous delivery and automation pipelines (source of the level 0/1/2 model)
- 📘 MLflow documentation

COURSE HANDBOOK 07

✨ Generative AI

How LLMs work, prompting techniques, and using AI for slides, web pages, sheets, code and images

Generative AI SP (26 classes, task-based videos)

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#m6**

Overview

This course is practical: understand what a large language model (LLM) is, learn named prompting techniques, then apply them to real tasks such as presentations, web pages, spreadsheets, code and images.

Class	Topic	Date · length	Notes
Generative AI · Class 1	Introduction to LLMs – Part 1		How LLMs work
Generative AI · Class 2	Introduction to LLMs – Part 2: basics of LLMs and key terminologies		How LLMs work
Generative AI · Class 3	Introduction to LLMs – Part 3: working of LLMs		How LLMs work
Generative AI · Class 4	Introduction to LLMs – Part 4: applications of LLMs		How LLMs work
Generative AI · Class 5	Introduction to the ChatGPT interface: creating an account, website and services, 3.5 vs 4 vs Turbo, tokens, OpenAI Playground, pricing		How LLMs work
Generative AI · Class 6	Introduction to prompting techniques		Prompting techniques
Generative AI · Class 7	PT1 – specifying output formats		Prompting techniques
Generative AI · Class 8	PT2 – assigning a role/character, setting comprehension level, tone		Prompting techniques
Generative AI · Class 9	PT4 – reverse prompting		Prompting techniques
Generative AI · Class 10	PT6 – chain of thought; PT7 – ask before you answer / tell me what I don't know		Prompting techniques
Generative AI · Class 11	Making presentations (Part 1)		Applied use cases
Generative AI · Class 12	Making presentations (Part 2)		Applied use cases

Class	Topic	Date · length	Notes
Generative AI · Class 13	Making presentations (Part 3)		Applied use cases
Generative AI · Class 14	Creating webpages		Applied use cases
Generative AI · Class 15	Using ChatGPT in Google Sheets (Part 1)		Applied use cases
Generative AI · Class 16	Using ChatGPT in Google Sheets (Part 2)	9 Mar 2024 · 19:40	Applied use cases
Generative AI · Class 17	Automating tasks in Google Sheets	10 Mar 2024 · 21:34	Applied use cases
Generative AI · Class 18	Using ChatGPT for writing better code	16 Mar 2024 · 20:16	Applied use cases
Generative AI · Class 19	Google Bard – Part 1 (now Gemini)	17 Mar 2024 · 31:25	Applied use cases
Generative AI · Class 20	Introduction to image generation	23 Mar 2024 · 10:36	Image generation
Generative AI · Class 21	Midjourney Part 1	24 Mar 2024 · 36:25	Image generation
Generative AI · Class 22	Midjourney Part 2	30 Mar 2024 · 46:14	Image generation
Generative AI · Class 23	Midjourney Part 3	31 Mar 2024 · 22:11	Image generation
Generative AI · Class 24	Midjourney Part 4 (list of artists by style)	6 Apr 2024 · 28:09	Image generation
Generative AI · Class 25	DALL·E introduction and usage	7 Apr 2024 · 47:33	Image generation
Generative AI · Class 26	Gen AI MCQ; summary of prompt starters + use cases (e-books)		Prompt library

Dates and lengths shown for Classes 16–26 are from the LMS class list. The course note says most videos are task-based with no separate material, so practise alongside each video.

> **Tip** The recorded classes use the ChatGPT versions of their time (3.5, 4, 4-Turbo) and Bard. Interfaces and model names change often; the techniques stay the same.

How LLMs work

What an LLM is

A large language model is a neural network (a **transformer**) trained on huge amounts of text to do one thing extremely well: **predict the next token**. Answering questions, summarising and coding all emerge from repeating that prediction one token at a time.

Your prompt → **Split into tokens** → **Transformer predicts next-token probabilities** →
Sample one token → **Append and repeat**

How they are built

Pre-training	Learn language and facts by predicting the next token on web pages, books and code (self-supervised).
Instruction tuning	Fine-tune on example prompts and good answers so the model follows instructions.
RLHF / preference tuning	Humans rank answers; the model is tuned toward preferred, helpful and safe responses.

Key terms

Token	A chunk of text (roughly ¾ of an English word). Pricing and limits are counted in tokens.
Context window	The maximum tokens the model can consider at once (prompt + conversation + answer).
Temperature	Randomness of sampling. Low (0–0.3) = focused and repeatable; high (0.8+) = creative and varied.
Top-p	Sample only from the smallest set of tokens whose probabilities add up to p.
Hallucination	Confident but false output. Verify facts, numbers and citations.
Embedding	A vector representing meaning; similar texts have nearby vectors. Used for search and RAG.
RAG	Retrieval-augmented generation: fetch relevant documents and include them in the prompt so answers are grounded.
System prompt	Standing instructions that shape every answer (role, rules, style).
Multimodal	Handles images, audio or files as well as text.

The ChatGPT interface (Class 5)

- **Chat**: conversation with memory of the current thread. Start a new chat for an unrelated task so old context does not leak in.
- **Model picker**: faster/cheaper vs more capable models. In the class: 3.5 (fast), 4 (stronger reasoning), 4-Turbo (larger context, cheaper than 4).
- **Playground / API**: control system prompt, temperature and max tokens directly; pay per input and output token.

- **Custom instructions**: tell it once about you and how you want answers.

Applications (Class 4)

Write
Emails, reports, summaries, rewriting for tone

Analyse
Explain data, draft SQL/Excel formulas, classify feedback

Create
Slides, web pages, images, code, learning material

Limits to remember

- Knowledge has a cut-off date unless the tool can browse.
- It can be wrong confidently, so check numbers by calculation.
- Do not paste confidential or personal data into tools your organisation has not approved.

Prompting techniques

The anatomy of a strong prompt

ROLE
Who should answer

CONTEXT
Background and audience

TASK
Exactly what to do

FORMAT
Shape of the output

CONSTRAINTS
Length, tone, must/must-not

EXAMPLES
One good sample

The techniques taught

Technique	Idea	Example prompt
PT1 Output formats	Ask for a specific structure	"List the 5 KPIs as a table with columns KPI, formula, why it matters." Also: JSON, bullet list, CSV, markdown, a fixed word count.
PT2 Role	Assign an expert persona	"You are a senior data analyst at a bank. Review this SQL query for errors."
PT2 Comprehension level	Set the audience's level	"Explain gradient descent to a 12-year-old." / "...to a statistics PhD."
PT2 Tone	Control the voice	"Rewrite this delay notice in a warm, apologetic but confident tone."
PT4 Reverse prompting	Show the output you like and ask for the prompt that produces it	"Here is a product description I love. Write the prompt that would generate descriptions like this for other products."
Few-shot	Give 2–3 input → output examples first	"Review: 'Late delivery' → Negative, Delivery. Review: 'Great fit' → Positive, Product. Review: '...' → "

Technique	Idea	Example prompt
PT6 Chain of thought	Ask it to reason step by step before answering	"Work out the profit step by step, showing each calculation, then give the final number."
PT7 Ask before you answer	Make it ask clarifying questions first	"Before you write my study plan, ask me up to 5 questions about my schedule and goals."
Iterative refinement	Treat the first answer as a draft	"Shorter. Remove jargon. Add one real example. Keep the table."

Before and after

Weak

"Write about churn."

Strong

"You are a telecom retention analyst. In 150 words for a non-technical manager, explain the top 3 reasons customers churn in prepaid mobile and one action for each. Format: numbered list. Avoid jargon."

Tip If the answer is wrong, do not just say "try again". Say what is wrong and what to keep.

Applied use cases

Presentations (Classes 11–13)

1. Ask for an outline: "Create a 10-slide outline for a 15-minute talk on X for Y audience. Title + 3 bullets per slide + speaker note."
2. Refine the storyline (problem → insight → recommendation).
3. Convert to slides: paste into a slide tool with AI design (Gamma, Copilot in PowerPoint, Canva), or ask for VBA/python-pptx code that builds the deck.
4. Add your real charts and numbers; check every fact.

Web pages (Class 14)

```
Create a single-file responsive landing page (HTML + CSS + a little JS) for a data analytics
bootcamp. Sections: hero with CTA, 3 benefits, curriculum accordion, testimonials placeholder,
FAQ, footer. Colour palette navy and amber. Mobile-first. No external libraries.
```

Save the answer as `index.html`, open it in a browser, and iterate ("make the hero shorter", "add a pricing table"). Host it free on Netlify or GitHub Pages.

ChatGPT in Google Sheets (Classes 15–17)

- Add-ons such as "GPT for Sheets" add formulas like `=GPT("Classify sentiment as Positive/Negative/Neutral", A2)` that run per row: classify feedback, extract entities, translate, clean text.
- Native AI in Sheets (Gemini) and Excel (Copilot) can now create formulas, pivots and charts from a prompt.
- **Automation**: ask ChatGPT to write Google Apps Script, for example "Write an Apps Script that emails me every Monday a summary of rows where Status = 'Overdue'". Paste it in Extensions → Apps Script and add a time-driven trigger.

> Per-row AI formulas cost tokens and can change on recalculation. Paste results as values once you are happy.

Better code (Class 18)

Write	"Write a pandas function that ... Include docstring and type hints."
Explain	"Explain this code line by line for a beginner."
Debug	Paste the error message *and* the code: "Why does this raise KeyError, and how do I fix it?"
Refactor	"Make this faster and more readable; keep behaviour identical."
Test	"Write pytest tests including edge cases (empty input, nulls)."
Convert	"Convert this Excel formula / SQL query to pandas."

Google Bard → Gemini (Class 19)

Google's assistant (renamed Gemini in 2024) is strong at live web results and integrates with Gmail, Docs, Sheets and Drive. Compare answers across assistants for important work; they make different mistakes.

Image generation

How image models work (Class 20)

Diffusion models start from random noise and remove it step by step, guided by your text prompt, until an image matching the description appears.

Prompt formula

SUBJECT
what

DETAILS
clothing, objects, action

SETTING
where, when

STYLE
photo, watercolour, 3D, flat vector

LIGHTING / MOOD

CAMERA

golden hour, moody | close-up, wide, 35mm

```
A product photo of a matte-black smartwatch on a marble desk, soft morning window light,
shallow depth of field, minimalist, high detail --ar 4:5 --v 6
```

Midjourney essentials (Classes 21–24)

`/imagine`	Generate 4 images from a prompt
U1–U4 / V1–V4	Upscale or create variations of an image
`--ar 16:9`	Aspect ratio
`--v`	Model version
`--stylize` / `--s`	How strongly Midjourney's own aesthetic is applied
`--chaos`	Variety between the 4 results
`--no text`	Negative prompt: exclude something
`--seed`	Reproduce a similar result
Image prompts	Paste an image URL before the text to guide style or composition
`/describe`	Upload an image and get prompts that describe it (reverse prompting for images)

DALL·E (Class 25)

- Available inside ChatGPT: describe the image conversationally and ask for edits ("make the background blue", "add a laptop on the table").
- Good at following detailed instructions and placing text; Midjourney is often preferred for artistic quality.

> Check usage rights before commercial use, avoid real people's likeness and trademarks, and label AI-generated images where required.

Prompt library

Ready-to-use prompt starters (Class 26)

Learn a topic

```
Act as my tutor for [topic]. Ask me 3
questions to gauge my level, then teach
the
concept in 5 short steps with one example
each, and end with a 5-question quiz.
```

Summarise a document

```
Summarise the text below for a busy
manager: 5 bullet key points, 3 risks,
2 decisions needed. Max 150 words. Quote
numbers exactly as written.
```

Data analysis helper

```
You are a data analyst. Columns: [list].
Suggest 8 business questions this data
can
answer, the chart for each, and the
pandas code to produce it.
```

SQL from English

```
Tables: [schema]. Write a SQL Server
query that [question]. Return one result
table. Then explain each clause in one
line.
```

Interview prep

```
Ask me 10 interview questions for a
[role] one at a time. After each answer,
score it out of 10 and show a stronger
answer.
```

E-book outline (use case)

```
Create an outline for a 10-chapter
beginner e-book on [topic] for
[audience].
For each chapter: title, 3 learning
outcomes, one practical exercise.
Then write chapter 1 in a friendly tone,
about 800 words.
```

Interview Q&A

Q What is the difference between AI, ML, deep learning and generative AI?

AI is the broad goal of machines doing intelligent tasks; ML learns patterns from data; deep learning uses multi-layer neural networks; generative AI is deep learning that creates new content (text, images, code).

Q Why do LLMs hallucinate, and how do you reduce it?

They generate the most plausible next tokens, not verified facts. Reduce it by grounding with documents (RAG), asking for sources, lowering temperature, and checking outputs.

Q What does temperature do?

It controls randomness. Low temperature for factual or code tasks, higher for brainstorming and creative writing.

Q Explain chain-of-thought prompting.

Asking the model to reason through intermediate steps before giving the answer. It improves multi-step maths and logic, and lets you check the reasoning.

Q What is RAG?

Retrieval-augmented generation: search a knowledge base for relevant chunks (using embeddings), add them to the prompt, and have the model answer from them. It keeps answers current and grounded without retraining.

Resources

- ▶ **YouTube: Andrej Karpathy intro to large language models**, the best one-hour overview.
- ▶ **YouTube: 3Blue1Brown but what is a GPT visual intro to transformers**
- ▶ **YouTube: Google prompting essentials** and ▶ **YouTube: prompt engineering tutorial for beginners**
- ▶ **YouTube: Midjourney beginner guide parameters**
- 📘 **OpenAI prompt engineering guide** · 📘 **Anthropic prompt engineering overview**
- 📘 **Midjourney documentation**
- 📘 **DeepLearning.AI short courses (free)**

COURSE HANDBOOK 08

📈 Power BI

Power Query, data modelling, DAX, visuals, interactivity and the Power BI Service

DVA Classes 18–21 (Power BI) · Power BI module · Power BI Services

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#m7**

Overview

Power BI turns raw tables into interactive reports. The workflow is always the same: **get and clean data** (Power Query) → **model** it (relationships, star schema) → **calculate** (DAX) → **visualise** → **publish and share** (Power BI Service).

Get data → **Transform (Power Query / M)** → **Model (relationships)** → **DAX measures** → **Report visuals** → **Publish to Service**

Class	Topics
DVA 18	Power BI components, the three views, ETL, HR analysis
DVA 19	Small multiples, DAX measures and calculated columns, cards, matrix
DVA 20	AdventureWorks case, hierarchies, tooltips, KPI and gauge visuals, filter levels, edit interactions
DVA 21	Parameters, dynamic measures, Q&A, forecasting, buttons, bookmarks, drill-through, connectivity modes
DVA 22	Unpivot (Power Query)

> The two Power BI case studies on the LMS (Basic and E-commerce) could not be downloaded (the transfer failed twice), so they are not solved here. The **Practice project** tab gives a full build you can do on the AdventureWorks sample or the E-commerce data from the Python capstone.

Components & views

Component	What it is
Power BI Desktop	Free Windows app where you build reports (.pbix)
Power BI Service	Cloud (app.powerbi.com) to publish, share, schedule refresh, build dashboards
Power BI Mobile	Phone/tablet apps to view reports
Gateway	Bridge that lets the cloud refresh data from on-premises sources
Report Server	On-premises hosting option

The three views in Desktop

Report view

Canvas where you place visuals, slicers and pages.

Table (Data) view

See the loaded rows, add calculated columns, set data types and formats.

Model view

Tables and relationships: cardinality, cross-filter direction, hide fields.

Connectivity modes (Class 21)

Mode	How	Pros	Cons
Import	Data copied into the compressed in-memory model	Fastest; full DAX and Power Query	Needs scheduled refresh; size limits
DirectQuery	Queries the source on every interaction	Always current; very large data	Slower; some DAX/transform limits; load on source
Live connection	Connects to an existing semantic model (Analysis Services / published dataset)	One governed model for many reports	Cannot change the model in the report
Composite	Mix of Import and DirectQuery tables	Balance of speed and freshness	More complex

Power Query

ETL with Power Query (Class 18)

Home → Transform data opens the Power Query Editor. Every click is recorded as an **Applied Step** in the M language, so refreshing next month replays all cleaning automatically.

Task	Where
Set data types	Click the icon left of the column header (check dates and decimals first)
Remove / choose columns	Home → Choose Columns
Promote headers	Home → Use First Row as Headers
Split column	Transform → Split Column → By Delimiter
Replace values / errors	Transform → Replace Values; right-click → Replace Errors
Unpivot (Class 22)	Select ID columns → Transform → Unpivot Other Columns (wide months → rows)
Merge queries	Home → Merge Queries (a join: left outer, inner, anti…)
Append queries	Home → Append (stack tables with the same columns)
Conditional column	Add Column → Conditional Column (if/else without code)
Group by	Transform → Group By

Task	Where
Column profiling	View → Column quality / distribution / profile

```
let
    Source   = Csv.Document(File.Contents("C:\Data\HR.csv"), [Delimiter=",", Encoding=65001]),
    Headers  = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
    Types    = Table.TransformColumnTypes(Headers, {{"HireDate", type date}, {"Salary",
Currency.Type}}),
    Active   = Table.SelectRows(Types, each [Status] = "Active")
in
    Active
```

Data modelling

Star schema



One central **fact** table (events with numbers: sales, quantity) linked to **dimension** tables (descriptions to slice by). One-to-many from each dimension to the fact.

- **Cardinality**: one-to-many (preferred), one-to-one, many-to-many (avoid unless needed).
- **Cross-filter direction**: single (dimension filters fact) is the default and safest; both-directions can cause ambiguity.
- **Active vs inactive relationships**: only one active path between two tables; activate another inside a measure with `USERELATIONSHIP` (for example, Ship Date vs Order Date).
- **Date table**: create one continuous calendar and Mark as Date Table so time-intelligence functions work.

```
Date = ADDCOLUMNS ( CALENDAR ( DATE(2020,1,1), DATE(2026,12,31) ),
        "Year", YEAR([Date]), "Month", FORMAT([Date], "MMM"), "MonthNo", MONTH([Date]),
        "Quarter", "Q" & QUARTER([Date]) )
```

Hierarchies (Class 20)

Drag Month onto Year in the Fields pane (or right-click → Create hierarchy) to build Year → Quarter → Month → Day, or Category → Sub-category → Product. Visuals then get drill-down arrows.

DAX

Calculated column vs measure (Class 19)

	Calculated column	Measure
Computed	Once per row at refresh, stored	At query time, for the current filter context
Use for	Categorising rows, slicer/axis fields (Age band)	Aggregations shown in visuals (Total Sales, Margin %)
Example	`Profit = Sales[Revenue] - Sales[Cost]`	`Total Profit = SUM(Sales[Profit])`

> **Tip** Rule of thumb: if you will put it on an axis or slicer, use a column; if it is a number to aggregate, use a measure. Measures keep the model small.

Must-know DAX

```
Total Sales      = SUM ( Sales[SalesAmount] )
Orders           = DISTINCTCOUNT ( Sales[OrderID] )
Avg Order Value  = DIVIDE ( [Total Sales], [Orders] )            -- DIVIDE avoids divide-by-zero
errors
Margin %         = DIVIDE ( [Total Profit], [Total Sales] )
Sales East       = CALCULATE ( [Total Sales], Territory[Region] = "East" )   -- change filter
context
% of All Sales   = DIVIDE ( [Total Sales], CALCULATE ( [Total Sales], ALL ( Product ) ) )
Sales LY         = CALCULATE ( [Total Sales], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
YoY %            = DIVIDE ( [Total Sales] - [Sales LY], [Sales LY] )
Sales YTD        = TOTALYTD ( [Total Sales], 'Date'[Date] )
Sales MTD        = TOTALMTD ( [Total Sales], 'Date'[Date] )
Running Total    = CALCULATE ( [Total Sales], FILTER ( ALL ( 'Date' ), 'Date'[Date] <= MAX (
'Date'[Date] ) ) )
Product Rank     = RANKX ( ALL ( Product[ProductName] ), [Total Sales] )
High Value Cust  = COUNTROWS ( FILTER ( Customer, [Total Sales] > 10000 ) )
Revenue (iter)   = SUMX ( Sales, Sales[Qty] * Sales[UnitPrice] )            -- row-by-row then sum
Ship Sales       = CALCULATE ( [Total Sales], USERELATIONSHIP ( Sales[ShipDate], 'Date'[Date] ) )
Age Band (col)   = SWITCH ( TRUE(), Emp[Age] < 30, "Under 30", Emp[Age] < 45, "30-44", "45+" )
```

Filter context in one picture

A measure has no single value; it is recomputed for every cell of a visual using the filters coming from rows/columns, slicers, page and report filters. `CALCULATE` is the function that **modifies** that filter context; `ALL` removes filters; `FILTER` returns a filtered table to iterate.

Variables

```
Growth Label =
VAR cur = [Total Sales]
VAR ly  = [Sales LY]
RETURN IF ( ISBLANK ( ly ), "New", FORMAT ( DIVIDE ( cur - ly, ly ), "0.0%" ) )
```

Visuals & interactivity

Visual picker

Card / Multi-row card	Headline KPIs (Total Sales, Orders)
KPI visual	Value vs target with trend (Class 20)
Gauge	Progress toward a target value
Matrix	Pivot-table style with drill-down and subtotals
Clustered bar / column	Compare categories
Line / area	Trends over time; Analytics pane → **Forecast** (Class 21)
Small multiples	Same chart repeated per category (Class 19): field well "Small multiples"
Map / filled map	Geography
Decomposition tree, Key influencers	AI visuals to explain a metric
Q&A visual	Type a question in plain English ("total sales by region last year")

Filters (Class 20)

Levels: **visual** → **page** → **report** (all pages), plus **drill-through** filters. Types: basic, advanced, Top N, relative date.

Interactivity toolkit (Classes 20–21)

Edit interactions	Format → Edit interactions: choose whether a selected visual filters, highlights or ignores each other visual
Tooltips	Add fields to Tooltips, or build a tooltip report page (Page information → Allow use as tooltip)
Drill-through	Detail page with a drill-through field; right-click a data point → Drill through
Bookmarks	Save a state (filters, visible visuals); combine with the Selection pane for show/hide toggles
Buttons	Actions: page navigation, bookmark, back, drill-through, Q&A
Field parameters	Modeling → New parameter → Fields: let users switch the measure or axis on a visual (dynamic measures)
What-if parameters	Modeling → New parameter → Numeric range: creates a slicer + value measure for scenarios (discount %)
Sync slicers	View → Sync slicers across pages

```
-- dynamic measure without field parameters
Selected Metric =
SWITCH ( SELECTEDVALUE ( MetricPicker[Metric] ),
    "Sales",  [Total Sales],
```

```
    "Profit", [Total Profit],
    "Orders", [Orders] )
```

Power BI Service

Publish and share

1. Desktop → Home → **Publish** → choose a workspace.
2. The Service now holds a **semantic model** (dataset) and a **report**.
3. Pin visuals to a **dashboard** (single-page, tiles from many reports).
4. Dataset settings → **Scheduled refresh** (needs a gateway for on-premises sources).
5. Share: workspace roles (Admin, Member, Contributor, Viewer), share links, or package as an **App**.

Report vs dashboard	Report: multi-page, interactive, built in Desktop. Dashboard: one canvas of pinned tiles in the Service; supports data alerts.
Row-level security	Modeling → Manage roles (e.g. `[Region] = "East"`); assign users in the Service so each person sees only their data
Subscriptions and alerts	Email a report on a schedule; alert when a card/KPI tile crosses a threshold
Export	PDF, PowerPoint, Excel (Analyze in Excel)
Licences	Free (personal), Pro (share with Pro users), Premium Per User / Fabric capacity (large models, wider sharing)

Practice project

Build: Sales performance report (AdventureWorks style)

1. **Data**: FactSales + Product, Customer, Territory, Date tables (AdventureWorks sample, or the E-commerce CSVs from the Python capstone: ORDERS, ORDER_ITEMS, PRODUCTS, CUSTOMERS, ORDER_PAYMENTS, ORDER_REVIEW_RATINGS).
2. **Power Query**: fix types, trim text, build a Date table, unpivot any wide tables.
3. **Model**: star schema, one-to-many single direction, mark the Date table.
4. **Measures**: Total Sales, Orders, AOV, Margin %, Sales LY, YoY %, YTD, Rank.
5. **Page 1, Executive**: 4 KPI cards, sales trend with forecast, sales by category (bar), map by state, slicers for year and channel.

6. **Page 2, Product**: matrix Category → Sub-category → Product with drill-down, Top-N filter, Pareto (column + cumulative % line).
7. **Page 3, Customer drill-through**: details for one customer/state; tooltips page.
8. **Interactivity**: field parameter to switch Sales/Profit/Orders; bookmarks for "chart view / table view"; navigation buttons.
9. **Publish**: workspace, dashboard with pinned KPIs, scheduled refresh, RLS by region.

> **Tip** Interview tip: be ready to explain every measure you used and why it is a measure, not a column.

Interview Q&A

Q Measure vs calculated column?

Columns are computed per row at refresh and stored (good for slicers/axes). Measures are computed at query time in the current filter context (good for aggregations) and do not increase model size.

Q What does CALCULATE do?

It evaluates an expression in a modified filter context: it can add, replace or remove filters. It is how you build "sales for East", "% of total" and time-intelligence measures.

Q SUM vs SUMX?

SUM adds one column. SUMX iterates row by row over a table, evaluates an expression per row (Qty × Price) and then sums the results.

Q Import vs DirectQuery?

Import copies data into memory: fastest, needs refresh. DirectQuery queries the source live: always current and suits very large data, but slower with some limitations.

Q Why a star schema?

Simple one-to-many filters, fewer ambiguous paths, smaller and faster models, and DAX that behaves predictably.

Q How do you implement row-level security?

Create roles with DAX filters in Desktop (Manage roles), test with View as, publish, then assign users or groups to roles in the Service.

Q Report vs dashboard?

A report is multi-page and fully interactive; a dashboard is a single page of pinned tiles from one or more reports, built in the Service, and supports alerts.

Resources

- ▶ YouTube: Guy in a Cube Power BI, tips from Microsoft engineers.
- ▶ YouTube: SQLBI DAX CALCULATE explained, the authority on DAX.
- ▶ YouTube: Pragmatic Works Power BI full course and ▶ YouTube: codebasics Power BI tutorial
- ▶ YouTube: Leila Gharani Power BI tutorial for beginners
- 📘 Microsoft Learn: Power BI documentation · 📘 DAX function reference
- 📘 DAX Guide (SQLBI) · 📘 Star schema guidance

COURSE HANDBOOK 09

📉 Tableau

Marks, table calculations, filters, calculated fields, parameters, LODs and dashboards

Tableau (Self-paced) V7 (9 classes)

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#m8**

Overview

Tableau is drag-and-drop visual analytics. You drop **dimensions** and **measures** on shelves, and Tableau writes the query and draws the chart. Mastery comes from understanding the order Tableau filters and computes things.

Class	Topics
1	Introduction and file types
2	Working in Tableau: connecting data, the workspace
3	Visuals and the Marks card
4	Table calculations
5	Filters, groups, bins, hierarchies
6	Calculated fields, parameters, dashboards
7	Visualisations, detailed dashboards, analytics pane, data blending
8	Level of Detail (LOD) expressions
9	Workbook optimisation and best practices

Tip Both Tableau case studies (Laptops and Sports Equipment) are solved under **Case Studies** with the exact Tableau steps and the answer numbers computed from the data, so you can check your workbook.

Basics & Marks

File types (Class 1)

.twb	Workbook: visual definitions only, no data
.twbx	Packaged workbook: workbook + data extract + images (submit case studies in this)
.hyper (.tde older)	Extract: compressed snapshot of the data
.tds / .tdsx	Saved data source (connection + metadata)
.tbm	Bookmark of a single worksheet

Key ideas (Class 2)

Dimensions vs measures

Dimensions (blue when discrete) are categories that slice data: Region, Product. **Measures** are numbers that get aggregated: SUM(Sales).

Discrete vs continuous

Blue pills are discrete and create headers. **Green pills** are continuous and create axes. A date can be either.

- **Live vs extract**: live queries the source; an extract is a fast snapshot that needs refreshing.
- **Joins** (row level, in the physical layer), **relationships** (logical layer, the default since 2020.2: tables stay separate and join only as needed), and **blending** (Class 7: combine a secondary source per sheet on a linking field; aggregated at the link level).

The Marks card (Class 3)

Mark type	Automatic, bar, line, area, circle, square, text, map, pie, Gantt, polygon
Color	Categories or a gradient for a measure
Size	Encode a measure in mark size
Label	Show values on marks
Detail	Split marks by a field without colour (one dot per store)
Tooltip	Extra fields on hover
Shape / Path / Angle	Shapes for categories; line order; pie slice size

Chart cookbook

Chart	Build
Bar	Dimension on Rows, measure on Columns; sort
Line (trend)	Continuous MONTH(Date) on Columns, measure on Rows
Dual axis	Two measures on Rows → right-click second → Dual Axis → Synchronize Axis; set marks bar + line
Scatter	Measure on Columns and Rows; dimension on Detail; Analytics → Trend Line
Pie	Marks = Pie; dimension on Color; measure on Angle; labels = % of total
Highlight table / heatmap	Dimensions on Rows and Columns, measure on Color, mark = Square
Map	Double-click a geographic field (Country, State, Postcode) or use lat/long
Pareto	Sorted bars + running-total % line on a dual axis
Bullet / Gantt / Box plot	Show Me suggestions with the needed fields

Table calculations

Table calculations run **on the results already in the view** (after aggregation). Right-click a measure → Quick Table Calculation, then **Compute Using** controls the direction (Table across, down, Pane, or specific dimensions).

Quick calc	Business use	Function
Running total	Cumulative sales, Pareto line	`RUNNING_SUM(SUM([Sales]))`
Difference	Change vs previous month	`ZN(SUM([Sales])) - LOOKUP(ZN(SUM([Sales])), -1)`
Percent difference	MoM %	(current – previous) / previous
Percent of total	Market share	`SUM([Sales]) / TOTAL(SUM([Sales]))`
Rank	Top products	`RANK(SUM([Sales]))`
Moving average	Smooth trends (3-month)	`WINDOW_AVG(SUM([Sales]), -2, 0)`
YoY growth	Year-over-year %	Quick calc → Year over Year Growth
Percentile	Position in distribution	`RANK_PERCENTILE(SUM([Sales]))`

> **Tip** If a table calc gives strange numbers, the problem is almost always **Compute Using**. Check the yellow highlight in Edit Table Calculation.

Filters, groups & sets

Order of operations (memorise this)

1. **Extract filters**
2. **Data source filters**
3. **Context filters**
4. **FIXED LOD calculations**
5. **Dimension filters (incl. Top N)**
6. **INCLUDE / EXCLUDE LODs**
7. **Measure filters**
8. **Table calculations and table-calc filters**

Example: to get "Top 15 products *in 2014*", put Year in **context** (right-click → Add to Context) so the year filter is applied before the Top N.

Class 5 tools

Groups	Combine members into a new category (e.g. Laptops + Tablets = Devices). Right-click field → Create → Group, or select marks → paperclip.
Bins	Turn a continuous measure into ranges (age 0–10, 10–20) for histograms. Right-click → Create → Bins.
Hierarchies	Drag one dimension onto another (Category → Sub-category → Product) for +/- drill-down.
Sets	In/Out membership by a condition or Top N (Top 5 product types by quantity); can be dynamic and combined; used for comparisons and highlighting.
Combined fields	Select several dimensions → Create → Combined Field (used in the Laptops case to make 32 configuration groups).

Calculations & parameters

Calculated fields (Class 6)

```
// Row-level
Profit            = [Revenue] * [Gross margin]
Distance          = SQRT( SQUARE([C OSx] - [St OS X]) + SQUARE([C OSy] - [St OS Y]) )
Order Size        = IF [Quantity] >= 100 THEN "Bulk" ELSEIF [Quantity] >= 20 THEN "Medium" ELSE
"Small" END
Region Group      = CASE [Retailer Country] WHEN "Canada" THEN "Americas" WHEN "France" THEN
"Europe" ELSE "Other" END
Year              = YEAR([Order Date])        Days to ship = DATEDIFF('day', [Order Date], [Ship
Date])
Clean Name        = TRIM(UPPER([Name]))       Contains = CONTAINS([Product], "Pro")
// Aggregate
Margin %          = SUM([Profit]) / SUM([Revenue])
Avg price         = SUM([Revenue]) / SUM([Quantity])
```

Parameters

A parameter is a user-controlled variable (number, date, string list). On its own it does nothing; you must use it in a calculation, filter, reference line or Top N.

```
// Dynamic Top N:     Set/Filter → Top → By field → Top [Top N] by SUM(Quantity)
// Dynamic year:      Year Filter = YEAR([Order Date]) = [Select Year]   → put on Filters = True
// Measure switcher:
Selected Measure =
CASE [Select Measure]
  WHEN "Revenue"  THEN SUM([Revenue])
  WHEN "Profit"   THEN SUM([Profit])
  WHEN "Quantity" THEN SUM([Quantity])
END
```

Right-click the parameter → Show Parameter to let viewers change it.

LOD expressions

Why LODs (Class 8)

Normally a measure is aggregated at the level of the dimensions in the view. A **Level of Detail** expression lets you compute at a different level, independent of the view.

Type	Meaning	Example
`{FIXED [dim] : agg}`	Compute at exactly these dimensions, ignoring the view (and dimension filters unless in context)	`{FIXED [Customer ID] : MIN([Order Date])}` = first purchase date (cohorts)
`{INCLUDE [dim] : agg}`	Add a finer dimension than the view, then re-aggregate	`AVG({INCLUDE [Order ID] : SUM([Sales])})` = average order value by region
`{EXCLUDE [dim] : agg}`	Remove a dimension from the view's level	`SUM([Sales]) - SUM({EXCLUDE [Month] : AVG([Sales])})` = difference from the average month
`{agg}`	Table-scoped: one value for the whole data source	`{SUM([Sales])}` = grand total

```
Customer Lifetime Sales = {FIXED [Customer ID] : SUM([Sales])}
Cohort Year             = YEAR({FIXED [Customer ID] : MIN([Order Date])})
Orders per Customer     = {FIXED [Customer ID] : COUNTD([Order ID])}      // then use as a
dimension to histogram
Share of Region         = SUM([Sales]) / SUM({FIXED [Region] : SUM([Sales])})
Store Avg Price         = {FIXED [Store Postcode] : AVG([Retail Price])}   // Laptops case Q5
```

> **Tip** FIXED is computed **before** dimension filters. If a FIXED number ignores your filter, add that filter to context.

Dashboards & stories

Dashboard build (Classes 6–7)

- Set a fixed size (e.g. 1200 × 800) or Automatic; plan a grid with containers (horizontal/vertical) for alignment.
- KPIs on top, main chart centre, detail or table bottom, filters on the right.
- **Actions** (Dashboard → Actions): Filter (click one chart to filter others), Highlight, URL, Go to Sheet, Change Parameter, Change Set Values.
- Use as Filter: the funnel icon on a sheet makes it filter the others.
- Apply a quick filter to "All using this data source" or selected worksheets.
- Show/hide buttons and floating legends keep the layout clean.

Analytics pane (Class 7)

Drag on: constant/average lines, reference bands, distribution bands, box plots, totals, trend lines (with R^2 and p-value), forecast, clusters.

Stories

A Story is a sequence of story points, each a sheet or dashboard with a caption. Use it for presentations: one insight per point, written as a headline ("Proximity drives store sales").

Optimisation & best practice

Performance (Class 9)

✔ Use extracts; hide unused fields before extracting; aggregate the extract to the level you need
✔ Filter early: extract and data-source filters, then context filters
✔ Prefer relationships/joins on integers; avoid blending large sources
✔ Fewer marks per view (thousands, not millions); fewer sheets per dashboard
✔ Avoid many quick filters with "Only relevant values" on large data
✔ Use Boolean and integer calcs instead of string comparisons where possible
✔ Help → Settings and Performance → Start Performance Recording to find slow queries

Design best practice

- One question per sheet; title states the insight.
- Consistent colours: one highlight colour, greys for context.
- Sort bars; start bar axes at zero; avoid 3D and too many pie slices.
- Label directly instead of relying on legends.
- Use tooltips for detail rather than crowding the chart.

Interview Q&A

Q Dimension vs measure, discrete vs continuous?

Dimensions are categorical fields that slice; measures are numeric and aggregated. Discrete (blue) creates headers; continuous (green) creates axes. A field's colour depends on how it is used, not its data type alone.

Q Join vs relationship vs blend?

A join physically merges tables row by row before analysis. A relationship keeps tables separate and joins at the right level for each view (avoids duplication). Blending combines two separate data sources per sheet, aggregated at the linking field.

Q Explain FIXED vs INCLUDE vs EXCLUDE.

FIXED computes at the stated dimensions regardless of the view. INCLUDE adds a finer dimension then aggregates to the view. EXCLUDE removes a view dimension from the calculation.

Q Why would a Top N filter return the wrong products after another filter?

Top N and other dimension filters are evaluated independently at the same stage, so the Top N is computed on the full data, not on the filtered subset. Put the other filter in context so it is applied first.

Q Context filter: when and why?

To make a filter apply before Top N, FIXED LODs and other dimension filters; also to improve performance by shrinking the data early.

Q What is a table calculation and what does Compute Using do?

A calculation on the aggregated results in the view (running total, % of total, rank). Compute Using sets the direction/partition across which it runs.

Resources

- ▶ YouTube: Andy Kriebel Tableau tip, hundreds of short how-tos.
- ▶ YouTube: Tableau LOD expressions explained and ▶ YouTube: Tableau table calculations explained
- ▶ YouTube: Alex The Analyst Tableau tutorial for beginners
- 📘 Tableau Help: order of operations
- 📘 Tableau Help: LOD expressions
- 📘 Tableau Public gallery (inspiration) · Practice: 📘 Workout Wednesday

LEARNING SESSION S1

📗 Advanced Excel: Basic to Advanced

Every Excel skill an analyst needs, from the first click to Power Query, dynamic arrays, VBA and Copilot

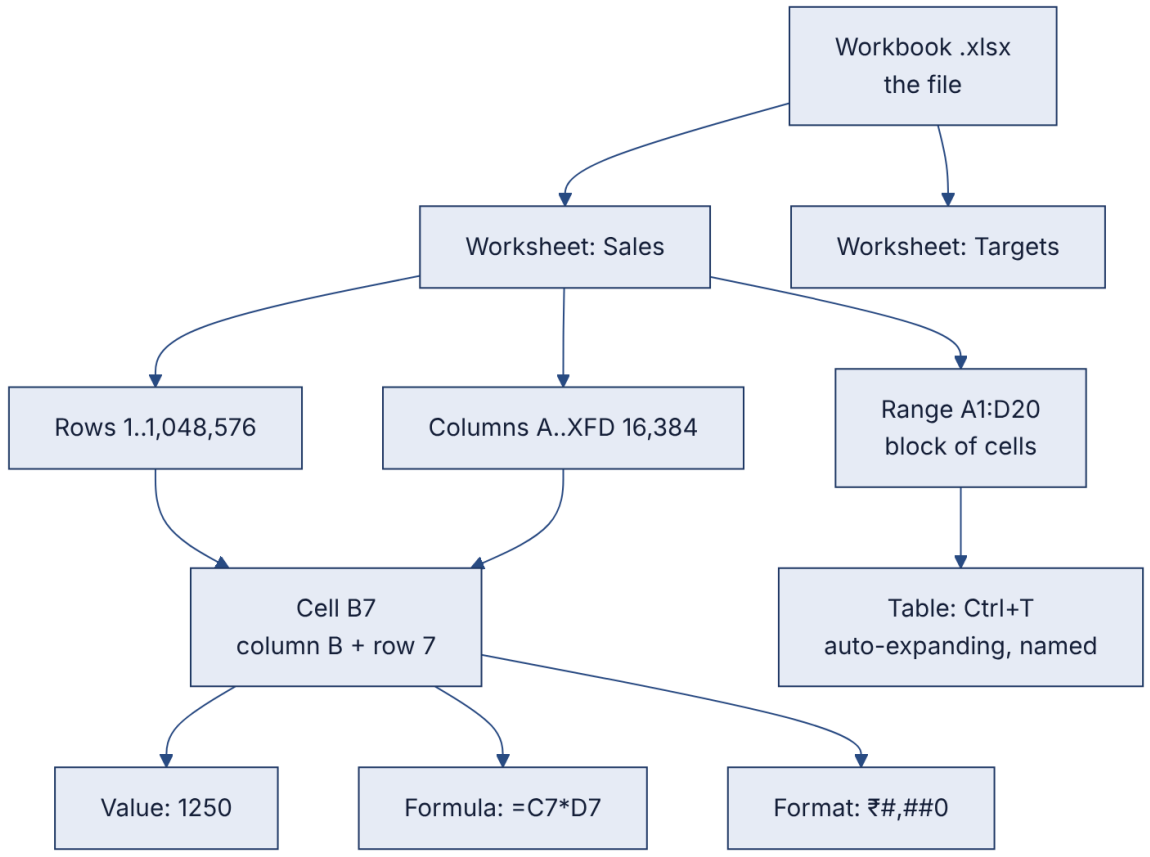
Learning session · non-tech friendly · companion workbook: Excel_Basic_to_Advanced.xlsx (one sheet per topic, live formulas)

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s1**

Roadmap

This session takes you from opening Excel for the first time to building automated dashboards. Each level builds on the previous one. Every topic has a matching sheet in the **Excel Basic-to-Advanced workbook** (see Downloads) with live examples you can edit.

Level	You will learn	Workbook sheets
1 · Foundations	Interface, data entry, cell references, formatting, number formats	01 Environment, 02 References, 03 Custom Formats
2 · Core functions	Math, text, date/time, aggregate and conditional aggregates	04 Number, 05 Text, 06 Date & Time, 07 Aggregate
3 · Logic & lookups	IF family, error handling, VLOOKUP, INDEX/MATCH, XLOOKUP	08 Logical, 09 Lookup
4 · Data tools	Sort, filter, tables, validation, conditional formatting, cleaning	10 Conditional Formatting, 11 Validation, 12 Sort & Filter
5 · Analysis	Pivot tables, slicers, what-if analysis, statistics	13 Pivot Data, 14 What-If
6 · Modern Excel	Dynamic arrays, LET, LAMBDA, GROUPBY, REGEX	15 Dynamic Arrays
7 · Power tools	Power Query, Power Pivot, DAX, data model	16 Power Query Steps
8 · Charts & dashboards	Chart choice, combo/Pareto/waterfall, sparklines, dashboard design	17 Charts, 18 Dashboard Case Study
9 · Automation & AI	Macros/VBA, Office Scripts, Power Automate, Copilot, Python in Excel	19 Automation & AI



The building blocks of every workbook.

Tip Learn by doing: open the workbook, change the blue input cells, and watch the black formula cells update. That feedback loop is how Excel skill sticks.

Level 1 · Foundations

Getting around

- Enter data: type and press `Enter` (down) or `Tab` (right). `Ctrl` + `Enter` fills the same value into all selected cells.
- Edit: `F2`; undo `Ctrl` + `Z`; repeat `F4`.
- Series: type 1, 2 and drag; or type Jan and drag; double-click the fill handle to fill down to the end of adjacent data.
- Flash Fill `Ctrl` + `E`: type the first answer by hand; Excel copies the pattern.

How Excel stores values

Numbers	Right-aligned by default. Up to 15 significant digits (longer IDs like card numbers must be stored as text).
Text	Left-aligned. Numbers stored as text show a green triangle; fix with "Convert to Number" or `=VALUE()`.

Dates	Serial numbers: 1 = 1 Jan 1900; 1 Oct 2026 = 46296. Times are fractions of a day (0.5 = noon).
Booleans / errors	TRUE/FALSE; errors such as #N/A, #VALUE!, #DIV/0!, #REF!, #NAME?, #SPILL!

Cell references

Reference	Copy down	Copy right	Typical use
A1	A2, A3...	B1, C1...	Row-by-row calculations
$A$1	stays	stays	A fixed rate (GST, exchange rate)
A$1	stays	B$1...	Header row in a grid
$A1	$A2...	stays	First column in a grid
Sheet2!B5 / 'Sales Data'!B5	Another sheet (quote names with spaces)		Link inputs
Sales[Amount]	Structured reference to a Table column		Auto-expanding formulas
Named range GST_Rate	Formulas → Define Name		Readable formulas: `=Amount*GST_Rate`

Formatting essentials

- Format Cells `Ctrl` + `1`: Number, Alignment (wrap, merge vs **Center Across Selection**, which is safer), Font, Border, Fill, Protection.
- Cell Styles and Format Painter (double-click to paint many times).
- Custom number formats: see the table in **Building Blocks · Class 1**, including the Indian lakh format and ▲▼ indicators.

Level 2 · Core functions

The 40 functions that cover most analyst work

Group	Functions	Real example
Math	SUM, PRODUCT, ROUND/ROUNDUP/ROUNDDOWN, INT, MOD, ABS, POWER, SQRT, CEILING.MATH, FLOOR.MATH, RAND, RANDBETWEEN, SUMPRODUCT	`=SUMPRODUCT(Qty,Price)` total sales without a helper column
Text	LEFT, RIGHT, MID, LEN, FIND, SEARCH, SUBSTITUTE, REPLACE, TRIM, CLEAN, UPPER, LOWER, PROPER, TEXT, VALUE, CONCAT, TEXTJOIN, EXACT, REPT	`=PROPER(TRIM(A2))` tidy imported names
Date & time	TODAY, NOW, DATE, YEAR, MONTH, DAY, EOMONTH, EDATE, WEEKDAY, WEEKNUM, NETWORKDAYS(.INTL), WORKDAY, DATEDIF, YEARFRAC, HOUR, MINUTE	`=EOMONTH(TODAY(),-1)+1` first day of this month
Aggregate	COUNT, COUNTA, COUNTBLANK, AVERAGE, MEDIAN, MODE, MIN, MAX, SMALL, LARGE, RANK.EQ, PERCENTILE, QUARTILE, STDEV.S	`=RANK.EQ(H2,$H$2:$H$100)` sales-rep ranking

Group	Functions	Real example
Conditional aggregate	SUMIF(S), COUNTIF(S), AVERAGEIF(S), MAXIFS, MINIFS	`=MAXIFS(Amount,City,"Pune")` biggest Pune order
Visible-rows aggregate	SUBTOTAL, AGGREGATE	`=SUBTOTAL(109,H:H)` sum of filtered rows

Criteria tricks in SUMIFS / COUNTIFS

```
=SUMIFS(Amount, City, "Pune")                          exact match
=SUMIFS(Amount, Product, "*phone*")                    wildcard: contains "phone"
=SUMIFS(Amount, Amount, ">=10000")                     comparison as text
=SUMIFS(Amount, Date, ">="&$K$1, Date, "<="&$K$2)    between two dates held in cells
=SUM(SUMIFS(Amount, City, {"Pune","Mumbai"}))          OR across two cities
=COUNTIFS(Region, "<>North")                          not equal
=COUNTIF(A:A, A2)>1                                    duplicate check
```

Text recipes

```
First name              =LEFT(A2, FIND(" ", A2) - 1)
Last name               =RIGHT(A2, LEN(A2) - FIND(" ", A2))
Email domain            =MID(B2, FIND("@", B2) + 1, 100)
Mask a phone number     ="XXXXXX" & RIGHT(C2, 4)
Count words             =LEN(TRIM(A2)) - LEN(SUBSTITUTE(A2, " ", "")) + 1
Pad an ID to 6 digits   =TEXT(D2, "000000")
```

Date recipes

```
Age in years            =DATEDIF(DOB, TODAY(), "Y")
Financial year (Apr–Mar) ="FY" & IF(MONTH(B2)>=4, YEAR(B2)+1, YEAR(B2))
Quarter (Apr start)     ="Q" & ROUNDUP(MOD(MONTH(B2)-4, 12)/3 + 0.01, 0)
Weekday name            =TEXT(B2, "dddd")
Due date in 10 workdays =WORKDAY(B2, 10, Holidays)
Hours between times     =(End - Start) * 24
```

Level 3 · Logic & lookups

Logical functions

```
=IF(H2>=50000, "Key account", "Regular")
=IF(H2>=50000, "Gold", IF(H2>=20000, "Silver", "Bronze"))          nested IF
=IFS(H2>=50000, "Gold", H2>=20000, "Silver", TRUE, "Bronze")         IFS (365/2019+)
=SWITCH(D2, "N", "North", "S", "South", "Other")                      match exact values
=IF(AND(Status="Paid", Days<=30), "On time", "Late")
=IF(OR(City="Pune", City="Mumbai"), "Maharashtra", "Other")
=IFERROR(H2/G2, 0)          =IFNA(VLOOKUP(...), "Not found")         error handling
=ISBLANK(A2)  =ISNUMBER(SEARCH("pro", B2))  =ISERROR(...)             IS functions
```

Lookups: the evolution

Function	Formula (find a product's price)	Pros	Cons
VLOOKUP	`=VLOOKUP(A2, Products!$A:$C, 3, FALSE)`	Known everywhere	Only looks right; breaks if a column is inserted; FALSE is easy to forget
HLOOKUP	`=HLOOKUP(J2, $B$1:$M$2, 2, FALSE)`	Horizontal tables	Same limits
INDEX + MATCH	`=INDEX(Products!C:C, MATCH(A2, Products!A:A, 0))`	Any direction, robust, fast	Two functions to learn
Two-way INDEX/MATCH	`=INDEX(Grid, MATCH(rep,Reps,0), MATCH(month,Months,0))`	Row + column lookup	
XLOOKUP (365/2021)	`=XLOOKUP(A2, Products[ID], Products[Price], "Missing")`	Any direction, default not-found, last-match search, returns whole rows	Not in Excel 2016/2019
Approximate match	`=VLOOKUP(Score, Slabs, 2, TRUE)` or `=XLOOKUP(Score, Slab, Rate, , -1)`	Tax slabs, grade bands, commission tiers	Slab table must be sorted ascending
CHOOSE	`=CHOOSE(WEEKDAY(B2,2), "Mon","Tue",...)`	Pick from a list by position; enables left VLOOKUP	
INDIRECT / OFFSET	`=SUM(INDIRECT(A1&"!H:H"))`	Dynamic references	Volatile (slow); avoid in big files

Real example: commission slabs

Slab table: 0 → 2%, 1,00,000 → 3%, 5,00,000 → 5%. Rep sales ₹3,40,000. `=XLOOKUP(340000, Slab, Rate, , -1)` finds the largest slab ≤ sales → 3% → commission ₹10,200.

Level 4 · Data tools

Excel Tables (Ctrl+T): do this first, always

- Formulas fill down automatically and use names: `=[@Qty]*[@Price]`.
- New rows are included in pivots, charts and formulas automatically.
- Built-in Total Row, slicers and banded styles.

Cleaning toolkit

Remove Duplicates	Data → Remove Duplicates (choose key columns)
Text to Columns	Split by delimiter or fixed width; also converts text dates
Find & Replace	Wildcards * and ?; replace double spaces; Options → match entire cell
Go To Special → Blanks	Fill blanks with the value above: select, type `=` + up arrow, `Ctrl` + `Enter`

TRIM/CLEAN/PROPER	Fix spaces, non-printing characters, case
Data Validation	Drop-down lists (List source), whole number limits, dates, custom formula rules (`=COUNTIF(A:A,A2)=1` blocks duplicates), input messages and error alerts
Copilot "Clean data" (365)	Detects inconsistent text, spaces and number formats and suggests fixes

Sorting and filtering

Multi-level and custom-list sorts; filter by value, colour, number/date conditions; Advanced Filter with criteria ranges; `SUBTOTAL` / `AGGREGATE` for visible rows; the modern `=FILTER()` and `=SORT()` formulas return live results without touching the source.

Conditional formatting

Rules, data bars, colour scales, icon sets, and formula-based rules (highlight the row, overdue items, weekends, duplicates). Manage rules: Home → Conditional Formatting → Manage Rules to set order and "Stop If True".

Protection and collaboration

Unlock input cells (Format Cells → Protection → uncheck Locked), then Review → Protect Sheet. Protect Workbook locks structure. Co-authoring in OneDrive/SharePoint; comments with @mentions; Show Changes history.

Level 5 · Analysis

Pivot tables step by step

1. Click in a clean Table → Insert → PivotTable → New Worksheet.
2. Drag fields: Rows (City), Columns (Year), Values (Sum of Amount), Filters (Category).
3. Value Field Settings: Sum/Count/Average; **Show Values As** % of Grand Total, % of Row, Difference From previous (MoM/YoY), Running Total, Rank.
4. Group dates (Years, Quarters, Months) and numbers into bins.
5. Insert Slicers and a Timeline; connect them to several pivots via Report Connections.
6. Design: Tabular layout, repeat labels, subtotals on/off, styles. Refresh with `Alt` + `F5`; Refresh All `Ctrl` + `Alt` + `F5`.

- **Calculated field**: Margin % = Profit / Sales (works on sums).

- **GETPIVOTDATA**: pull a number from a pivot into a report cell that survives re-layout.
- **Pivot charts** follow the pivot's filters and slicers.
- Formula alternatives in 365: `=GROUPBY()` and `=PIVOTBY()` give pivot results that update automatically.

What-if analysis

Goal Seek	Data → What-If → Goal Seek: "What price gives ₹10 lakh profit?" Set cell = profit, To value = 1000000, By changing = price
Scenario Manager	Save Best / Base / Worst sets of inputs and produce a summary report
Data Table	One- or two-variable sensitivity grid: EMI for different rates × tenures
Solver (add-in)	Optimise with constraints: product mix that maximises profit within machine hours
Forecast Sheet	Data → Forecast Sheet: exponential-smoothing forecast with confidence bounds (`FORECAST.ETS`)

Real example: loan EMI

₹10 lakh, 9% annual, 5 years: `=PMT(9%/12, 60, -1000000)` = **₹20,758** per month. A two-variable Data Table shows EMI for 8–11% × 3–7 years in one grid.

Statistics in Excel

Enable the **Analysis ToolPak** (File → Options → Add-ins): Descriptive Statistics, Histogram, Correlation, Regression, t-Tests, ANOVA. Functions: CORREL, SLOPE, INTERCEPT, RSQ, FORECAST.LINEAR, T.TEST, CHISQ.TEST, NORM.DIST.

Level 6 · Modern Excel

Dynamic arrays (Excel 365 / 2021+)

One formula returns many values that "spill" into neighbouring cells. Refer to the whole spill with `#` : `=SUM(F2#)` . A #SPILL! error means something blocks the spill range.

Function	Example	Returns
UNIQUE	`=UNIQUE(Sales[City])`	Distinct cities
SORT / SORTBY	`=SORT(Sales, 5, -1)`	Table sorted by 5th column descending
FILTER	`=FILTER(Sales, (Sales[City]="Pune")*(Sales[Amount]>5000), "None")`	Rows meeting both conditions (* = AND, + = OR)
SEQUENCE	`=SEQUENCE(12,1,DATE(2026,4,1),31)`	Series of numbers/dates
XLOOKUP / XMATCH	`=XLOOKUP(A2, IDs, Sales[[Name]:[City]])`	Several columns at once

Function	Example	Returns
TAKE / DROP / CHOOSECOLS / CHOOSEROWS	`=TAKE(SORT(Sales,5,-1), 10)`	Top 10 rows
VSTACK / HSTACK	`=VSTACK(Jan!A2:E100, Feb!A2:E100)`	Append sheets
TEXTSPLIT / TEXTBEFORE / TEXTAFTER	`=TEXTSPLIT(A2, ",")`	Split text to columns
GROUPBY / PIVOTBY / PERCENTOF	`=GROUPBY(Sales[City], Sales[Amount], SUM, , , -2)`	Summary sorted by total
REGEXTEST / REGEXEXTRACT / REGEXREPLACE	`=REGEXEXTRACT(A2, "[A-Z]{5}[0-9]{4}[A-Z]")`	PAN-format code from text

LET and LAMBDA

```
=LET(sales, SUM(Sales[Amount]), cost, SUM(Sales[Cost]),
     margin, (sales - cost) / sales,
     TEXT(margin, "0.0%"))                                  name intermediate results

Define a reusable function: Formulas → Name Manager → New
   Name:  GSTINCL     Refers to:  =LAMBDA(amount, rate, amount * (1 + rate))
Use:   =GSTINCL(H2, 18%)
```

Level 7 · Power tools

Power Query: repeatable data cleaning

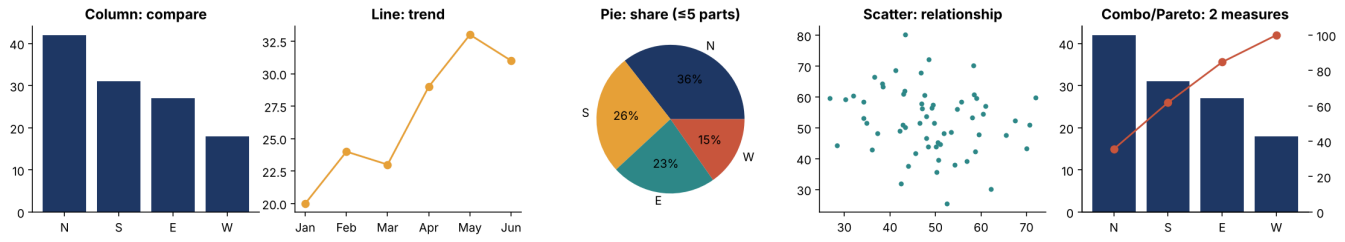
1. Data → Get Data → From File/Folder/Database/Web. "From Folder" combines every monthly file in a folder automatically.
2. In the editor: set types, remove columns, filter rows, split columns, replace values, trim, fill down, unpivot, merge (join) and append queries, group by, add conditional and custom columns.
3. Close & Load to a Table or only to the Data Model.
4. Next month: drop the new file in the folder → Refresh. Every step reruns.

Tip Power Query records each click as an M-language step (Applied Steps). The same engine is in Power BI, so this skill transfers directly.

Power Pivot and the Data Model

- Load several tables (Sales, Products, Calendar) and connect them with relationships (Data → Relationships or the Diagram View), instead of VLOOKUPs.
- Write DAX measures: `Total Sales := SUM(Sales[Amount])`, `Sales LY := CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Calendar[Date]))`.
- Pivot tables built on the Data Model can handle millions of rows and use those measures.

Level 8 · Charts & dashboards



Start from the question, not the chart gallery.

Advanced chart	Use	How
Combo (column + line, secondary axis)	Revenue and margin % together	Change Chart Type → Combo
Pareto	80/20: few products, most complaints	Insert → Statistic → Pareto
Waterfall	Profit bridge: revenue → costs → profit	Insert → Waterfall; set totals
Histogram / Box & Whisker	Distributions	Insert → Statistic charts
Funnel	Visit → cart → checkout → paid	Insert → Funnel
Map chart	Sales by state	Insert → Maps (needs region names)
Sparklines	Mini trend inside a cell	Insert → Sparklines
Dynamic chart	Chart changes with a drop-down	Chart on a FILTER/INDEX result driven by a validation list

Dashboard design rules

✔ One screen; KPIs in the top row (cards with value + vs target + trend arrow)
✔ Main trend chart, then breakdowns; details last
✔ Slicers/timelines on one side, connected to all pivots
✔ Consistent colours: one highlight colour, greys for context
✔ Gridlines off, aligned shapes, titles that state the insight
✔ Raw data and calculations on hidden or separate sheets; protect the dashboard

The workbook's **18 Dashboard Case Study** sheet builds a monthly sales dashboard for a sample retailer with live KPIs, a store league table and charts.

Level 9 · Automation & AI

Macros and VBA

1. Show the Developer tab (File → Options → Customize Ribbon).
2. Record Macro → do the steps (format, sort, filter) → Stop. Use Relative References for steps that should work from the current cell.
3. Save as **.xlsm**. Run from View → Macros or assign to a button.
4. Edit in the VBA editor (`Alt` + `F11`):

```
Sub FormatReport()
    Dim ws As Worksheet
    For Each ws In ThisWorkbook.Worksheets          ' loop every sheet
        ws.Rows(1).Font.Bold = True
        ws.Columns("A:H").AutoFit
    Next ws
    MsgBox "Formatted " & ThisWorkbook.Worksheets.Count & " sheets"
End Sub
```

Office Scripts and Power Automate (cloud)

Excel for the web → Automate tab → Record Actions creates a TypeScript Office Script. Power Automate can run it on a schedule or when a file arrives: e.g. "every Monday 9 AM, refresh the sales workbook and email the summary to the team".

AI in Excel

Copilot chat	"Show sales by city as a chart", "Add a column for margin %", "What are the top 3 insights?"
Formula help	Generate, explain ("Explain this formula") and fix formulas from the error indicator
`=COPILOT()`	AI inside a cell: classify reviews, extract cities, summarise comments row by row
Agent Mode	Multi-step builds (a full model or dashboard) from a single request
Python in Excel `=PY()`	pandas, seaborn and scikit-learn inside the grid, run in Microsoft's cloud
External assistants	ChatGPT, Claude or Gemini to write formulas, VBA or Power Query M; always test on a copy

> Check AI-written formulas on a few rows you can verify by hand, and keep sensitive data out of tools your company has not approved.

Errors & auditing

Error	Cause	Fix
#N/A	Lookup value not found (extra spaces, text vs number)	TRIM, VALUE, IFNA/XLOOKUP not-found argument
#VALUE!	Wrong type (text in maths)	Clean the input; VALUE()
#DIV/0!	Divide by zero/blank	`=IFERROR(a/b,0)` or `=IF(b=0,"",a/b)`
#REF!	Referenced cell deleted	Undo; rebuild with INDEX/MATCH or Tables
#NAME?	Misspelt function or unquoted text; function not in your Excel version	Check spelling and version
#SPILL!	Dynamic array blocked	Clear the cells in the way
#NUM!	Impossible number (IRR not converging, SQRT of negative)	Check inputs
Circular reference	Formula refers to itself	Formulas → Error Checking → Circular References

Auditing tools (Formulas tab): Trace Precedents/Dependents, Evaluate Formula (step through), Watch Window, Show Formulas (`Ctrl`+`` ` ``), Inquire add-in to compare workbooks.

Interview Q&A

Q VLOOKUP vs INDEX/MATCH vs XLOOKUP?

VLOOKUP looks only to the right and breaks when columns move. INDEX/MATCH looks any direction and is robust. XLOOKUP does both in one function with a built-in not-found value and is the modern default in Excel 365.

Q How would you find duplicates?

Conditional formatting → Duplicate Values; `=COUNTIF($A:$A,A2)>1`; or UNIQUE / Remove Duplicates to get a clean list.

Q What is a pivot cache and why does it matter?

A snapshot of the source data stored with the pivot; you must Refresh to see changes. Pivots built from the same source can share one cache.

Q When would you use Power Query instead of formulas?

When the same cleaning must be repeated on new files (monthly exports), when combining many files, or when the data is too messy for formulas. It is recorded once and refreshed.

Q How do you make a chart update automatically?

Base it on an Excel Table or a dynamic array formula; new rows are then included without editing the chart range.

Q What does SUMPRODUCT do?

Multiplies arrays element by element and sums the results, so it can do weighted sums and multi-condition counts without helper columns.

Q How do you protect formulas but let users enter data?

Unlock input cells, keep formula cells locked, then Protect Sheet (optionally with a password and allowed actions).

Resources

- ▶ YouTube: Simplilearn Excel full course · ▶ YouTube: Simplilearn advanced Excel tutorial · ▶ YouTube: Simplilearn Excel pivot table tutorial
- ▶ YouTube: Leila Gharani XLOOKUP · ▶ YouTube: Leila Gharani dynamic arrays FILTER SORT UNIQUE · ▶ YouTube: Leila Gharani Power Query
- ▶ YouTube: Chandoo Excel dashboard tutorial · ▶ YouTube: ExcelIsFun Excel basics to advanced · ▶ YouTube: MyOnlineTrainingHub Excel GROUPBY PIVOTBY
- ▶ YouTube: Kenji Excel copilot tutorial · ▶ YouTube: Excel VBA macros for beginners
- 📘 Microsoft Excel help and learning · 📘 Excel functions (alphabetical)
- 📘 Excel Campus: 2025 updates · 📘 Excel blog (What's new)

LEARNING SESSION S2

SQL: Basic to Advanced

Query, combine and analyse data in databases, from SELECT to window functions, CTEs, performance and analytics patterns

Learning session · non-tech friendly · examples in SQL Server (T-SQL) with notes for MySQL/PostgreSQL

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s2**

Roadmap

SQL is the most-asked skill in data analyst job descriptions. You write a sentence-like query describing the data you want, and the database returns it. This session goes from your first query to the patterns used in real analytics teams.

Stage	Topics	Practice
1 · Foundations	Tables, keys, data types, SELECT, WHERE, ORDER BY, DISTINCT, TOP	**Building Blocks · Class 4**
2 · Summaries	Aggregates, GROUP BY, HAVING, CASE	SQL Retail case Q1–Q10
3 · Combining	Joins, set operators, subqueries	SQL Retail case Q11–Q15
4 · Advanced querying	CTEs, window functions, pivot/unpivot, date logic	SQL Mobile case
5 · Building	DDL, DML, constraints, views, procedures, functions, transactions	Create your own RetailDB
6 · Performance	Indexes, execution plans, sargable filters	Compare plans
7 · Analytics patterns	Top-N per group, running totals, MoM growth, cohorts, retention, funnels, dedupe, gaps & islands	Patterns tab



Memorise this order: it explains most "why doesn't my query work" moments.

1 · Foundations

```sql
-- Read a table
SELECT TOP 100 * FROM Orders;
-- Choose and rename columns, calculate
SELECT order_id, amount, amount * 0.18 AS gst, amount * 1.18 AS amount_incl_gst FROM Orders;
-- Filter rows
SELECT * FROM Orders
WHERE order_date >= '2026-04-01' AND store_type IN ('e-Shop','MBR') AND amount BETWEEN 1000 AND
50000;
```

```sql
-- Patterns and NULLs
SELECT * FROM Customer WHERE name LIKE 'Ra%' AND email IS NOT NULL;
-- Unique values and sorting
SELECT DISTINCT city FROM Customer ORDER BY city;
SELECT TOP 5 WITH TIES product_id, price FROM Product ORDER BY price DESC;   -- ties included
```

Concept	Remember
NULL	"Unknown". `NULL = NULL` is not TRUE; use IS NULL. Aggregates ignore NULLs; COUNT(*) counts rows, COUNT(col) counts non-NULL values.
Strings	Single quotes: 'Pune'. Dates as 'YYYY-MM-DD' strings are safest.
Integer division	`5/2 = 2` in SQL Server; write `5/2.0` or `CAST(x AS DECIMAL(10,2))`.
Comments	`-- one line`, `/* block */`

2 · Summaries

```sql
SELECT store_type,
       COUNT(*)                       AS orders,
       COUNT(DISTINCT customer_id)    AS customers,
       SUM(amount)                    AS revenue,
       AVG(amount * 1.0)              AS aov,
       MIN(order_date)                AS first_order,
       MAX(order_date)                AS last_order
FROM Orders
WHERE amount > 0                        -- row filter (before grouping)
GROUP BY store_type
HAVING SUM(amount) > 1000000            -- group filter (after grouping)
ORDER BY revenue DESC;
```

CASE: if-then logic in SQL

```sql
SELECT order_id, amount,
       CASE WHEN amount >= 10000 THEN 'High'
            WHEN amount >=  2000 THEN 'Medium'
            ELSE 'Low' END                          AS order_band
FROM Orders;

-- Conditional aggregation (pivot-like counts in one pass)
SELECT city,
       SUM(CASE WHEN gender = 'F' THEN 1 ELSE 0 END)      AS female,
       SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END)      AS male,
       AVG(CASE WHEN qty < 0 THEN 1.0 ELSE 0 END)        AS return_rate
FROM Orders o JOIN Customer c ON o.customer_id = c.customer_id
GROUP BY city;
```

Functions you will use daily

Dates	`GETDATE()`, `CAST(GETDATE() AS DATE)`, `YEAR(d)`, `DATEPART(QUARTER,d)`, `DATENAME(WEEKDAY,d)`, `DATEADD(MONTH,-1,d)`, `DATEDIFF(DAY,a,b)`, `EOMONTH(d)`, `DATETRUNC(MONTH,d)`, `FORMAT(d,'yyyy-MM')`
Text	`LEN`, `LEFT`, `RIGHT`, `SUBSTRING`, `CHARINDEX`, `REPLACE`, `TRIM`, `UPPER`, `CONCAT`, `CONCAT_WS`, `STRING_AGG`, `STRING_SPLIT`
Numbers	`ROUND`, `CEILING`, `FLOOR`, `ABS`, `POWER`, `SQRT`, `GREATEST`, `LEAST`
NULL & type	`ISNULL`, `COALESCE`, `NULLIF (avoid divide by zero: x / NULLIF(y,0))`, `CAST`, `TRY_CONVERT`

3 · Combining



Joins add columns from another table; set operators add rows.

```sql
-- Orders with customer and product details
SELECT o.order_id, c.name, c.city, p.product_name, oi.qty, oi.qty * p.price AS line_value
FROM Orders o
JOIN Customer   c  ON o.customer_id = c.customer_id
JOIN Order_Item oi ON o.order_id    = oi.order_id
JOIN Product    p  ON oi.product_id = p.product_id;

-- Anti-join: products never sold
SELECT p.* FROM Product p
WHERE NOT EXISTS (SELECT 1 FROM Order_Item oi WHERE oi.product_id = p.product_id);
```

```
-- Join on two keys (category + sub-category)
... JOIN prod_cat_info pc ON t.prod_cat_code = pc.prod_cat_code AND t.prod_subcat_code =
pc.prod_sub_cat_code
```

> **Join fan-out:** if the right table has several rows per key, rows multiply and SUMs inflate. Check counts before and after a join, and aggregate the many-side first if needed.

Subqueries

Scalar	`WHERE amount > (SELECT AVG(amount) FROM Orders)`
List (IN)	`WHERE customer_id IN (SELECT customer_id FROM Orders WHERE YEAR(order_date)=2026)`
Derived table (inline view)	`FROM (SELECT ... GROUP BY ...) AS t`
Correlated	Inner query uses outer row: orders above that customer's own average
EXISTS / NOT EXISTS	Stops at first match; NULL-safe, often faster than IN / NOT IN

4 · Advanced querying

CTEs (WITH): readable step-by-step queries

```
WITH monthly AS (
    SELECT DATETRUNC(MONTH, order_date) AS month, SUM(amount) AS revenue
    FROM Orders GROUP BY DATETRUNC(MONTH, order_date)
),
growth AS (
    SELECT month, revenue,
           LAG(revenue) OVER (ORDER BY month) AS prev_revenue
    FROM monthly
)
SELECT month, revenue,
       ROUND(100.0 * (revenue - prev_revenue) / NULLIF(prev_revenue, 0), 1) AS mom_growth_pct
FROM growth ORDER BY month;
```

Window functions

Function	Use
ROW_NUMBER, RANK, DENSE_RANK, NTILE(4)	Rankings, top-N per group, quartiles
SUM/AVG/COUNT ... OVER (PARTITION BY ...)	Group totals next to each row (% of total)
... OVER (ORDER BY ... ROWS BETWEEN ...)	Running totals, moving averages
LAG / LEAD	Previous/next row: MoM change, days between orders
FIRST_VALUE / LAST_VALUE	First purchase category per customer

Function	Use
PERCENT_RANK, CUME_DIST, PERCENTILE_CONT	Percentiles and distributions

```
SELECT customer_id, order_date, amount,
       ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date)        AS order_no,
       SUM(amount)  OVER (PARTITION BY customer_id ORDER BY order_date
                          ROWS UNBOUNDED PRECEDING)                            AS
lifetime_value_so_far,
       DATEDIFF(DAY, LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date),
order_date) AS days_since_prev,
       100.0 * amount / SUM(amount) OVER ()                                    AS
pct_of_all_revenue
FROM Orders;
```

Pivot and unpivot

```
SELECT * FROM (SELECT city, store_type, amount FROM Sales) s
PIVOT (SUM(amount) FOR store_type IN ([e-Shop],[MBR],[TeleShop],[Flagship store])) p;

SELECT product, month, units
FROM MonthlyUnits UNPIVOT (units FOR month IN ([Jan],[Feb],[Mar])) u;
```

Recursive CTE (hierarchies, date series)

```
WITH org AS (
    SELECT emp_id, name, manager_id, 1 AS lvl FROM Employee WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.name, e.manager_id, o.lvl + 1 FROM Employee e JOIN org o ON e.manager_id =
o.emp_id
)
SELECT * FROM org ORDER BY lvl;
```

5 · Building

DDL and DML recap

CREATE / ALTER / DROP / TRUNCATE build structure; INSERT / UPDATE / DELETE / MERGE change rows. Full syntax with examples: **Building Blocks · Class 3**.

```
-- Upsert: update existing customers, insert new ones
MERGE Customer AS tgt
USING Staging_Customer AS src ON tgt.customer_id = src.customer_id
WHEN MATCHED THEN UPDATE SET tgt.city = src.city, tgt.email = src.email
WHEN NOT MATCHED THEN INSERT (customer_id, name, city, email) VALUES (src.customer_id, src.name,
src.city, src.email);
```

Views, stored procedures, functions

```
CREATE VIEW vw_monthly_sales AS
SELECT DATETRUNC(MONTH, order_date) AS month, store_type, SUM(amount) AS revenue
FROM Orders GROUP BY DATETRUNC(MONTH, order_date), store_type;

CREATE PROCEDURE usp_sales_by_city @from DATE, @to DATE AS
BEGIN
    SET NOCOUNT ON;
    SELECT c.city, SUM(o.amount) AS revenue
    FROM Orders o JOIN Customer c ON o.customer_id = c.customer_id
    WHERE o.order_date BETWEEN @from AND @to
    GROUP BY c.city ORDER BY revenue DESC;
END;
EXEC usp_sales_by_city '2026-04-01', '2026-09-30';

CREATE FUNCTION dbo.fn_fy(@d DATE) RETURNS VARCHAR(6) AS
BEGIN RETURN 'FY' + CAST(YEAR(@d) + CASE WHEN MONTH(@d) >= 4 THEN 1 ELSE 0 END AS VARCHAR(4)); END;
```

Transactions

```
BEGIN TRY
    BEGIN TRAN;
        UPDATE Account SET balance = balance - 5000 WHERE acc_id = 1;
        UPDATE Account SET balance = balance + 5000 WHERE acc_id = 2;
    COMMIT;              -- both or nothing (atomicity)
END TRY
BEGIN CATCH
    ROLLBACK; THROW;
END CATCH;
```

Temp tables (`#tmp`) hold intermediate results for a session; table variables (`@t`) for small sets.

6 · Performance

- **Indexes** are like a book index: a **clustered** index sorts the table itself (usually the primary key); **non-clustered** indexes are extra sorted lookups. `CREATE INDEX ix_orders_date ON Orders(order_date) INCLUDE (amount);`
- Read the **execution plan** (`Ctrl` + `M` in SSMS): look for table scans on large tables and big estimated vs actual row differences.
- Write **sargable** filters (index-friendly): `WHERE order_date >= '2026-01-01' AND order_date < '2027-01-01'` instead of `WHERE YEAR(order_date) = 2026` .
- Select only needed columns; filter early; avoid functions on indexed columns; prefer EXISTS over IN for large subqueries; avoid SELECT DISTINCT as a fix for bad joins.
- Use `SET STATISTICS IO, TIME ON` to measure.

7 · Analytics patterns

```
-- 1. Top 3 products per category
SELECT * FROM (
```

```sql
  SELECT category, product_name, SUM(line_value) AS sales,
         DENSE_RANK() OVER (PARTITION BY category ORDER BY SUM(line_value) DESC) AS rk
  FROM vw_order_lines GROUP BY category, product_name) t
WHERE rk <= 3;

-- 2. Remove duplicates, keep the latest row
WITH d AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY email ORDER BY updated_at DESC) rn FROM
Customer_Raw)
DELETE FROM d WHERE rn > 1;

-- 3. Monthly cohort retention
WITH first_month AS (
  SELECT customer_id, DATETRUNC(MONTH, MIN(order_date)) AS cohort FROM Orders GROUP BY
customer_id),
activity AS (
  SELECT DISTINCT o.customer_id, f.cohort, DATEDIFF(MONTH, f.cohort, DATETRUNC(MONTH,
o.order_date)) AS month_no
  FROM Orders o JOIN first_month f ON o.customer_id = f.customer_id)
SELECT cohort, month_no, COUNT(*) AS active_customers,
       100.0 * COUNT(*) / FIRST_VALUE(COUNT(*)) OVER (PARTITION BY cohort ORDER BY month_no) AS
retention_pct
FROM activity GROUP BY cohort, month_no ORDER BY cohort, month_no;

-- 4. Funnel conversion
SELECT COUNT(DISTINCT CASE WHEN event = 'view'     THEN user_id END) AS viewers,
       COUNT(DISTINCT CASE WHEN event = 'add_cart' THEN user_id END) AS carted,
       COUNT(DISTINCT CASE WHEN event = 'purchase' THEN user_id END) AS buyers
FROM Events WHERE event_date = '2026-10-26';

-- 5. 7-day moving average of daily sales
SELECT sale_date, daily_sales,
       AVG(daily_sales) OVER (ORDER BY sale_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma_7d
FROM DailySales;

-- 6. Customers whose spend dropped more than 50% vs previous year
WITH y AS (SELECT customer_id, YEAR(order_date) AS yr, SUM(amount) AS spend
           FROM Orders GROUP BY customer_id, YEAR(order_date)),
     z AS (SELECT *, LAG(spend) OVER (PARTITION BY customer_id ORDER BY yr) AS prev_spend FROM y)
SELECT customer_id, yr, spend, prev_spend
FROM z WHERE spend < 0.5 * prev_spend;      -- filter on a window result in the outer query
```

> **Tip** Pattern 6 shows a dialect difference: Snowflake, BigQuery and Databricks support `QUALIFY` to filter on window results; in SQL Server, wrap the query in a CTE and filter in the outer WHERE.

Dialects & new features

Task	SQL Server	MySQL	PostgreSQL
First N rows	`SELECT TOP 10`	`LIMIT 10`	`LIMIT 10`
Today	`GETDATE()`	`NOW()`	`NOW()` / `CURRENT_DATE`
Add days	`DATEADD(DAY,7,d)`	`DATE_ADD(d, INTERVAL 7 DAY)`	`d + INTERVAL '7 day'`
Null replace	`ISNULL / COALESCE`	`IFNULL / COALESCE`	`COALESCE`

Task	SQL Server	MySQL	PostgreSQL
Concatenate	`CONCAT, +`	`CONCAT`	`||, CONCAT`
Auto number	`IDENTITY(1,1)`	`AUTO_INCREMENT`	`GENERATED ALWAYS AS IDENTITY`
Regex	`REGEXP_LIKE` (2025)	`REGEXP_LIKE`	`~`, `regexp_match`

Newer SQL Server features (2022 and 2025: GREATEST/LEAST, DATETRUNC, GENERATE_SERIES, WINDOW clause, regex, JSON type, VECTOR type) are listed with examples in **Building Blocks · What's new**.

AI and SQL

- Copilot in SSMS and in Azure SQL / Fabric can write and explain queries from English.
- ChatGPT, Claude or Gemini: give the schema (table and column names, types, keys) and a precise question; ask for one query plus a line-by-line explanation; run it on a small sample and check against a manual count.
- Text-to-SQL tools in BI products (Metabase, Power BI Q&A, Tableau Agent) do the same inside dashboards.

Interview Q&A

Q WHERE vs HAVING?

WHERE filters rows before grouping (no aggregates); HAVING filters groups after GROUP BY (can use aggregates).

Q INNER vs LEFT JOIN with an example.

INNER returns only customers who have orders; LEFT returns all customers, with NULL order columns for those without orders.

Q Second highest salary?

```sql
SELECT MAX(salary) FROM Emp WHERE salary < (SELECT MAX(salary) FROM Emp);
```

Q RANK vs DENSE_RANK vs ROW_NUMBER?

For 100, 90, 90, 80 → ROW_NUMBER 1,2,3,4; RANK 1,2,2,4; DENSE_RANK 1,2,2,3.

Q UNION vs UNION ALL?

UNION removes duplicates (sort cost); UNION ALL keeps all rows and is faster.

Q What is a CTE and when is it better than a subquery?

A named temporary result defined with WITH for one statement. It makes multi-step logic readable, can be referenced more than once, and can be recursive.

Q How do you find duplicate rows?

```sql
SELECT email, COUNT(*) FROM Customer GROUP BY email HAVING COUNT(*) > 1;
```

Q What is an index and its trade-off?

A sorted structure that speeds reads on the indexed columns; it costs storage and slows inserts/updates because the index must be maintained.

Q Why does NOT IN return no rows sometimes?

If the subquery returns a NULL, `x NOT IN (…, NULL)` is never TRUE. Use NOT EXISTS.

Q Primary key vs unique constraint?

One primary key per table, never NULL, usually the clustered index; several unique constraints allowed, and they allow one NULL in SQL Server.

Resources

- ▶ YouTube: Simplilearn SQL full course · ▶ YouTube: Simplilearn SQL interview questions and answers
- ▶ YouTube: Alex The Analyst SQL tutorial · ▶ YouTube: techTFQ SQL window functions · ▶ YouTube: techTFQ SQL query practice
- ▶ YouTube: Kudvenkat SQL Server tutorial · ▶ YouTube: Brent Ozar how to think like the SQL Server engine
- 📘 Microsoft Learn: T-SQL reference · 📘 What's new in SQL Server 2025
- Practice: 📘 LeetCode SQL 50 · 📘 HackerRank SQL · 📘 SQLZoo · 📘 DataLemur SQL interview questions

LEARNING SESSION S3

📈 Power BI: Basic to Advanced

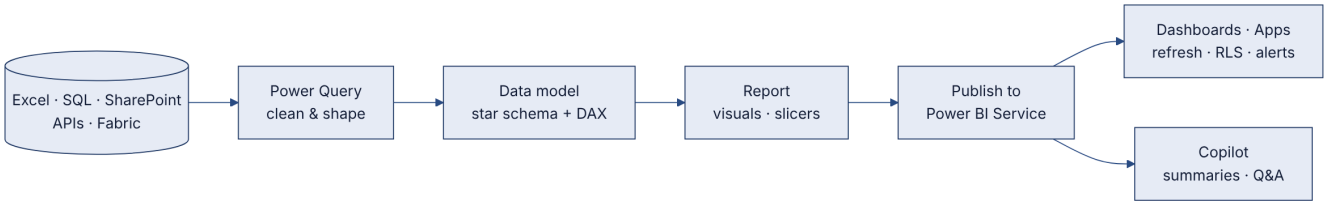
From your first report to DAX, data modelling, Copilot, Fabric and deployment, updated for 2026

Learning session · non-tech friendly · aligned to the PL-300 Power BI Data Analyst certification

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s3**

Roadmap

Power BI turns spreadsheets and databases into interactive dashboards that refresh themselves. This session builds the skill in seven steps; the course notes in **Power BI (course module)** stay the quick reference.



The Power BI flow: get data → Power Query → model + DAX → report → publish → share, refresh, Copilot.

Step	Skill	Milestone
1	Install, interface, first report	A 1-page sales report in 30 minutes
2	Power Query	Clean and combine monthly files automatically
3	Data modelling	Star schema with a date table
4	DAX: basic → advanced	Time intelligence, ranking, dynamic measures
5	Visual design & interactivity	Drill-through, bookmarks, field parameters
6	Service, Fabric & security	Published app with scheduled refresh and RLS
7	Copilot, performance, DevOps	Fast, AI-assisted, version-controlled reports

1 · First report

1. Install **Power BI Desktop** free from the Microsoft Store (Windows). Mac users can use the Power BI service in a browser or a Windows VM.
2. **Get data → Excel workbook** → tick the Sales table → **Transform Data** (always check types first) → Close & Apply.
3. Report view: click a **Card** visual, drag Amount → shows Total Sales.
4. Add a **Clustered column chart**: Axis = City, Values = Amount; sort descending.
5. Add a **Line chart**: X-axis = Order Date (Month), Values = Amount.
6. Add a **Slicer** on Store Type; click values and watch all visuals filter together (cross-filtering).

7. Format: theme (View → Themes), titles that state the insight, consistent colours.

8. Save as .pbix; Publish to a workspace to share.

Pane / view	Purpose
Report view	Canvas and pages
Table view	Look at rows, set formats, add calculated columns
Model view	Relationships and properties
DAX query view / TMDL view	Write and test DAX queries; edit the model as code
Visualizations / Data / Filters panes	Choose visuals, fields, filter levels

2 · Power Query

Home → Transform data. Every action becomes an **Applied Step**; refreshing replays them on new data.

Need	Do this
Combine 12 monthly files	Get Data → Folder → Combine & Transform
Fix types	Icon left of each header; set Date, Decimal, Text (locale for dd-mm-yyyy: Using Locale → English (India))
Wide months to rows	Select ID columns → Unpivot Other Columns
Lookup another table	Merge Queries (Left Outer) → expand the needed column
Stack tables	Append Queries
Fill blanks from above	Transform → Fill → Down
Categories without code	Add Column → Conditional Column
Data quality check	View → Column quality, distribution, profile (based on entire data)
Parameterise a source path	Manage Parameters → use in Source step

> **Tip** **Query folding**: with databases, Power Query translates steps into SQL that runs on the server. Keep foldable steps (filters, column selection) early; right-click a step → View Native Query to check.

3 · Data modelling





Star schema: one fact table in the middle, dimension tables around it, one-to-many relationships flowing from dimension to fact.

- Facts = events with numbers (sales lines, transactions). Dimensions = descriptions (who, what, where, when).
- Relationships: one-to-many, single direction by default; avoid bi-directional unless needed.
- **Date table**: one row per day covering the full range; mark as date table; use it for all time slicing.
- Hide technical columns (keys) from report view; set sort-by columns (Month name sorted by month number); set default summarisation; organise measures in a measure table.
- Role-playing dates (order vs ship date): one active relationship, others activated with `USERELATIONSHIP`.

4 · DAX

Level 1: basic measures

```
Total Sales   = SUM ( Sales[Amount] )
Orders        = DISTINCTCOUNT ( Sales[OrderID] )
AOV           = DIVIDE ( [Total Sales], [Orders] )
Total Cost    = SUMX ( Sales, Sales[Qty] * RELATED ( Product[UnitCost] ) )
Margin %      = DIVIDE ( [Total Sales] - [Total Cost], [Total Sales] )
```

Level 2: CALCULATE and filter context

Every visual cell evaluates a measure under filters from rows, columns, slicers and filters. **CALCULATE** changes those filters.

```
Sales Pune        = CALCULATE ( [Total Sales], Customer[City] = "Pune" )
Sales All Cities  = CALCULATE ( [Total Sales], ALL ( Customer[City] ) )
% of All Cities   = DIVIDE ( [Total Sales], [Sales All Cities] )
Sales Ignoring Slicers Except Year = CALCULATE ( [Total Sales], ALLEXCEPT ( 'Date', 'Date'[Year] )
)
Big Orders        = CALCULATE ( [Orders], Sales[Amount] > 10000 )
```

Level 3: time intelligence

```
Sales YTD      = TOTALYTD ( [Total Sales], 'Date'[Date], "31/3" )          -- Indian FY ends 31
March
Sales LY       = CALCULATE ( [Total Sales], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
YoY %          = DIVIDE ( [Total Sales] - [Sales LY], [Sales LY] )
Sales Last 3M  = CALCULATE ( [Total Sales], DATESINPERIOD ( 'Date'[Date], MAX ( 'Date'[Date] ), -3,
MONTH ) )
MoM %          = VAR prev = CALCULATE ( [Total Sales], DATEADD ( 'Date'[Date], -1, MONTH ) )
                 RETURN DIVIDE ( [Total Sales] - prev, prev )
```

Level 4: advanced patterns

```
Product Rank   = RANKX ( ALLSELECTED ( Product[Name] ), [Total Sales], , DESC, DENSE )
Top 5 Sales    = CALCULATE ( [Total Sales], TOPN ( 5, ALL ( Product[Name] ), [Total Sales] ) )
New Customers  = COUNTROWS ( FILTER ( VALUES ( Sales[CustomerID] ),
                     CALCULATE ( MIN ( Sales[Date] ) ) >= MIN ( 'Date'[Date] ) ) )
Pareto Cum %   = VAR cur = [Total Sales]
                 VAR tbl = ADDCOLUMNS ( ALLSELECTED ( Product[Name] ), "s", [Total Sales] )
                 RETURN DIVIDE ( SUMX ( FILTER ( tbl, [s] >= cur ), [s] ), SUMX ( tbl, [s] ) )
Selected Metric = SWITCH ( SELECTEDVALUE ( Metric[Name] ),
                     "Sales", [Total Sales], "Margin %", [Margin %], [Orders] )
```

Visual calculations (generally available since May 2026)

Select a visual → **New visual calculation**. Write simpler DAX that runs on the visual's own grid, without touching the model:

```
Running Total = RUNNINGSUM ( [Sales] )
Moving Avg 3  = MOVINGAVERAGE ( [Sales], 3 )
% of Parent   = DIVIDE ( [Sales], COLLAPSE ( [Sales], ROWS ) )
vs Previous   = [Sales] - PREVIOUS ( [Sales] )
```

> **Tip** Rule of thumb: model measures for numbers reused across reports; visual calculations for one-off running totals, ranks and comparisons in a single visual.

5 · Visuals & interactivity

Cards / KPI / Gauge	Headline numbers vs target
Bar, column, line, combo, area, ribbon, waterfall, funnel, scatter, map	Standard analysis visuals
Matrix	Pivot-style table with drill-down, conditional formatting and custom totals
Decomposition tree, Key influencers, Smart narrative	AI visuals that explain a metric
Small multiples	One chart repeated per category
Gantt (preview, Oct 2026)	Project timelines natively

Interactivity

- **Edit interactions**: filter vs highlight vs none per visual pair.
- **Drill-through** pages and **tooltip pages**.
- **Bookmarks** + Selection pane + Buttons: toggles, guided stories, reset filters.
- **Field parameters**: let users switch the measure or the axis.
- **Slicers**: list, dropdown, button slicer, date range; sync across pages.
- **Personal bookmarks** and personalise visuals for viewers in the service.

Design checklist

✔ Top-left = most important KPI; Z-pattern layout
✔ Max 6–8 visuals per page; one message per page
✔ Colour has meaning (red = bad), consistent theme JSON
✔ Accessible: contrast, alt text, tab order
✔ Titles state insights; units in labels (₹ lakh)

6 · Service, Fabric & security

- **Workspaces** (team folders) with roles Admin / Member / Contributor / Viewer; **Apps** package reports for wide audiences.
- **Semantic model** (dataset) settings: credentials, scheduled refresh (up to 8/day on Pro, 48 on Premium/Fabric capacity), **incremental refresh** for large facts, on-premises data gateway.
- **Row-level security**: roles with DAX filters such as `[Region] = "North"` or dynamic `[Email] = USERPRINCIPALNAME()`; assign users in the service. Object-level security hides tables/columns.
- **Dashboards** with pinned tiles and data alerts; **subscriptions** by email; export to PDF/PowerPoint; Analyze in Excel.
- **Microsoft Fabric**: the wider platform (OneLake, Lakehouse, Warehouse, Data Factory pipelines, notebooks, Direct Lake mode). Microsoft is moving enterprise capacity from Premium P-SKUs to Fabric F-SKUs.
- **Licences**: Free (personal use), Pro (share with Pro users), Premium Per User, Fabric capacity (share with free viewers on F64 and above).

7 · Copilot, performance & DevOps

Copilot in Power BI

Report creation	"Create a page showing sales trend, top products and regional split"
Summaries	Copilot Summarize at report and visual level in the service (May 2026)
Q&A over the model	Ask questions in English; answers respect RLS
DAX help	Write and explain measures in the DAX query view
Prep data for AI	Add synonyms, descriptions and verified answers so Copilot answers correctly

Copilot needs a paid Fabric capacity (F2 or above) or Premium, and an admin must enable it.

Performance

- View → **Performance Analyzer**: time per visual (DAX query, visual display).
- Smaller model = faster: remove unused columns, reduce cardinality (split datetime into date and time), prefer measures over calculated columns, integers for keys.
- External tools: DAX Studio (query timing, VertiPaq Analyzer), Tabular Editor (bulk editing, best-practice analyser).

Version control and deployment

- Save as **Power BI Project (.pbip)**: report (PBIR format) and model (**TMDL**) as text files that work with Git.
- Deployment pipelines: Development → Test → Production workspaces.
- Fabric Git integration with Azure DevOps or GitHub.

Interview Q&A

Q Measure vs calculated column?

A column is computed per row at refresh and stored (use for slicing); a measure is computed at query time in the current filter context (use for aggregation).

Q Explain filter context vs row context.

Filter context = filters applied to the model by the visual, slicers and CALCULATE. Row context = the current row inside a calculated column or an iterator (SUMX). CALCULATE converts row context into filter context (context transition).

Q What is a star schema and why use it?

A central fact table linked one-to-many to dimension tables. It keeps DAX simple and fast and avoids ambiguous filter paths.

Q Import vs DirectQuery vs Direct Lake?

Import: data cached in memory, fastest, needs refresh. DirectQuery: live queries to the source, current but slower. Direct Lake (Fabric): reads Delta tables in OneLake directly with import-like speed.

Q How do you implement dynamic RLS?

Have a user–region mapping table, relate it to the model, and use a role filter like `[Email] = USERPRINCIPALNAME()`.

Q What are visual calculations?

DAX calculations defined on a visual that operate on the visual's data grid (running sums, moving averages, previous value) without adding model measures.

Q How would you speed up a slow report?

Use Performance Analyzer to find slow visuals; optimise DAX (variables, avoid FILTER on whole tables), reduce model size, limit visuals per page, use aggregations or incremental refresh.

Resources

- ▶ YouTube: Simplilearn Power BI full course · ▶ YouTube: Simplilearn Power BI tutorial for beginners
- ▶ YouTube: Guy in a Cube Power BI · ▶ YouTube: SQLBI CALCULATE explained · ▶ YouTube: Power BI visual calculations tutorial
- ▶ YouTube: Pragmatic Works Power BI full course · ▶ YouTube: codebasics Power BI project
- 📘 Microsoft Learn: Power BI · 📘 PL-300 study guide
- 📘 Power BI monthly feature summaries (blog) · 📘 DAX Guide

LEARNING SESSION S4

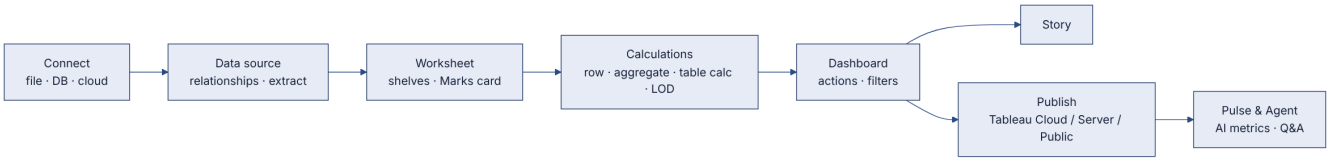
📉 Tableau: Basic to Advanced

Connect, visualise, calculate, build dashboards and use Tableau's AI (Pulse, Agent, Tableau Next), updated for 2026

Learning session · non-tech friendly · aligned to the Tableau Desktop Foundations and Data Analyst certifications

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s4**

Roadmap



From connection to published, AI-assisted dashboards.

Step	Skill	Milestone
1	Install, connect, first viz	Bar, line and map in 20 minutes
2	Marks, filters, sorting, formatting	Clean, sorted, labelled views
3	Calculated fields, parameters, sets, groups, bins	Dynamic Top N
4	Table calculations and LOD expressions	% of total, running totals, cohorts
5	Dashboards, actions, stories	An interactive executive dashboard
6	Publishing, performance, AI features	Live on Tableau Public / Cloud

Course reference: **Tableau (course module)**. Practice: **Laptops case**, **Sports Equipment case**.

1 · Start

1. Download **Tableau Public** (free, saves to the public web) or Tableau Desktop (trial/licensed). Tableau Cloud runs in the browser.
2. Connect → Microsoft Excel / Text file / SQL Server. Drag tables into the canvas; Tableau creates **relationships** (noodles) between them.
3. Go to Sheet 1. Left: Data pane with dimensions (blue) and measures (green). Centre: Columns/Rows shelves, Marks card, view.
4. Double-click Category, then Sales → a bar chart. Drag Order Date to Columns → line chart. Double-click State → map.
5. **Show Me** suggests chart types for the fields you selected.

Live vs Extract	Live queries the source; Extract (.hyper) is a fast snapshot refreshed on a schedule
Discrete vs continuous	Blue = headers; green = axis

Aggregation	Measures default to SUM; right-click → Measure → Average/Count/etc.

2 · Marks & filters

- **Marks card**: Color, Size, Label, Detail, Tooltip, Shape, Path; mark type dropdown.
- Sort by toolbar or field; nested sorts; sort by a different measure.
- **Filters**: dimension (pick values, wildcard, condition, Top N), measure (range), date (relative "last 3 months"). Show filter cards; apply to multiple sheets.
- Formatting: Format → Workbook; remove gridlines; reference lines; consistent fonts; number formats (₹, K, M).
- Hierarchies and drill (+/-); groups (combine members); bins (histograms).

> **Tip** Remember the order of operations: extract → data source → context → FIXED LOD → dimension filters → INCLUDE/EXCLUDE → measure filters → table calcs. Details in **Tableau module**.

3 · Calculations

```
// Row-level
Profit           = [Sales] - [Cost]
Profit Ratio     = SUM([Profit]) / SUM([Sales])                    // aggregate calc
Order Size       = IF [Quantity] >= 10 THEN "Bulk" ELSE "Single" END
Days to Ship     = DATEDIFF('day', [Order Date], [Ship Date])
Financial Year   = "FY" + STR(YEAR(DATEADD('month', 9, [Order Date])))   // Apr-Mar FY
Clean City       = TRIM(PROPER([City]))
Is Weekend       = DATEPART('weekday', [Order Date]) IN (1, 7)
// Parameter-driven
Top N Filter     = RANK(SUM([Sales])) <= [Top N]                   // table-calc filter
Measure Switch   = CASE [Choose Measure] WHEN "Sales" THEN SUM([Sales]) WHEN "Profit" THEN
SUM([Profit]) END
```

Parameters	User inputs (number/date/string list); used in calcs, filters, reference lines. Parameter actions let a click set a parameter.
Sets	In/out membership (top 10 customers); set actions make them interactive
Groups / bins	Combine categories; bucket numbers
Combined fields	Concatenate dimensions (used in the Laptops case for 32 groups)

4 · Table calcs & LOD

Table calculations (on the results in the view)

Running total, difference, % difference, % of total, rank, percentile, moving average, YoY growth. Always check **Compute Using**.

Level of Detail expressions

```
Customer First Order  = {FIXED [Customer ID] : MIN([Order Date])}        // cohort start
Customer Lifetime $   = {FIXED [Customer ID] : SUM([Sales])}
Orders per Customer   = {FIXED [Customer ID] : COUNTD([Order ID])}        // use as dimension → histogram
Avg Order Value       = AVG({INCLUDE [Order ID] : SUM([Sales])})
Diff from Avg Month   = SUM([Sales]) - SUM({EXCLUDE [Month] : AVG([Sales])})
Share of Region       = SUM([Sales]) / SUM({FIXED [Region] : SUM([Sales])})
```

Real example: customer cohorts

Cohort Year = YEAR({FIXED [Customer ID] : MIN([Order Date])}). Put Cohort Year on Rows, YEAR(Order Date) on Columns, COUNTD(Customer ID) on Text → a retention table showing how many customers from each starting year keep buying.

5 · Dashboards

- Size: fixed (1200×800) for desktop; add a phone layout (Device Preview).
- Layout containers (horizontal/vertical) for alignment; padding; blank objects for spacing.
- **Actions**: Filter, Highlight, URL, Go to Sheet, Change Parameter, Change Set Values.
- KPI tiles (BANs: big-ass numbers), trend, breakdowns, detail table; filters in one place.
- **Stories**: sequence of story points with captions for presentations.
- **Dashboard extensions** and Viz in Tooltip (a chart inside a tooltip).

6 · Publish, performance & AI

Tableau Public	Free; everything is public: ideal for a portfolio
Tableau Cloud / Server	Secure sharing, schedules, subscriptions, data-driven alerts, row-level security via user filters/entitlements
Performance	Extracts, fewer marks, context filters, avoid many quick filters with "only relevant values", Performance Recorder

Tableau AI in 2026

Tableau Pulse	Metrics on governed data sources delivered to email, Slack or the app, with AI-written insights and natural-language Q&A ("Tableau Agent in Pulse"). Requires a Tableau+ licence for the AI features.
Tableau Agent	Helps authors build calculations and visuals from prompts; in dashboards (beta on Cloud) offers overviews, insights and Q&A that shows its reasoning
Tableau Next	Salesforce's separate agentic analytics platform built on Data Cloud and Tableau Semantics, with agents for data prep (Data Pro), questions (Concierge) and monitoring (Inspector)
Certification	Tableau Desktop Foundations replaced Desktop Specialist as the entry certificate; next is Tableau Certified Data Analyst

Interview Q&A

Q Tableau vs Power BI?

Both are leading BI tools. Tableau is strong in flexible visual exploration and design; Power BI is cheaper for Microsoft-365 organisations, with deep Excel/Fabric integration and DAX modelling. Choose by ecosystem and cost.

Q What is an LOD expression?

A calculation that sets its own level of aggregation independent of the view: FIXED, INCLUDE or EXCLUDE.

Q Context filter use case?

To make a filter apply before Top N or FIXED calculations, e.g. Top 10 products within the selected region.

Q Extract vs live?

Extract is faster and works offline but needs refreshing; live is always current but depends on source speed.

Q How do you show the top N dynamically?

Create a parameter and use it in a Top N filter or a RANK table-calc filter; show the parameter control.

Q Blending vs relationships?

Relationships combine tables in one data source at the right level per view; blending combines separate data sources on a linking field per sheet, aggregating the secondary source.

Resources

- ▶ YouTube: Simplilearn Tableau full course · ▶ YouTube: Simplilearn Tableau tutorial for beginners
- ▶ YouTube: Andy Kriebel Tableau tip · ▶ YouTube: Tableau LOD expressions explained · ▶ YouTube: Tableau Pulse demo
- ▶ YouTube: Alex The Analyst Tableau tutorial for beginners
- 📘 Tableau new features · 📘 Tableau free training videos
- 📘 Tableau Public · 📘 Workout Wednesday · 📘 Makeover Monday

LEARNING SESSION S5

🐍 Python: Basic to Advanced (for non-tech learners)

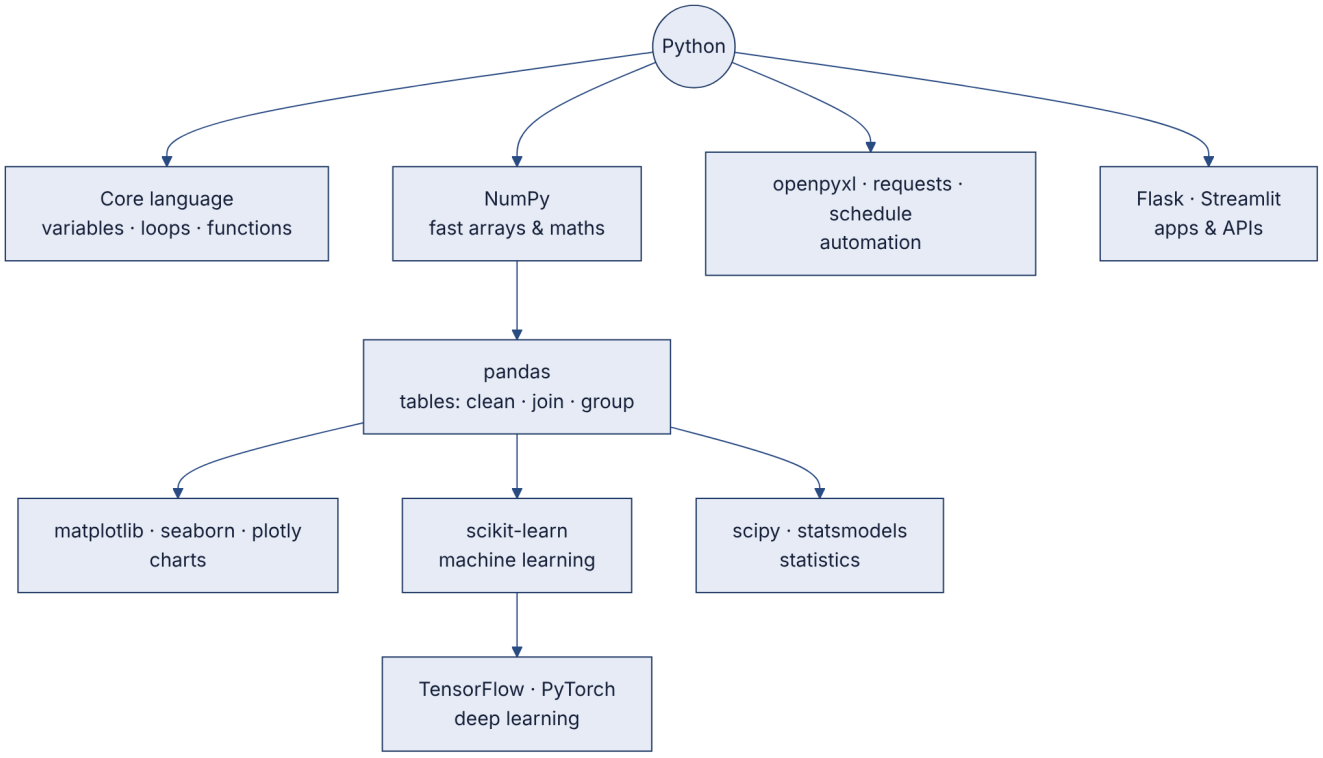
Learn Python step by step with everyday business examples: from your first line of code to pandas, charts, automation and ML

Learning session · no programming background needed · runs in Google Colab or Jupyter

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s5**

Start here

If you can write an Excel formula, you can learn Python. Python is a set of instructions written in near-English that the computer follows exactly. Analysts use it because it repeats work perfectly, handles millions of rows, and connects Excel, databases, the web and AI.



Core Python first, then the libraries that make it a data tool.

Set up in 5 minutes

- **Easiest**: open Google Colab with a Google account → New notebook. Nothing to install.
- **On your laptop**: install Anaconda (includes Python, Jupyter, pandas) → launch Jupyter Notebook; or install Python + VS Code.
- Run a cell with `Shift` + `Enter`.

Excel → Python translation

Excel	Python (pandas)
Open a file	`df = pd.read_excel("sales.xlsx")`
Filter	`df[df["City"] == "Pune"]`
Sort	`df.sort_values("Amount", ascending=False)`
New column =C2*D2	`df["Total"] = df["Qty"] * df["Price"]`
SUMIFS / pivot	`df.groupby("City")["Amount"].sum()` / `df.pivot_table(...)`
VLOOKUP	`df.merge(products, on="ProductID", how="left")`
Remove duplicates	`df.drop_duplicates()`
Chart	`df.plot.bar(x="City", y="Amount")`

30-day plan

Days 1–7	Levels 1–2 (basics, data structures), 30 minutes daily
Days 8–12	Level 3 (functions, files, errors)
Days 13–22	Levels 4–5 (pandas + charts) on your own Excel files
Days 23–27	Level 7 automation: automate one weekly report
Days 28–30	Mini project: clean → analyse → chart → one-page summary

Level 1 · Basics

```
print("Hello, Data!")              # show something
name = "Asha"                      # variable = a labelled box holding a value
orders = 12                        # int (whole number)
avg_value = 1310.50                # float (decimal)
is_prime = True                    # bool (True/False)
print(f"{name} placed {orders} orders worth ₹{orders * avg_value:,.0f}")   # f-string
# Asha placed 12 orders worth ₹15,726

type(avg_value)                    # float
int("42") + 1                      # 43  convert text to number
round(1310.567, 1)                 # 1310.6

city = input("Enter city: ")       # ask the user (always returns text)

# Decisions
amount = 7200
if amount > 5000:
    print("Free delivery")
elif amount > 2000:
    print("₹49 delivery")
else:
    print("₹99 delivery")
```

```
# Repetition
for month in ["Jan", "Feb", "Mar"]:
    print("Report for", month)
for i in range(1, 6):               # 1,2,3,4,5
    print(i * 100)
```

Tip Indentation (4 spaces) is part of Python's grammar: it shows which lines belong inside an if or for. A missing colon or wrong indent is the most common beginner error.

Level 2 · Data structures

```
cities = ["Pune", "Delhi", "Mumbai"]          # list: ordered, changeable
cities.append("Chennai"); cities[0]; cities[-1]; len(cities)
sales = {"Pune": 120000, "Delhi": 95000}      # dict: key → value (like a lookup table)
sales["Mumbai"] = 87000
for city, value in sales.items():
    print(city, value)
point = (28.46, 77.03)                        # tuple: fixed (latitude, longitude)
unique_ids = set([101, 102, 101])             # set: unique values → {101, 102}

# List comprehension: build a list in one line
amounts = [450, 7200, 3100, 980]
big = [a for a in amounts if a > 1000]         # [7200, 3100]
with_gst = [round(a * 1.18) for a in amounts]  # add 18% GST

# Strings
email = "  Asha.Verma@Example.com "
email.strip().lower()                          # 'asha.verma@example.com'
email.split("@")[1]                            # domain part
"Pune" in "Pune, Maharashtra"                  # True
```

Level 3 · Functions, files, errors

```
def delivery_fee(amount, is_prime=False):
    """Return the delivery fee for an order."""
    if is_prime or amount > 5000:
        return 0
    return 49 if amount > 2000 else 99

delivery_fee(1500)          # 99
delivery_fee(1500, True)    # 0

gst = lambda x: x * 0.18    # small one-line function
list(map(gst, [100, 200]))  # [18.0, 36.0]

# Files
with open("notes.txt", "w") as f:
    f.write("Report generated\n")
import csv, os, datetime
os.listdir(".")             # files in the folder

# Errors: handle them instead of crashing
```

```
try:
    rate = int("18%")
except ValueError:
    rate = 18
    print("Could not read the rate; using 18")
```

Modules: `import math`, `from datetime import date`, `import pandas as pd`. Install new ones with `pip install package-name` (in Colab: `!pip install ...`).

Level 4 · pandas

A complete mini-analysis

```
import pandas as pd
df = pd.read_csv("orders.csv", parse_dates=["order_date"])   # or read_excel / read_sql

df.shape                       # (rows, columns)
df.head()                      # first 5 rows
df.info()                      # columns, types, missing values
df.describe()                  # quick statistics

# Clean
df = df.drop_duplicates()
df["city"] = df["city"].str.strip().str.title()
df["amount"] = pd.to_numeric(df["amount"], errors="coerce")
df["amount"] = df["amount"].fillna(df["amount"].median())

# New columns
df["month"] = df["order_date"].dt.to_period("M")
df["band"] = pd.cut(df["amount"], [0, 2000, 10000, float("inf")], labels=["Low", "Medium", "High"])

# Questions
df[df["city"] == "Pune"]["amount"].sum()                        # Pune revenue
df.groupby("city")["amount"].agg(["count", "sum", "mean"]).sort_values("sum", ascending=False)
df.pivot_table(index="city", columns="month", values="amount", aggfunc="sum", fill_value=0)
top5 = df.groupby("customer_id")["amount"].sum().nlargest(5)

# Combine with another table (VLOOKUP)
products = pd.read_excel("products.xlsx")
df = df.merge(products[["product_id", "category"]], on="product_id", how="left")

# Save
df.to_excel("clean_orders.xlsx", index=False)
```

Full pandas reference: **Python module → pandas**. 90 practice exercises solved: **Data Manipulation exercise**.

Level 5 · Charts

```
import matplotlib.pyplot as plt
import seaborn as sns

monthly = df.groupby("month")["amount"].sum()
```

```
monthly.plot(kind="line", marker="o", title="Monthly revenue")
plt.ylabel("₹"); plt.show()

df.groupby("city")["amount"].sum().sort_values().plot(kind="barh", title="Revenue by city")
sns.histplot(df["amount"], bins=30)                          # distribution
sns.boxplot(data=df, x="city", y="amount")                   # spread and outliers
sns.heatmap(df.corr(numeric_only=True), annot=True)         # correlations

import plotly.express as px                                  # interactive charts
px.bar(df, x="city", y="amount", color="band").show()
```

Level 6 · Stats & ML

```
from scipy import stats
a = df[df.city == "Pune"]["amount"]; b = df[df.city == "Delhi"]["amount"]
stats.ttest_ind(a, b, equal_var=False)           # do Pune and Delhi spend differently?

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
X = df[["amount", "orders_last_90d", "days_since_last"]]
y = df["churned"]                                  # 1 = left, 0 = stayed
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=1)
model = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)
print(classification_report(y_te, model.predict(X_te)))
```

Explained without jargon in Machine Learning for non-tech.

Level 7 · Automation

```
# Combine every monthly Excel file in a folder into one
import glob
files = glob.glob("reports/sales_*.xlsx")
all_sales = pd.concat([pd.read_excel(f).assign(source=f) for f in files], ignore_index=True)

# Write a formatted Excel report with several sheets
with pd.ExcelWriter("weekly_report.xlsx") as xw:
    all_sales.groupby("city")["amount"].sum().to_excel(xw, sheet_name="By city")
    all_sales.to_excel(xw, sheet_name="Data", index=False)

# Get data from a web API
import requests
rates = requests.get("https://api.frankfurter.app/latest?from=USD&to=INR").json()

# Read from a database
import sqlalchemy as sa
engine = sa.create_engine("mssql+pyodbc://@localhost/RetailDB?
driver=ODBC+Driver+18+for+SQL+Server&trusted_connection=yes&TrustServerCertificate=yes")
orders = pd.read_sql("SELECT * FROM Orders WHERE order_date >= '2026-10-01'", engine)
```

Schedule the script with Windows Task Scheduler, cron, or a cloud scheduler; see Automation on Data.

Level 8 · Advanced

OOP (classes)	Bundle data + behaviour: `class Report: def __init__(self, df): ...`
Comprehensions & generators	`{k: v for k, v in ...}` ; `yield` to stream large files
Decorators	Wrap functions (timing, caching: `@functools.lru_cache`)
Vectorisation	Use pandas/NumPy column operations instead of loops (10–100× faster)
Virtual environments	`python -m venv .venv` or conda; pin versions in requirements.txt
Git & GitHub	Version control and portfolio (see ML Ops module)
Testing	`pytest` for functions that compute money
Type hints & linting	`def fee(amount: float) -> int:` ; ruff/black for clean code
Big data	Polars, DuckDB, PySpark when pandas runs out of memory
Apps	Streamlit dashboards and Flask/FastAPI APIs

Coding with AI assistants

- Ask for small pieces with context: "pandas DataFrame df has columns city, amount, order_date. Write code for monthly revenue by city as a pivot table."
- Ask it to explain errors: paste the code and the full error message.
- Use Colab's AI, GitHub Copilot, Cursor or Claude Code inside the editor; review every line before trusting it.

Interview Q&A

Q List vs tuple vs dict vs set?

List: ordered, changeable. Tuple: ordered, fixed. Dict: key-value lookups. Set: unique values, fast membership tests.

Q How do you handle missing values in pandas?

Find them with `isna().sum()` ; drop with `dropna` or fill with `fillna` (median for skewed numbers, mode for categories), or flag them.

Q merge vs concat?

merge joins on key columns (like VLOOKUP/SQL JOIN); concat stacks DataFrames on top of or beside each other.

Q What does groupby do?

Split rows into groups, apply an aggregation to each, combine the results: like SUMIFS or a pivot table.

Q Why avoid loops over DataFrame rows?

They are slow; vectorised column operations run in optimised C code.

Q What is a virtual environment?

An isolated set of packages for one project, so versions don't clash between projects.

Resources

- ▶ YouTube: Simplilearn Python full course · ▶ YouTube: Simplilearn Python for data science · ▶ YouTube: Simplilearn pandas tutorial
- ▶ YouTube: Corey Schafer Python beginner tutorial · ▶ YouTube: Keith Galli pandas tutorial · ▶ YouTube: freeCodeCamp Python for everybody
- ▶ YouTube: Automate the Boring Stuff with Python
- 📘 Google Colab · 📘 Python tutorial · 📘 pandas: getting started
- 📘 Kaggle Learn: Python and pandas (free) · 📘 Automate the Boring Stuff (free online book)

LEARNING SESSION S6

🤖 Machine Learning: Basic to Advanced (for non-tech learners)

What ML is, every major algorithm in plain English, how models are judged, and how they reach production

Learning session · no maths background needed · links to the illustrated ML Class 1–7 notes for depth

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s6**

What ML is

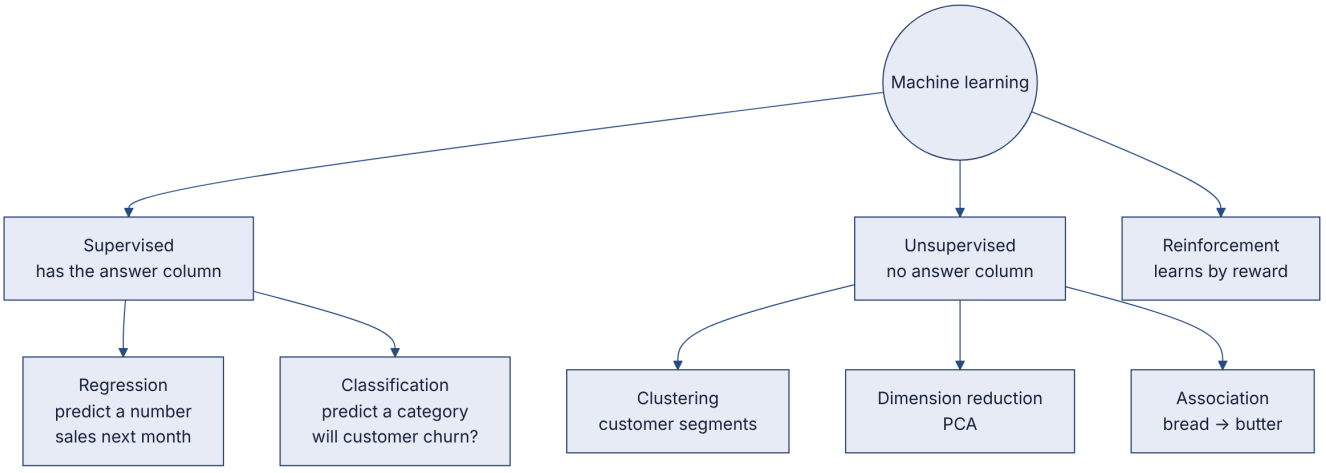
Machine learning is teaching a computer by **examples** instead of rules. Show it thousands of past loan applications labelled "repaid" or "defaulted", and it learns the pattern well enough to judge a new application.

Traditional rule

IF income < 3 lakh AND existing loans > 2 THEN reject. A person writes and maintains every rule.

Machine learning

Feed 50,000 past applications with outcomes. The algorithm finds the combinations of income, debt, age and history that predict default, and outputs a probability.



The three families of machine learning and their main tasks.

Everyday examples

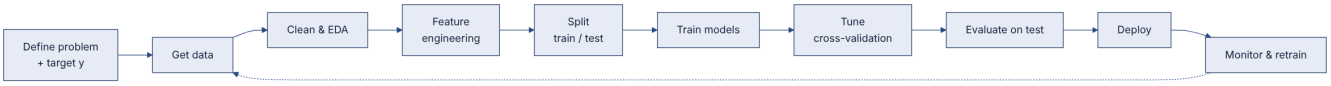
Where you see it	Task	Type
Gmail spam folder	Spam or not	Classification
Zomato delivery time estimate	Minutes to deliver	Regression
Netflix / Spotify recommendations	What you'll like next	Recommendation

Where you see it	Task	Type
Bank fraud SMS	Unusual transaction	Anomaly detection
Customer segments in marketing	Group similar customers	Clustering
Google Maps traffic, UPI demand planning	Future values	Forecasting
Review sentiment, ChatGPT	Understand/generate text	NLP / deep learning

Key words

Features (X)	Inputs: income, age, number of orders
Target / label (y)	What to predict: default yes/no, next month's sales
Training	The algorithm adjusts itself to fit past examples
Model	The learned pattern, saved and reused
Prediction / inference	Applying the model to new data
Train / test split	Learn on 70–80%, check on the unseen 20–30%

The ML workflow



Every ML project follows this loop. Most time goes into the first four boxes.

Step	Plain English	Example: predicting churn for a telecom
Define	What decision will the prediction support?	Who to call with a retention offer this month
Target	Exact definition of the outcome	Churn = ports out within 60 days
Data	Gather history with known outcomes	12 months of usage, bills, complaints, plan
Clean & explore	Fix gaps, errors, outliers; look at patterns	Missing tenure, duplicate accounts
Features	Turn raw data into useful signals	Drop in data usage, complaints last 30 days, days since recharge
Train & tune	Try models; tune settings with cross-validation	Logistic regression vs random forest vs XGBoost
Evaluate	Test on unseen data with business metrics	Catches 75% of churners in the top 20% risk list
Deploy & monitor	Score customers regularly; watch accuracy	Weekly scores to the CRM; retrain quarterly

Deep dive with real data and charts: **ML Class 3**, **Class 4**, **Class 6**.

Algorithms explained

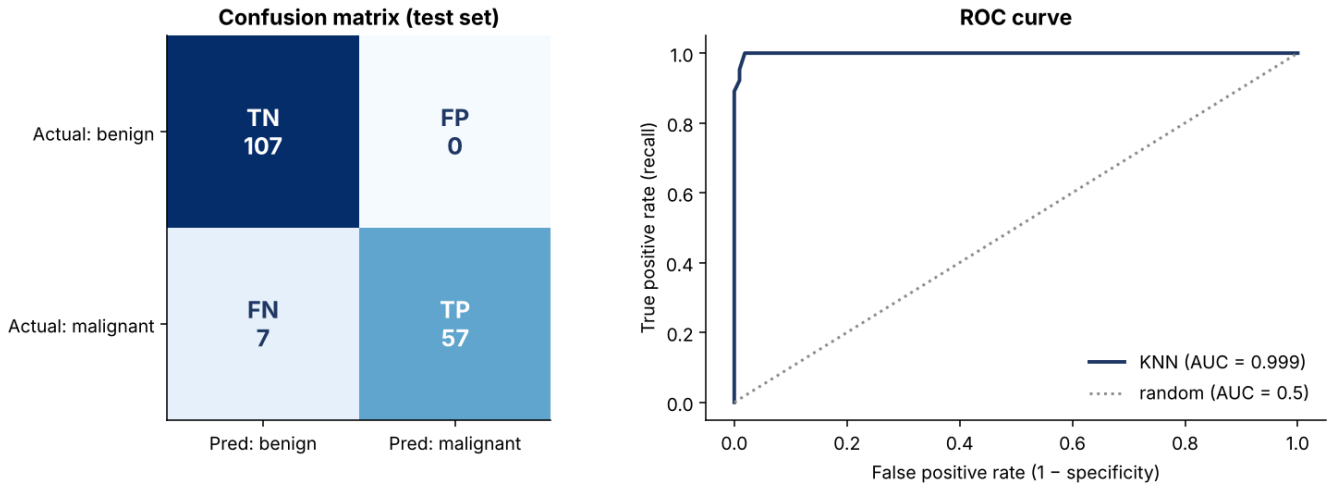
Algorithm	Plain-English idea	Good for	Watch out
Linear regression	Draws the best straight line through the points	Sales vs ad spend; price estimates; explaining drivers	Only straight-line relationships unless you add features
Logistic regression	Linear score squeezed into a 0–1 probability	Credit risk, churn; explainable (regulators like it)	Misses complex interactions
Decision tree	A flowchart of yes/no questions learnt from data	Easy-to-explain rules	One deep tree overfits
Random forest	Hundreds of trees on random samples vote	Strong all-rounder for tables	Less interpretable, bigger
Gradient boosting (XGBoost, LightGBM, CatBoost)	Trees built one after another, each fixing the previous errors	Best accuracy on most business tabular data	Needs tuning; can overfit
K-nearest neighbours	"You are like your closest neighbours"	Simple baselines, recommendations	Needs scaling; slow on big data
Naive Bayes	Bayes' rule assuming features independent	Spam and text classification	Independence assumption is rough
Support vector machine	Widest possible margin between classes	Smaller, high-dimensional data	Slow on large data; hard to explain
K-means clustering	Puts customers into K groups around centres	Segmentation	Choose K (elbow/silhouette); scale features
Hierarchical clustering	Builds a tree of nested groups (dendrogram)	Small data, exploring structure	Slow for big data
PCA	Compresses many correlated columns into a few	Visualising, removing redundancy	Components are hard to name
Association rules (Apriori)	"Bread → butter" with support, confidence, lift	Market basket, cross-sell	Many trivial rules
Time-series (ARIMA, ETS, Prophet)	Uses trend + seasonality to forecast	Demand, cash flow, call volumes	Breaks when patterns shift
Neural networks / deep learning	Layers of simple units learning complex patterns	Images, speech, text, LLMs	Needs lots of data and compute

Is the model good?

Regression metrics

MAE	Average size of error: "off by ₹1,200 on average"
RMSE	Like MAE but punishes big misses more
MAPE	Average % error: "forecast is 8% off"
R^2	Share of variation explained (0 to 1)

Classification metrics



Confusion matrix and ROC curve (see ML Class 7 for the full worked example).

Accuracy	Share of correct predictions. Misleading if one class is rare.
Precision	Of those flagged, how many were right? (Avoid wasting offers/false alarms.)
Recall	Of the real positives, how many did we catch? (Don't miss fraud or disease.)
F1	Balance of precision and recall
ROC-AUC	How well the model ranks positives above negatives (0.5 random, 1.0 perfect)
Lift / gains	"Top 10% of scores contain 40% of churners" – the metric business teams love

Overfitting in one sentence

A model that memorises the training data (like a student memorising last year's paper) scores brilliantly in practice and fails the real exam. We prevent it with test sets, cross-validation, simpler models and more data. Picture: ML Class 6.

Advanced topics

Feature engineering	Ratios, recency, trends, date parts, text length; often matters more than the algorithm
Imbalanced data	Class weights, SMOTE oversampling, choosing the right threshold
Hyper-parameter tuning	GridSearchCV, RandomizedSearchCV, Bayesian search (Optuna)
Ensembles	Bagging, boosting, stacking several models
Explainability	Feature importance, SHAP values ("income lowered this applicant's risk by 8 points"), partial dependence
Text mining / NLP	Tokenise → clean (stop words, lemmatise) → bag-of-words / TF-IDF / embeddings → classify sentiment or topics
Recommenders	Collaborative filtering, matrix factorisation, embeddings
Deep learning	CNNs for images, RNN/LSTM and Transformers for sequences and text
AutoML / no-code	Azure AutoML, Google Vertex AI, DataRobot, H2O, PyCaret; Copilot or ChatGPT data analysis for quick models
MLOps	Versioning, deployment, monitoring and retraining (see **ML Ops module**)
Responsible AI	Fairness across groups, privacy (DPDP Act in India), transparency, human review of high-impact decisions

Use cases by industry

Industry	Use case	Typical model	Business metric
Banking	Credit scoring, fraud, collections priority	Logistic regression, gradient boosting	Default rate, fraud loss, recovery %
Retail / e-commerce	Demand forecasting, recommendations, churn, pricing	Time series, collaborative filtering, boosting	Stock-outs, conversion, AOV
Telecom	Churn prediction, network fault prediction	Boosting, survival models	Churn rate, retention ROI
Healthcare	Readmission risk, image diagnosis support	Logistic regression, CNNs	Readmissions, sensitivity
Operations / call centres	Call-volume forecasting, staffing, intent routing	ETS/ARIMA, text classification	Service level, AHT, cost per contact
HR	Attrition risk, resume screening (with bias checks)	Logistic regression, NLP	Attrition, time-to-hire
Marketing	Segmentation, propensity to buy, uplift modelling	K-means, boosting	Campaign ROI, CAC

Interview Q&A

Q Explain machine learning to a 10-year-old.

It's like learning to recognise dogs: after seeing many pictures labelled "dog" and "not dog", you can tell for a new picture. The computer does the same with data.

Q Supervised vs unsupervised?

Supervised learns from examples with known answers (predict churn). Unsupervised finds structure without answers (group customers).

Q What is overfitting and how do you prevent it?

Memorising noise in training data so the model fails on new data. Prevent with train/test splits, cross-validation, simpler models, regularisation and more data.

Q Precision vs recall with a business example?

Fraud: recall = share of frauds caught; precision = share of alerts that were real fraud. Missing fraud is costly, so prioritise recall while keeping alerts manageable.

Q Why is random forest better than one decision tree?

Averaging many trees trained on different samples reduces variance, so predictions are more stable and accurate.

Q How do you explain a model's prediction to a manager?

Show the top drivers (feature importance or SHAP), a few example customers, and the business result (e.g. calls saved, revenue protected).

Resources

- ▶ YouTube: Simplilearn machine learning full course · ▶ YouTube: Simplilearn machine learning tutorial for beginners
- ▶ YouTube: StatQuest machine learning playlist · ▶ YouTube: Andrew Ng machine learning specialization
- ▶ YouTube: codebasics machine learning tutorial python · ▶ YouTube: Krish Naik complete machine learning playlist
- 📘 Google Machine Learning Crash Course · 📘 Kaggle Learn: Intro to ML
- 📘 scikit-learn: choosing the right estimator

LEARNING SESSION S7

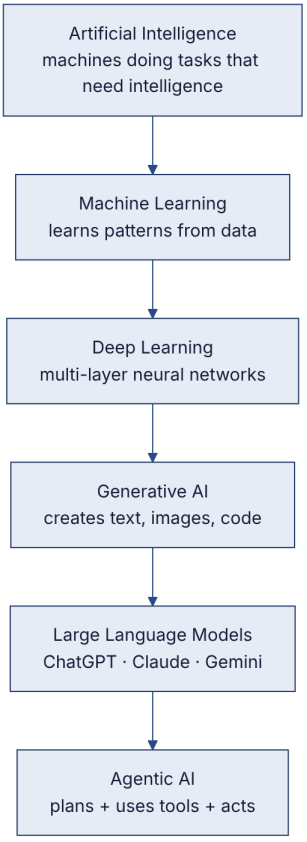
🧠 AI, Generative AI & Agentic AI (for non-tech learners)

How AI, LLMs, RAG and AI agents work, what they are good and bad at, and how to use them safely at work

Learning session · no technical background needed · complements the Generative AI course module

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s7**

The big picture



Each layer is a subset of the one above: agentic AI is the newest layer, built on LLMs.

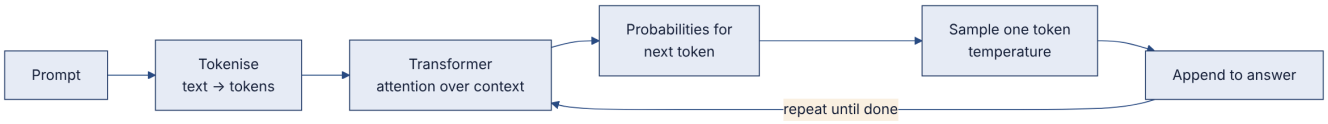
Term	Plain meaning	Example
Artificial Intelligence	Machines doing tasks that normally need human intelligence	Chess engines, voice assistants
Machine Learning	Learns patterns from data instead of hand-written rules	Credit scoring, fraud alerts
Deep Learning	ML with large neural networks	Face unlock, speech-to-text
Generative AI	Creates new content: text, images, audio, video, code	ChatGPT, Claude, Midjourney

Term	Plain meaning	Example
LLM	Large language model trained on huge text to predict the next word	GPT, Claude, Gemini, Llama, Grok
Agentic AI	AI that plans steps and uses tools to complete a goal, checking its own progress	"Pull last week's sales, build the chart, draft the email"

A short timeline

1950s–1990s	Rule-based AI, early neural networks, expert systems
2010s	Deep learning breakthroughs in images and speech
2017	The Transformer architecture ("Attention Is All You Need")
Nov 2022	ChatGPT launch brings generative AI to everyone
2023–2024	Multimodal models (text + images + voice), copilots in Office and coding tools, RAG in enterprises
2025–2026	Reasoning models, AI agents that browse, code and use apps; agent standards such as the Model Context Protocol (MCP); AI inside Excel, Power BI and Tableau

How LLMs work



An LLM writes one token at a time, each time predicting the most suitable next piece of text.

An analogy

Imagine someone who has read most of the internet and can finish any sentence plausibly. They are brilliant at language and patterns, but they don't "look up" facts unless given a source, and they can sound confident while wrong. That's an LLM.

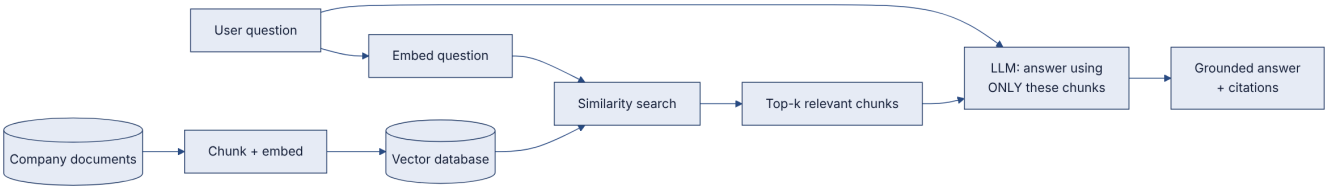
Token	A word piece; pricing and limits are in tokens (~¾ of a word each)
Context window	How much text the model can consider at once (a few pages to entire books)
Training	Pre-training on huge text → instruction tuning → human-feedback tuning for helpfulness and safety
Temperature	Creativity dial: low for facts and code, higher for ideas
Hallucination	A fluent but false answer; always verify numbers, names and citations

Knowledge cut-off	The model knows little after its training date unless it can search
Multimodal	Reads images, PDFs, spreadsheets; speaks and listens
Reasoning models	Spend extra "thinking" steps on hard problems (maths, code, planning)

Prompting techniques (role, format, chain of thought, ask-before-answer, reverse prompting) are covered in **Generative AI → Prompting**.

RAG & fine-tuning

Retrieval-Augmented Generation (RAG)



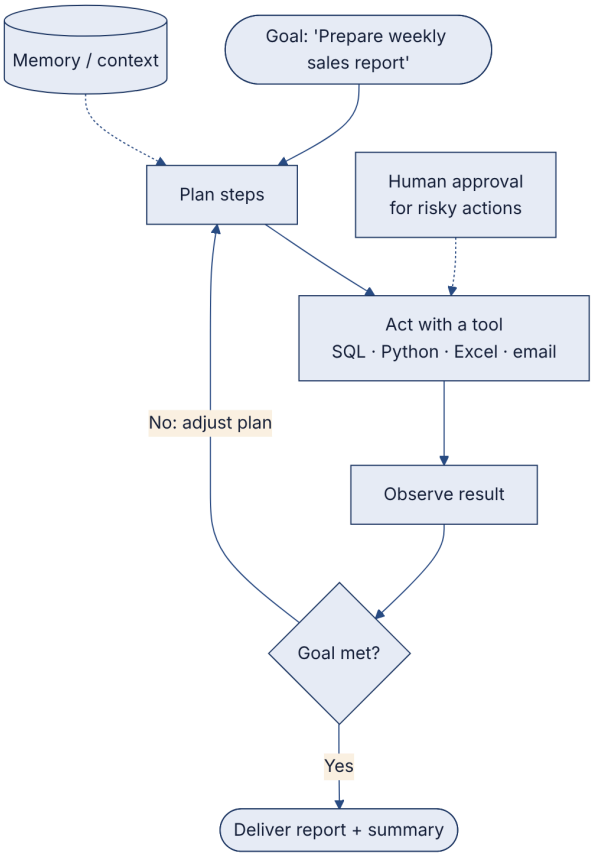
RAG lets an LLM answer from your own documents: it searches first, then writes the answer using only what it found.

- **Embeddings** turn text into number vectors so that similar meanings are close together (cosine similarity, see **Building Blocks ML · Class 7**).
- A **vector database** stores those vectors for fast similarity search (Pinecone, Chroma, pgvector, and now SQL Server 2025's VECTOR type).
- Example: an HR bot that answers "How many casual leaves do I get?" from the company's policy PDF, quoting the clause.

When to use what

Approach	Use when	Effort
Prompting	General tasks; give context in the prompt	Minutes
RAG	Answers must come from your documents and stay current	Days
Fine-tuning	A fixed style/format or specialised task at scale	Weeks, needs labelled examples
Training from scratch	Almost never for a business	Huge

Agentic AI



An agent loops: plan → act with a tool → observe → check the goal → adjust, with human approval for risky actions.

Component	What it means
Goal	A task, not a single question: "Prepare the weekly sales review"
Planning	Breaks the goal into steps
Tools	Things it can use: SQL database, Python, Excel, web search, email, calendar, CRM
Memory	Keeps context across steps or sessions
Guardrails	Permissions, approvals, logging, spending limits

Chatbot vs copilot vs agent

Chatbot	Answers questions in a conversation
Copilot	Helps inside an app while you stay in control (Copilot in Excel, GitHub Copilot)
Agent	Takes multi-step actions on its own toward a goal (Excel Agent Mode, Claude Code, ChatGPT agent, Tableau Next agents)

Agentic examples for data work

- Data-cleaning agent: profiles a new file, fixes types, flags anomalies, writes a summary.
- Report agent: runs SQL every Monday, updates Power BI, writes commentary, emails stakeholders.
- Research agent: gathers market data from the web with citations into a spreadsheet.
- Coding agent: builds and tests a Python pipeline from a description.

> **Tip** **Model Context Protocol (MCP)** is an open standard for connecting AI assistants to tools and data sources (databases, Google Drive, GitHub, Slack) in a consistent way, so one agent can use many apps.

Using AI at work

High-value uses for analysts

Write and explain	Excel formulas, SQL, DAX, Python; explain someone else's code
Clean and classify	Standardise names, categorise free-text feedback, extract fields from PDFs
Analyse	Upload a CSV and ask for summaries, charts and tests (verify the numbers)
Communicate	Draft executive summaries, slides, emails in the right tone
Learn	Tutor mode: explain a concept, quiz you, review your answer

Risks and safe practice

✔ Verify facts and numbers; ask for sources; recompute key figures

✔ Never paste confidential, customer or personal data into unapproved tools (India's DPDP Act 2023 governs personal data)

✔ Use company-approved enterprise versions with data-protection terms

✔ Watch for bias in outputs about people

✔ Keep a human in the loop for decisions affecting customers or money

✔ Label AI-generated content where required

Glossary

Prompt · **system prompt** · **token** · **context window** · **embedding** · **vector database** · **RAG** · **fine-tuning** · **hallucination** · **temperature** · **multimodal** · **agent** · **tool use / function calling** · **MCP** · **guardrails** · **evaluation (evals)** · **open-weight model** (downloadable, e.g. Llama, Mistral) vs **closed model** (API only).

Interview Q&A

Q AI vs ML vs generative AI vs agentic AI?

AI is the broad field; ML learns from data; generative AI creates new content; agentic AI plans and acts with tools to complete goals.

Q How does an LLM generate text?

It splits text into tokens and repeatedly predicts the next token using a transformer trained on vast text, sampling one token at a time.

Q What is RAG and why do companies use it?

Retrieve relevant internal documents, then have the LLM answer from them. It reduces hallucinations, keeps answers current and cites sources without retraining.

Q What risks would you flag before using GenAI on customer data?

Privacy and consent (DPDP), data leaving the company, hallucinated outputs, bias, security (prompt injection), and lack of audit trails.

Q Give an example of an AI agent in analytics.

An agent that, every Monday, queries sales, refreshes the dashboard, spots anomalies, drafts a commentary and asks a human to approve before emailing.

Resources

- ▶ YouTube: Simplilearn generative AI full course · ▶ YouTube: Simplilearn agentic AI tutorial · ▶ YouTube: Simplilearn what is AI
- ▶ YouTube: Andrej Karpathy intro to large language models · ▶ YouTube: 3Blue1Brown large language models explained briefly
- ▶ YouTube: IBM Technology what is RAG · ▶ YouTube: IBM Technology what are AI agents
- 📘 Google: Introduction to Generative AI (free course) · 📘 DeepLearning.AI short courses
- 📘 Anthropic: Building effective agents · 📘 Model Context Protocol
- 📘 Microsoft: AI fundamentals learning path

LEARNING SESSION S8

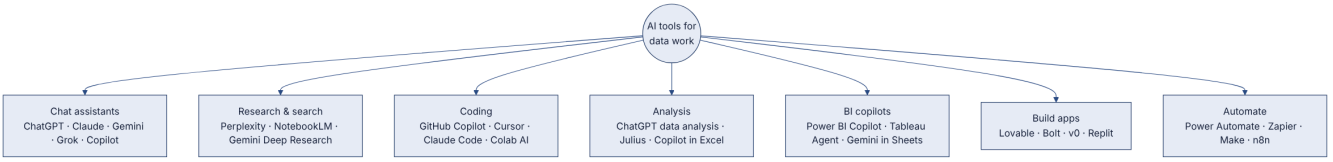
🧰 Tools for Data Science & AI

ChatGPT, Claude, Gemini, Copilot, Grok, Perplexity, Lovable, NotebookLM, Cursor and the core data stack: what each is for and how to use it

Learning session · tool landscape as of October 2026 · features and plans change often, so check each vendor's site

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s8**

Tool map



Seven groups of AI tools an analyst uses.

Which tool when

I need to…	First choice	Alternatives
Write/fix an Excel formula, SQL or Python	ChatGPT or Claude	Copilot in Excel, Gemini, GitHub Copilot
Analyse an uploaded CSV/Excel and make charts	ChatGPT data analysis	Claude (analysis + artifacts), Julius AI, Copilot in Excel, Gemini in Sheets
Research a topic with sources	Perplexity	ChatGPT / Gemini deep research, Claude research, Grok DeepSearch
Study from my own PDFs and notes	NotebookLM	Claude Projects, ChatGPT Projects
Real-time public sentiment / news on X	Grok	Perplexity
Build a small web app or dashboard without coding	Lovable	Bolt, v0, Replit, Claude artifacts
Write production code in an editor	GitHub Copilot or Cursor	Claude Code, Gemini in Colab
AI inside Office documents	Microsoft 365 Copilot	Gemini in Google Workspace
Automate workflows between apps	Power Automate (Microsoft shops)	Zapier, Make, n8n
Slides from an outline	Copilot in PowerPoint / Gamma	Canva Magic, Claude/ChatGPT drafts

AI assistants

Tool (maker)	Best at	Data-science uses	Notes
ChatGPT (OpenAI)	All-round assistant; runs Python on uploaded files; image generation; voice; custom GPTs; agent mode	Upload sales.xlsx → "clean it, show monthly trend, test if Pune differs from Delhi"; write SQL/DAX; explain errors	Free tier; Plus/Pro/Team/Enterprise paid plans. The course's GenAI classes used ChatGPT 3.5/4.
Claude (Anthropic)	Long documents, careful reasoning and writing, coding; Artifacts (interactive pages/dashboards); Projects with your files; Claude Code for agentic coding	Read a 100-page report and extract KPIs; build an interactive dashboard as an artifact; refactor a Python pipeline	Free, Pro, Max, Team, Enterprise plans. Also available inside Excel's Agent Mode and via API.
Gemini (Google)	Google Workspace integration (Sheets, Docs, Gmail, Drive), multimodal input, Deep Research	Gemini in Sheets: formulas, tables, analysis; Gemini in Colab: write and fix notebook code	Free tier; Google AI plans; Workspace business plans
Microsoft Copilot	Microsoft 365 apps: Excel (formulas, analysis, Agent Mode, `=COPILOT()`), PowerPoint, Word, Teams meeting recaps; Analyst and Researcher agents	"Analyse this table and add a pivot of sales by region"; turn Excel findings into a PowerPoint	Free Copilot app; Microsoft 365 Copilot licence for work data
Grok (xAI)	Real-time information from X (Twitter) and the web; DeepSearch; witty conversational style	Track brand sentiment or trending topics; quick current-events context	Available in X and the Grok app; free and paid tiers. Verify claims; social posts are noisy.
Perplexity	Answer engine with citations for every claim; Pro/Deep Research; Spaces for collections; Comet AI browser	Market sizing, competitor research, finding datasets and documentation with sources	Free and Pro; best for research, not for heavy file analysis
Others	Meta AI (in WhatsApp/Instagram), DeepSeek and Qwen (open-weight models), Mistral (European), Microsoft Copilot Studio (build your own agents)	Low-cost or self-hosted alternatives	Check data-privacy terms before uploading work data

One task, different assistants: prompt templates

```
ANALYSE (ChatGPT / Claude / Julius)
"You are a data analyst. The attached file has one row per order (columns: order_id, date,
city, category, amount). 1) Check data quality and list issues. 2) Monthly revenue trend
chart. 3) Top 5 cities and their share. 4) Three insights for a sales manager in plain English.
Show the code you ran."

RESEARCH (Perplexity)
"Size of India's quick-commerce market in 2025–26 with sources; list 5 credible reports and
their estimates in a table with publication dates."

LEARN (NotebookLM)
```

```
Upload: course PDFs and your notes → "Make a 20-question quiz on hypothesis testing and an
audio overview I can listen to on my commute."

BUILD (Lovable)
"Build a web app where a store manager uploads a CSV of daily sales and sees KPI cards, a
monthly trend chart and a top-products table. Clean, mobile-friendly design."
```

Build, code & research

Tool	What it is	Use it for
Lovable	AI app builder: describe an app in chat and it generates a full-stack web app (React front end, Supabase database/auth), with GitHub sync and one-click hosting	Prototypes, internal tools, a portfolio app (e.g. a churn-risk calculator) without writing code
Bolt, v0 (Vercel), Replit	Similar prompt-to-app builders; v0 focuses on UI; Replit has a full cloud IDE and agent	Quick front ends, dashboards, demos
NotebookLM (Google)	Research notebook grounded only in sources you upload; cites passages; generates study guides, mind maps and audio/video overviews	Revising course material, summarising long reports
GitHub Copilot	AI pair programmer inside VS Code, JetBrains, GitHub; chat and agent modes	Python, SQL, notebooks, unit tests
Cursor	AI-first code editor (VS Code-based) that edits across a whole project	Larger refactors, building pipelines
Claude Code	Agentic coding tool that reads a codebase, runs commands and makes changes	Automating data pipelines, building apps end to end
Julius AI	Chat-based data analyst for non-technical users: upload data, get charts and stats	Fast exploratory analysis
Hex, Deepnote	Collaborative AI notebooks mixing SQL, Python and charts	Team analytics
Gamma, Canva, Copilot in PowerPoint	AI presentations from prompts or documents	Turning analysis into slides

Core data stack

Category	Tools	Learn first
Spreadsheets	Excel (365), Google Sheets	Excel
Databases & SQL	SQL Server + SSMS, MySQL, PostgreSQL, SQLite, DuckDB; cloud: Snowflake, BigQuery, Databricks, Microsoft Fabric, Amazon Redshift	SQL Server or PostgreSQL
Programming	Python (pandas, NumPy, matplotlib/seaborn/plotly, scikit-learn, statsmodels), R	Python
Notebooks / IDEs	Jupyter, Google Colab, VS Code, PyCharm	Jupyter + VS Code

Category	Tools	Learn first
BI	Power BI, Tableau, Looker Studio (free), Metabase (open source), Qlik	Power BI (and Tableau)
Data engineering	Power Query, dbt, Airflow, Azure Data Factory, Fabric pipelines	Power Query
ML / MLOps	scikit-learn, XGBoost, TensorFlow/PyTorch, MLflow, Azure ML, Vertex AI, SageMaker	scikit-learn
Version control	Git, GitHub	GitHub for your portfolio
Automation	Power Automate, Zapier, Make, n8n, Windows Task Scheduler, cron	Power Automate
Collaboration	Jira, Confluence, Notion, Teams, Slack	Jira basics

Safe & smart use

✔ Use your company's approved AI tools for work data; personal accounts may use your inputs for training unless you opt out

✔ Mask personal data (names, phone, PAN, Aadhaar) before uploading

✔ Recalculate any number you will present; ask the tool to show its code

✔ Ask for sources (Perplexity, deep research) and open them

✔ Keep prompts that work in a personal library

✔ Compare two assistants on important questions; they make different mistakes

✔ Declare AI assistance where your organisation or course requires it

Resources

- ▶ YouTube: Simplilearn ChatGPT tutorial for beginners · ▶ YouTube: Simplilearn AI tools for data analysis
- ▶ YouTube: Claude AI tutorial for data analysis · ▶ YouTube: Perplexity AI tutorial for research · ▶ YouTube: NotebookLM tutorial
- ▶ YouTube: Lovable AI tutorial build app · ▶ YouTube: Microsoft Copilot in Excel tutorial · ▶ YouTube: GitHub Copilot for data science
- 📘 ChatGPT · 📘 Claude · 📘 Gemini · 📘 Microsoft Copilot · 📘 Grok · 📘 Perplexity
- 📘 Lovable · 📘 NotebookLM · 📘 Cursor · 📘 GitHub Copilot · 📘 Julius AI
- 📘 CompareGen: AI tools for data analysis 2026

LEARNING SESSION S9

🎯 Interview Questions & Answers

68 questions with model answers: HR, Excel, SQL, statistics, Python, Power BI/Tableau, ML, GenAI, case studies and guesstimates

Learning session · for Data Analyst, Business Analyst, BI Developer and junior Data Scientist roles

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s9**

How interviews work



A typical analytics hiring process.

- **Resume**: one page; each bullet = action + tool + measurable result ("Built a Power BI dashboard tracking 12 KPIs, cutting weekly reporting from 6 hours to 30 minutes").
- **Portfolio**: 3 projects on GitHub/Tableau Public/Netlify: one SQL, one Python, one dashboard, each with a business question, approach, and result.
- **Technical round**: live SQL (most common), Excel tasks, Python/pandas, statistics questions.
- **Case round**: business problem → metrics → analysis plan → recommendation.
- **Behavioural**: STAR method (Situation, Task, Action, Result) with numbers.

HR & behavioural

Q Tell me about yourself.

Present → past → future in 60–90 seconds: "I'm a [role] with [x] years in [domain]. Recently I [project with a number]. Before that I [relevant experience]. I'm now moving into data analytics because [reason], and I've built [projects/certification]. This role fits because [company link]."

Q Why data analytics?

Connect a real moment: "In operations I saw decisions made on gut feel; when I built a simple Excel tracker, call volume forecasting improved and we staffed better. I want to do that full-time with stronger tools."

Q Describe a time you found an insight that changed a decision.

STAR: Situation (rising complaints), Task (find the cause), Action (pivot by issue + Pareto showed 3 issues = 70%), Result (process fix reduced complaints 25% in a quarter).

Q How do you handle a stakeholder who disagrees with your numbers?

Walk through definitions and sources together, reconcile differences on a small sample, document the agreed definition, and fix the metric dictionary.

Q How do you prioritise multiple urgent requests?

Clarify business impact and deadlines, agree priority with the manager, communicate timelines, and automate recurring requests.

Q What is your biggest weakness?

A real, improving one: "I used to over-polish reports; now I share a draft early to confirm direction, then refine."

Excel

Q VLOOKUP vs INDEX/MATCH vs XLOOKUP?

VLOOKUP: right-only, column-number fragile. INDEX/MATCH: any direction, robust. XLOOKUP: any direction, built-in not-found and match modes, returns arrays; Excel 365/2021.

Q How do you remove duplicates while keeping the latest record?

Sort by date descending, then Data → Remove Duplicates on the key; or Power Query: sort, then Remove Duplicates (keeps first).

Q Count unique customers who bought in Pune.

`=COUNTA(UNIQUE(FILTER(Cust, City="Pune")))` (365) or a pivot with "Add to Data Model" → Distinct Count.

Q What is a pivot table's "Show Values As"?

Displays values as % of total/row/column, difference from previous, running total, rank, etc., without formulas.

Q Explain absolute vs relative references with an example.

`=B2*$F$1` copied down multiplies each row's B by the fixed rate in F1.

Q How do you handle #N/A from lookups?

Find the cause (trailing spaces, number stored as text, missing key), clean with TRIM/VALUE, and show a friendly value with IFNA or XLOOKUP's if_not_found.

Q What is Power Query and why use it?

Excel's ETL tool: records cleaning steps once and replays them on refresh; combines files and sources.

Q SUMIFS with an OR condition?

`=SUM(SUMIFS(Amt, City, {"Pune","Mumbai"}))`.

Q How do you build a dynamic chart?

Base it on a Table or a FILTER/INDEX range driven by a drop-down; the chart updates automatically.

Q Goal Seek vs Solver?

Goal Seek changes one input to hit one target; Solver optimises an objective with multiple variables and constraints.

SQL

Q Order of execution of a SELECT?

FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → TOP/OFFSET.

Q Find the 3rd highest salary in each department.

```
SELECT * FROM (SELECT dept, name, salary, DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary
DESC) r FROM Emp) t WHERE r = 3;
```

Q Customers who ordered in January but not February.

```
SELECT customer_id FROM Orders WHERE order_date >= '2026-01-01' AND order_date < '2026-02-01'
EXCEPT
SELECT customer_id FROM Orders WHERE order_date >= '2026-02-01' AND order_date < '2026-03-01';
```

Q Running total of sales by date.

```
SELECT sale_date, amount, SUM(amount) OVER (ORDER BY sale_date ROWS UNBOUNDED PRECEDING) AS
running_total FROM Sales;
```

Q Month-over-month growth.

```
WITH m AS (SELECT DATETRUNC(MONTH, order_date) mth, SUM(amount) rev FROM Orders GROUP BY
DATETRUNC(MONTH, order_date))
SELECT mth, rev, 100.0*(rev - LAG(rev) OVER (ORDER BY mth)) / NULLIF(LAG(rev) OVER (ORDER BY
mth),0) AS mom_pct FROM m;
```

Q Delete duplicates keeping one row.

```
WITH d AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) rn FROM Users) DELETE
FROM d WHERE rn > 1;
```

Q Employees earning more than their manager.

```
SELECT e.name FROM Emp e JOIN Emp m ON e.manager_id = m.emp_id WHERE e.salary > m.salary;
```

Q Percentage of total revenue by category.

```
SELECT category, SUM(amount) rev, 100.0*SUM(amount)/SUM(SUM(amount)) OVER () AS pct FROM Sales
GROUP BY category;
```

Q Users active 3 consecutive days.

```
WITH d AS (SELECT DISTINCT user_id, CAST(login_time AS DATE) dt FROM Logins),
g AS (SELECT user_id, dt, DATEADD(DAY, -ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY dt),
dt) grp FROM d)
SELECT DISTINCT user_id FROM g GROUP BY user_id, grp HAVING COUNT(*) >= 3;
```

Q INNER vs LEFT join row counts: when can LEFT return more rows than the left table?

When the right table has multiple matches per key (one-to-many), each left row repeats for every match.

Q CTE vs subquery vs temp table vs view?

CTE: readable, one statement. Subquery: inline. Temp table: stored for the session, indexable. View: saved query object in the database.

Q What is normalisation?

Organising tables so each fact is stored once (1NF, 2NF, 3NF), reducing redundancy and update errors.

Statistics

Q Mean vs median: when to use which?

Median for skewed data or outliers (income, delivery time); mean for symmetric data.

Q What is a p-value?

The probability of results at least this extreme if H0 were true. Below α (0.05) → reject H0. It is not the probability H0 is true.

Q Type I vs Type II error?

Type I: false positive (reject a true H0). Type II: false negative (miss a real effect).

Q Explain a 95% confidence interval.

A range built so that 95% of intervals made the same way would contain the true value.

Q What is the Central Limit Theorem?

Sample means from large enough samples are approximately normal, centred on the population mean with standard error σ/√n.

Q How would you design an A/B test for a new checkout page?

Define the primary metric (conversion) and guardrails (AOV, errors); compute sample size for the minimum detectable effect; randomise users 50/50; run full weeks; test with a two-proportion z-test; check for novelty effects.

Q Correlation vs causation?

Correlation is association; causation needs experiments or causal methods to rule out confounders.

Q Which test compares average spend across 4 regions?

One-way ANOVA (Kruskal–Wallis if data is very skewed/small).

Q What is standard deviation in plain words?

The typical distance of values from the average.

Q Simpson's paradox?

A trend in combined data reverses within subgroups because of a confounding variable (e.g. overall conversion falls while every channel improves, due to a mix shift).

Python

Q loc vs iloc?

loc by labels/boolean masks (end inclusive); iloc by integer positions (end exclusive).

Q apply vs vectorisation?

Vectorised column operations are much faster; use apply only when logic can't be vectorised.

Q How do you merge two DataFrames like VLOOKUP?

`df.merge(lookup, on="key", how="left")` ; check row counts before and after.

Q Group and aggregate multiple metrics.

```
df.groupby("city").agg(orders=("id","nunique"), revenue=("amount","sum"))
```

Q Handle outliers?

Detect with IQR or z-score; investigate; cap (clip), transform (log) or remove with justification.

Q What is a lambda?

A small anonymous one-line function, e.g. `lambda x: x * 1.18`.

Q Difference between a list and a NumPy array?

Arrays hold one data type and support fast vectorised maths; lists hold anything and need loops.

Q How do you read only some columns of a big CSV?

`pd.read_csv(f, usecols=[...], dtype={...}, chunksize=100000)`.

Power BI & Tableau

Q Measure vs calculated column (Power BI)?

Columns are stored per row at refresh; measures compute at query time in filter context.

Q What does CALCULATE do?

Evaluates an expression with modified filter context; the core of DAX.

Q Star schema?

Fact table linked to dimension tables with one-to-many relationships; simple, fast, predictable.

Q Row-level security?

Roles with DAX filters (static or USERPRINCIPALNAME-based) assigned to users in the service.

Q Import vs DirectQuery?

Import caches data (fast, needs refresh); DirectQuery queries live (current, slower).

Q Tableau LOD expressions?

FIXED/INCLUDE/EXCLUDE set the aggregation level independent of the view.

Q Tableau context filter?

A filter applied before Top N and FIXED LODs, also improving performance.

Q How do you design a dashboard for executives?

Start from their decisions; 4–6 KPIs with targets and trends at the top; drill-downs below; one message per page; consistent colours; mobile layout.

ML & GenAI

Q Supervised vs unsupervised learning?

Supervised uses labelled outcomes (predict churn); unsupervised finds structure without labels (segments).

Q Bias–variance trade-off?

Simple models underfit (high bias); complex models overfit (high variance); choose complexity by validation error.

Q Precision vs recall?

Precision: of predicted positives, share correct. Recall: of actual positives, share found.

Q How do you handle class imbalance?

Stratified splits, class weights, resampling (SMOTE), threshold tuning, and metrics like recall, F1, PR-AUC.

Q Why scale features?

Distance-based (KNN, k-means) and gradient-based models are sensitive to feature scale.

Q Random forest vs gradient boosting?

RF averages independent trees (robust, less tuning); boosting builds trees sequentially to fix errors (often more accurate, needs tuning).

Q What is cross-validation?

Splitting training data into k folds, training on k–1 and validating on 1, rotating, to estimate performance reliably.

Q What is an LLM hallucination and how do you reduce it?

A confident false output. Reduce with RAG/grounding, asking for sources, lower temperature, and verification.

Q What is RAG?

Retrieve relevant documents by embedding similarity and give them to the LLM to answer from.

Q How have you used GenAI in analytics?

Example: classify 10,000 free-text complaints into themes with an LLM, validate on a 300-row hand-labelled sample (92% agreement), then trend themes in Power BI.

Case & guesstimates

Case framework

1. Clarify the goal and the metric (what does "success" mean?).
2. Break the metric into drivers (revenue = users × conversion × AOV).
3. Hypothesise causes; say which data would confirm each.
4. Analyse by segments: time, channel, region, product, new vs repeat.
5. Recommend, quantify impact, mention risks and next steps.

Q Sales dropped 15% last month. How do you investigate?

Confirm data correctness; check seasonality/YoY; decompose into traffic × conversion × AOV; segment by region, channel, category, new vs returning; check external events (competitor sale, stock-outs, app outage); find the segment explaining most of the drop; recommend actions.

Q Which metrics would you track for a food-delivery app?

North star: orders delivered per week. Drivers: active users, order frequency, conversion, AOV, delivery time, cancellation rate, restaurant availability, rating; guardrails: refunds, delivery partner earnings.

Q Guesstimate: daily UPI transactions in a city of 1 crore people.

Adults with smartphones ~60% → 60 lakh; UPI users ~70% → 42 lakh; average 1.5 transactions/day → ~63 lakh transactions/day. State assumptions clearly; the structure matters more than the number.

Q How would you measure whether a new feature worked?

A/B test with a pre-defined primary metric and guardrails; if not possible, difference-in-differences against a comparable group.

Resources

- ▶ YouTube: Simplilearn data analyst interview questions and answers · ▶ YouTube: Simplilearn SQL interview questions · ▶ YouTube: Simplilearn Power BI interview questions
- ▶ YouTube: techTFQ SQL interview questions · ▶ YouTube: Alex The Analyst data analyst interview
- 📘 DataLemur (SQL and statistics interview practice) · 📘 StrataScratch · 📘 LeetCode SQL 50
- Course-specific Q&A also sits in the Interview Q&A tab of every module and session.

LEARNING SESSION S10

🔍 Reverse Engineering for Non-Tech Learners

Learn fast by taking finished work apart: dashboards, Excel models, SQL, notebooks, ML models, metrics and AI prompts

Learning session · a skill-building method used by analysts to understand inherited work and learn from good examples

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s10**

The method

Reverse engineering means starting from a finished result and working backwards to understand how it was built. For a non-technical learner it is the fastest way to learn: instead of a blank page, you study a working example, rebuild a small version, and compare.



Six steps that work for any artefact.

Step	Questions to ask
1 Observe	What question does this answer? Who uses it? Which numbers matter most?
2 Decompose	What are the parts (pages, sheets, CTEs, cells, features)? Which part feeds which?
3 Trace	For one number: which data, which filters, which formula produced it?
4 Rebuild	Recreate one part on a small sample yourself
5 Compare	Do your numbers match? If not, what is different (filter, join, definition)?
6 Document & improve	Write the logic in plain English; note risks and improvements

> Only reverse-engineer work you are allowed to access: your company's files, public examples (Tableau Public, Kaggle notebooks, GitHub), or your own course material. Respect licences, terms of service and confidentiality.

Excel workbooks

- **Show Formulas** (`Ctrl` + `` ` ``) to see all logic at once.
- **Trace Precedents / Dependents** (Formulas tab) draws arrows from inputs to outputs; **Evaluate Formula** steps through a formula one calculation at a time.
- **Name Manager** lists named ranges and LAMBDA functions; **Go To Special → Formulas / Constants** highlights hard-coded numbers hiding among formulas.

- Data → Queries & Connections shows Power Query steps; Data → Relationships / Power Pivot shows the data model.
- **Inquire** add-in (Workbook Analysis, Compare Files) or Copilot's "Explain this formula".

Exercise

Take a finished dashboard workbook (e.g. a free Excel dashboard template). Pick the headline KPI, trace it back to raw data, write the calculation in one sentence, then rebuild it on a new sheet with your own formula and check it matches.

SQL queries

1. Format the query (SSMS: Edit → Advanced; or an online formatter) so each clause is on its own line.
2. Read in **execution order**: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.
3. For nested queries, read the **innermost** subquery first; for CTEs read top to bottom and run each CTE alone: `WITH a AS (...) SELECT * FROM a;`
4. Check row counts after each join to spot duplication.
5. Replace the final SELECT with `SELECT TOP 20 *` at each stage to see intermediate data.
6. Look at the execution plan (`Ctrl` + `M`) to understand how tables are combined.
7. Ask an AI assistant: "Explain this query line by line for a beginner and describe the business question it answers."

Dashboards

Power BI

- Model view: tables, relationships, direction.
- Select a visual → see fields in the Build pane; check visual/page/report filters.
- Right-click a measure → see its DAX; use the DAX query view to evaluate it alone.
- Performance Analyzer → Copy query shows the exact DAX each visual runs.
- Open a .pbip project to read the model as TMDL text.

Tableau

- On Tableau Public, many authors allow "Download" of the .twbx: open each sheet and look at shelves, the Marks card and filters.
- Right-click calculated fields → Edit to read formulas; check Compute Using on table calcs.
- Dashboard → Actions shows interactivity.

Exercise

Pick a Tableau Public "Viz of the Day". Identify its chart types, the calculated fields it must need, and rebuild one chart with a similar public dataset.

Notebooks & ML models

Python notebooks (Kaggle, GitHub)

- Run cell by cell; after each, print `df.shape`, `df.head()` to see what changed.
- Write a one-line comment above each cell in your own words.
- Change one parameter at a time and observe the effect.

ML models

What does it predict?	Find the target column and the business decision it supports
What drives it?	Feature importance, coefficients, SHAP summary plots
How does it behave?	Score a few hand-made examples ("what if income doubles?") – sensitivity analysis
How good is it?	Recompute the confusion matrix on a hold-out sample
Is it fair?	Compare error rates across groups

Metrics, apps & AI

Reverse-engineer a business metric

Build a **metric tree**: Revenue = Orders × AOV; Orders = Visitors × Conversion; AOV = Items per order × Price per item. When a number moves, walk down the tree to find which branch moved.

Reverse-engineer a product (case-study thinking)

Look at a feature (e.g. "Frequently bought together") and ask: what data does it need (order history), what algorithm (association rules), what metric would prove it works (attach rate, AOV)? This is how product and data interview cases are answered.

Reverse prompting (AI)

Give an AI an output you like (a report, an email, a chart description) and ask: "Write the prompt that would produce this." Save the resulting prompt as a template. Covered in **Generative AI → PT4 Reverse prompting**.

Public web data (responsibly)

Browser developer tools (F12 → Network) show the data a public page loads (often JSON). Use official APIs or downloads where provided, follow the site's terms, and never collect personal data.

Resources

- **▶ YouTube: Excel trace precedents evaluate formula tutorial · ▶ YouTube: how to read complex SQL queries**

- ▶ YouTube: Power BI performance analyzer tutorial · ▶ YouTube: Tableau Public download workbook learn
- ▶ YouTube: SHAP values explained simply
- 📘 Tableau Public Viz of the Day · 📘 Kaggle Code (public notebooks)

LEARNING SESSION S11

⚙️ Automation on Data

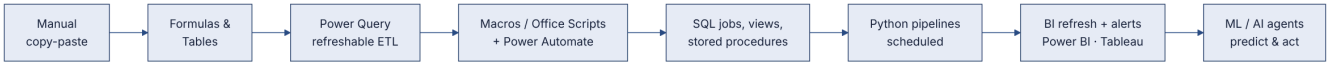
Automate the boring work with Excel, SQL, Python, Power BI, Tableau and ML/AI, with a complete weekly-report example

Additional chapter · from manual copy-paste to scheduled, self-refreshing pipelines

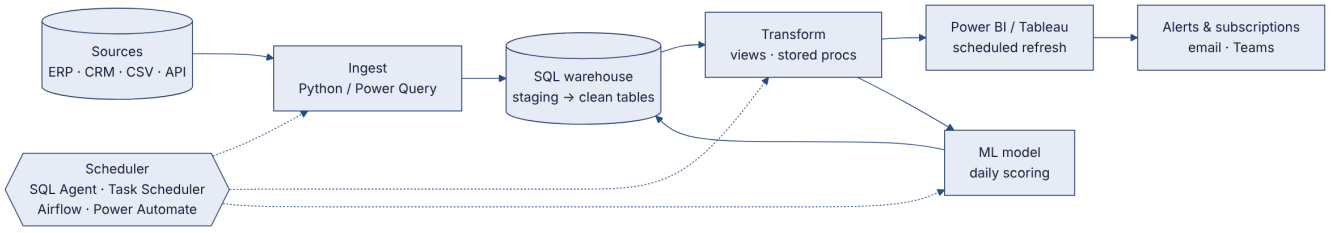
Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s11**

Why & how

Automation turns a report you rebuild every week into one that rebuilds itself. It saves hours, removes copy-paste errors, and frees you to analyse instead of assemble.



The automation ladder: climb one rung at a time.



A typical automated analytics pipeline with a scheduler driving each step.

What to automate first

Signal	Example
Repeated on a schedule	Weekly MIS, month-end reconciliation
Same steps every time	Download → clean → pivot → paste into PPT
Error-prone	Manual VLOOKUPs across 10 files
Time-consuming	More than 30 minutes per run

Tip ROI check: (minutes saved per run × runs per year) vs build time. A 2-hour weekly task automated in 8 hours pays back in a month.

Excel

Technique	Automates	How
Tables + formulas	Calculations on new rows	Ctrl+T; structured references

Technique	Automates	How
Power Query	Import, clean, combine	Get Data → From Folder; Refresh All; set "Refresh on open"
Pivot auto-refresh	Summaries	PivotTable Options → Refresh data when opening the file; auto-refresh (365)
Macros / VBA	Formatting, exporting PDFs, emailing	Record a macro; edit in Alt+F11
Office Scripts + Power Automate	Cloud runs on a schedule or trigger	Automate tab → Record; Power Automate "Run script" on a recurrence
Copilot Agent Mode	Multi-step builds from a prompt	"Refresh, update the summary and highlight anomalies"

```
' VBA: save every sheet as a separate PDF
Sub ExportSheetsToPDF()
    Dim ws As Worksheet
    For Each ws In ThisWorkbook.Worksheets
        ws.ExportAsFixedFormat Type:=xlTypePDF, _
            Filename:=ThisWorkbook.Path & "\" & ws.Name & ".pdf"
    Next ws
End Sub
```

SQL

```
-- 1. A view: the cleaned, reusable dataset every report reads
CREATE VIEW rpt.vw_weekly_sales AS
SELECT DATETRUNC(WEEK, o.order_date) AS week_start, c.city, p.category,
       SUM(oi.qty * oi.price) AS revenue, COUNT(DISTINCT o.order_id) AS orders
FROM Orders o JOIN Customer c ON o.customer_id = c.customer_id
JOIN Order_Item oi ON o.order_id = oi.order_id JOIN Product p ON oi.product_id = p.product_id
GROUP BY DATETRUNC(WEEK, o.order_date), c.city, p.category;

-- 2. A stored procedure that loads a summary table
CREATE PROCEDURE rpt.usp_load_weekly AS
BEGIN
    TRUNCATE TABLE rpt.weekly_sales;
    INSERT INTO rpt.weekly_sales SELECT * FROM rpt.vw_weekly_sales;
END;
-- 3. Schedule it: SQL Server Agent → Jobs → New Job → Step: EXEC rpt.usp_load_weekly → Schedule:
Mondays 06:00
--    (Express edition has no Agent: use Windows Task Scheduler with sqlcmd, or Azure Data Factory
/ Fabric pipelines)
```

Also: triggers (react to inserts/updates, e.g. audit logs), `MERGE` for incremental upserts, Database Mail for alert emails.

Python

```python
# weekly_report.py — pull data, build Excel report, email it
import pandas as pd, sqlalchemy as sa, smtplib, datetime as dt
from email.message import EmailMessage

engine = sa.create_engine("mssql+pyodbc://@localhost/RetailDB?
driver=ODBC+Driver+18+for+SQL+Server&trusted_connection=yes&TrustServerCertificate=yes")
df = pd.read_sql("SELECT * FROM rpt.weekly_sales WHERE week_start >= DATEADD(WEEK, -8, GETDATE())",
engine)

summary = df.pivot_table(index="city", columns="week_start", values="revenue",
aggfunc="sum").round(0)
out = f"weekly_sales_{dt.date.today():%Y%m%d}.xlsx"
with pd.ExcelWriter(out) as xw:
    summary.to_excel(xw, sheet_name="Summary")
    df.to_excel(xw, sheet_name="Data", index=False)

msg = EmailMessage(); msg["Subject"] = "Weekly sales report"; msg["From"] = "reports@company.com";
msg["To"] = "team@company.com"
msg.set_content("Attached: last 8 weeks by city.")
msg.add_attachment(open(out, "rb").read(), maintype="application", subtype="octet-stream",
filename=out)
with smtplib.SMTP("smtp.company.com", 587) as s:
    s.starttls(); s.login("reports@company.com", "USE_A_SECRET_STORE"); s.send_message(msg)
```

Schedule with	How
Windows Task Scheduler	Create Basic Task → Weekly → Start a program: `python C:\jobs\weekly_report.py`
cron (Linux/Mac)	`0 7 * * 1 /usr/bin/python3 /jobs/weekly_report.py` (Mondays 07:00)
Cloud	GitHub Actions schedule, Azure Functions, AWS Lambda, Fabric notebooks
Orchestrators	Apache Airflow, Prefect, Dagster for multi-step pipelines with retries and monitoring

> Never hard-code passwords in scripts; use environment variables or a secrets manager.

Power BI & Tableau

Power BI

- **Scheduled refresh** of semantic models (gateway for on-premises data); **incremental refresh** for large tables.
- **Subscriptions**: email report snapshots on a schedule; **data alerts** on dashboard tiles.
- **Power Automate**: trigger flows from an alert, a button in the report, or after refresh ("refresh dataset then post to Teams").
- **Fabric**: Data Factory pipelines and notebooks feed Lakehouse tables; Direct Lake reports update without import refresh.
- **Copilot** summaries for automated narrative.

Tableau

- **Extract refresh schedules** on Tableau Cloud/Server.
- **Tableau Prep** flows scheduled with Prep Conductor for repeatable cleaning.
- **Subscriptions** and **data-driven alerts**.
- **Tableau Pulse** pushes metric changes with AI-written insights to email or Slack.

ML & AI

Automation	Example	Tools
Batch scoring	Score all customers for churn every Sunday; write scores to SQL for the CRM and Power BI	Python script + scheduler; Azure ML / Fabric
Retraining triggers	Retrain when accuracy drops or data drifts	MLflow, Azure ML pipelines
Anomaly alerts	Flag days when sales fall outside expected range	Prophet/ETS forecast + rule; Power BI anomaly detection
LLM text classification	Tag 10,000 complaints by theme nightly	OpenAI/Anthropic API, Excel `=COPILOT()` for small sets
AI agents	"Every Monday: query sales, update dashboard, write commentary, ask for approval, email"	Copilot Studio, ChatGPT/Claude agents, n8n/Power Automate with AI steps

```python
# Nightly: classify new complaints with an LLM (pattern; use your company-approved provider)
import pandas as pd
from anthropic import Anthropic          # or the OpenAI / Azure OpenAI SDK
client = Anthropic()                     # API key from environment variable
themes = ["Delivery delay", "Refund", "Product quality", "App issue", "Other"]
def classify(text):
    r = client.messages.create(model="claude-haiku-4-5-20251001", max_tokens=10,
        messages=[{"role": "user", "content": f"Classify into one of {themes}. Reply with the label only.\n\n{text}"}])
    return r.content[0].text.strip()
new = pd.read_csv("complaints_today.csv")
new["theme"] = new["complaint_text"].map(classify)
new.to_csv("complaints_tagged.csv", index=False)
```

> **Tip** Validate AI labels on a hand-checked sample (say 200 rows) before trusting them, and re-check monthly.

End-to-end example

Weekly sales review, fully automated

1. **06:00 Mon** – SQL Agent runs `usp_load_weekly`: cleaned weekly table updated.

2. **06:15** – Python scores churn risk for active customers and writes scores back to SQL.
3. **06:30** – Power BI scheduled refresh; Copilot summary updated.
4. **06:45** – Power Automate checks KPIs; if revenue is down more than 10% week on week, posts an alert in Teams with the top 3 contributing cities.
5. **07:00** – Subscription emails the report PDF to managers; analyst reviews exceptions only.

Before: 4 hours every Monday of downloads, VLOOKUPs and copy-paste. **After:** 15 minutes reviewing exceptions.

Resources

- ▶ YouTube: Simplilearn Excel VBA tutorial · ▶ YouTube: Simplilearn Power Automate tutorial · ▶ YouTube: Simplilearn Python automation
- ▶ YouTube: Leila Gharani Office Scripts Power Automate Excel · ▶ YouTube: SQL Server Agent job schedule tutorial
- ▶ YouTube: Power BI scheduled refresh gateway tutorial · ▶ YouTube: Tableau Prep Conductor schedule flow
- 📘 Automate the Boring Stuff with Python · 📘 Power Automate documentation
- 📘 Office Scripts in Excel · 📘 Apache Airflow

LEARNING SESSION S12

🗣️ Data Storytelling, Portfolio & Career

Turn analysis into decisions, build a portfolio on GitHub/Tableau Public/Netlify, and plan your data career

Additional chapter · fills gaps common in technical courses: communication, portfolio, Git basics, career plan

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s12**

Data storytelling

An analysis only matters if someone acts on it. Storytelling is how you get from "here are 20 charts" to "do this, because of that".

The structure: Context → Insight → Action

Context	What question, for whom, and why now? "Monthly churn rose from 2.1% to 2.9%."
Insight	The one finding that explains it: "80% of the increase is prepaid users in 3 cities after the price change."
Action	A specific recommendation with expected impact: "Offer a ₹49 top-up pack in those cities; expected to save ~4,000 customers a month."

Chart and slide rules

✔ Title = the takeaway, not the topic ("South drives 60% of growth", not "Sales by region")
✔ One message per chart; remove gridlines, 3D and legends you can replace with labels
✔ Highlight the key bar/line in colour; everything else grey
✔ Start bar axes at zero; label units (₹ lakh, %)
✔ Executive summary first (pyramid principle); appendix for details
✔ Say how confident you are and what you assumed

Common mistakes

Too many numbers; no recommendation; pie charts with 10 slices; dual axes that mislead; mixing definitions (gross vs net revenue); presenting averages without spread.

Portfolio

Three projects that get interviews

Project	Skills shown	Example
SQL analysis	Joins, CTEs, window functions	E-commerce cohort retention and RFM in SQL with a README of findings
Python end-to-end	Cleaning, EDA, stats test or ML model	Churn prediction with a clear business recommendation

Project	Skills shown	Example
Dashboard	Power BI or Tableau design, DAX/LOD	Sales performance dashboard on Tableau Public / published Power BI

README template for each project

```
# Project title
Business question: ...
Data: source, size, period
Approach: cleaning → analysis → model/dashboard (tools)
Key findings: 3 bullets with numbers
Recommendation and impact: ...
Links: dashboard / notebook / slides
```

Where to publish

- **GitHub**: code, notebooks, READMEs (this handbook's repository is an example).
- **Tableau Public** / Power BI publish-to-web (public data only).
- **Netlify**: free hosting for an HTML portfolio or project pages (drag-and-drop a folder at app.netlify.com/drop).
- **LinkedIn**: a short post per project with one chart and the insight.

Git & GitHub basics

```
git clone https://github.com/<you>/data-science-and-ai.git     # copy a repository
cd data-science-and-ai
git status                          # what changed
git add .                           # stage changes
git commit -m "Add SQL cohort project"
git push                            # upload to GitHub
git pull                            # get latest changes
```

No command line? Use **GitHub Desktop** or upload files on github.com (Add file → Upload files). Turn on **GitHub Pages** (Settings → Pages) to host an HTML site directly from the repository.

Career roadmap

Role	Core skills	Typical first 90 days
Data Analyst	Excel, SQL, Power BI/Tableau, statistics, storytelling	Own a recurring report; automate it; deliver one insight
Business Analyst	Requirements, process mapping, SQL, Excel, dashboards, stakeholder management	Map a process, define KPIs, build a tracker
BI Developer	Data modelling, DAX/LOD, SQL, ETL, performance	Build a governed semantic model and dashboards

Role	Core skills	Typical first 90 days
Data Scientist	Python, statistics, ML, experimentation, MLOps basics	Ship a baseline model with a business metric
AI/GenAI Analyst	Prompting, RAG basics, evaluation, automation tools, Python	Automate a text-heavy workflow with validation

Certifications worth considering

Microsoft PL-300 (Power BI Data Analyst), Tableau Desktop Foundations / Certified Data Analyst, Microsoft DP-600/700 (Fabric), Google Data Analytics Certificate, AWS/Azure AI fundamentals (AI-900), plus the AnalytixLabs programme certificate.

Resources

- ▶ YouTube: Simplilearn data storytelling · ▶ YouTube: storytelling with data Cole Nussbaumer Knaflic talk
- ▶ YouTube: Alex The Analyst data analyst portfolio project · ▶ YouTube: GitHub for beginners
- 📘 Storytelling with Data (blog) · 📘 GitHub Docs: Hello World
- 📘 Netlify Drop · 📘 GitHub Pages